INSTITUT NATIONAL
DE PREVOYANCE SOCIALE

Institut National de Prévoyance Sociale du Mali

INPS Formation

Formation technique complète de l'application
Architecture, code, base de données, modules, exploitation et maintenance

Version de l'application : 1.1.0 · Laravel 13.37 · PHP 8.3+ · MySQL 8.0
Document établi le 21/10/2026 à partir du code source réel du projet

Sommaire

Introduction : comment utiliser cette formation

- Organisation
- Ce dont tu as besoin
- Conventions de lecture
- Méthode de travail conseillée

Partie 1 — Architecture générale

- 1.1 Vue d'ensemble
- 1.2 Les technologies et leur rôle
- 1.3 Comment les parties communiquent
- 1.4 L'arborescence du projet

Partie 2 — Le code de A à Z

- 2.1 Le point d'entrée : public/index.php
- 2.2 L'assemblage : bootstrap/app.php
- 2.3 La configuration : .env et config/
- 2.4 Les routes : routes/web.php
- 2.5 Les middlewares
- 2.6 La configuration globale : AppServiceProvider
- 2.7 Les contrôleurs
- 2.8 La validation des données
- 2.9 Les autorisations : permissions et policies
- 2.10 Les modèles Eloquent
- 2.11 Les services métier
- 2.12 Les vues Blade
- 2.13 Le CSS : Tailwind 4 et les composants maison
- 2.14 Le JavaScript : resources/js/app.js
- 2.15 La compilation : Vite
- 2.16 Authentification, rôles, permissions et sécurité
- 2.17 Parcours complet : du clic à la base de données

Partie 3 — La base de données

- 3.1 Les migrations : la structure de la base sous forme de code
- 3.2 Les tables et leur rôle
- 3.3 Les modèles Eloquent
- 3.4 Les seeders : les données initiales
- 3.5 Consulter, ajouter, modifier et supprimer des données
- 3.6 Commandes utiles sur la base

Partie 4 — Les fonctionnalités, module par module

- 4.1 Connexion, comptes et accueil
- 4.2 Tableau de bord
- 4.3 Référentiel du personnel et importations (module B)
- 4.4 Recensement des besoins (module C)
- 4.5 Plans annuels et circuits de validation (module D)
- 4.6 Actions de formation et sessions (module E)

4.7 Organismes et formateurs (module F)
4.8 Inscriptions et participants (module G)
4.9 Présences (module H)
4.10 Évaluations et attestations (module I)
4.11 Suivi financier (module J)
4.12 Documents (module K)
4.13 Rapports et exports (module L)
4.14 Notifications et échéances
4.15 Espace agent
4.16 Administration et paramétrage (module N)
4.17 Sauvegarde et restauration
4.18 Identité visuelle (version 1.1.0)

Partie 5 — Installation et exécution sous Windows

5.1 Les outils et leur rôle
5.2 Installation complète (nouvelle machine)
5.3 Démarrer et arrêter au quotidien
5.4 Mettre à jour le projet
5.5 Les commandes Artisan à connaître
5.6 Diagnostiquer les erreurs courantes

Partie 6 — Maintenance et évolution

6.1 Modifier une page existante
6.2 Ajouter un champ à un formulaire et à la base de données
6.3 Créer une nouvelle fonctionnalité complète
6.4 Mémo : ajouter une route, un contrôleur, un modèle, une vue
6.5 Modifier les droits d'accès
6.6 Corriger un bug : un cas réel de ce projet
6.7 Écrire et exécuter des tests
6.8 Sauvegarder et restaurer
6.9 Préparer une mise en production sécurisée
6.10 Conseil : utiliser Git

Partie 7 — La formation vidéo

7.1 Ce qui a été produit
7.2 Les épisodes
7.3 Réenregistrer les vidéos avec ta propre voix

Glossaire

Annexes

Annexe A — Liste complète des routes
Annexe B — Matrice des permissions par rôle
Annexe C — Catalogue des tables MySQL
Annexe D — Scripts des épisodes vidéo

Introduction : comment utiliser cette formation

Cette formation explique l'application **INPS Formation** telle qu'elle existe réellement : chaque nom de fichier, de classe, de méthode, de route et de table cité ici provient du code source de la version 1.1.0. Les extraits de code ne sont pas recopiés à la main : ils sont **insérés automatiquement depuis les fichiers du projet** au moment où ce document est produit. Chaque extrait indique le fichier et les numéros de lignes, pour que tu puisses l'ouvrir dans ton éditeur et suivre.

L'objectif est que tu deviennes capable de **lire, maintenir, corriger et faire évoluer** l'application seul. On part de l'architecture générale, on descend jusqu'au code ligne par ligne, puis on remonte vers les modules métier, l'exploitation sous Windows et la maintenance.

Organisation

Partie	Contenu	Niveau
1. Architecture générale	Les technologies, leur rôle, comment elles communiquent, l'arborescence	Débutant
2. Le code de A à Z	Point d'entrée, routes, middlewares, contrôleurs, validation, policies, services, modèles, vues, CSS, JavaScript, sécurité ; un parcours complet « du clic à la base »	Débutant → intermédiaire
3. Base de données	Migrations, tables, clés, relations, seeders, modèles Eloquent, manipulation sûre des données	Intermédiaire
4. Fonctionnalités	Chaque module : fichiers, fonctionnement, données, règles, erreurs et corrections	Intermédiaire
5. Installation et exécution	Composer, npm, Vite, Artisan, `.env`, MySQL sous Windows ; diagnostic des erreurs	Débutant
6. Maintenance et évolution	Modifier une page, ajouter un champ, créer une fonctionnalité, droits, bugs, tests, sauvegardes, mise en production	Intermédiaire → avancé
7. Vidéos	Les épisodes vidéo et le kit pour les réenregistrer	—
Annexes	Toutes les routes, la matrice des permissions, le catalogue des tables	Référence

Ce dont tu as besoin

- L'application installée sur ton poste : `C:\dev\inps-formation-1.1.0\inps-formation`.
- Un éditeur de code. **Visual Studio Code** (gratuit) convient très bien : *Fichier › Ouvrir le dossier* puis choisir le dossier `inps-formation`. Installe l'extension « PHP Intelephense » pour la navigation dans le code (Ctrl + clic sur un nom de classe ouvre son fichier).
- MySQL démarré (`net start MySQL80` dans une invite de commandes administrateur) et l'application lancée (`php artisan serve`).
- Les comptes de démonstration, si tu as chargé les données fictives (`DemoSeeder`) : `demo.<rôle>@exemple.invalid` (par exemple `demo.responsable_drh@exemple.invalid`), avec le mot de passe choisi dans la variable `DEMO_PASSWORD` au moment du chargement. Ils n'existent que dans une base de démonstration et ne doivent jamais être créés en production.

Conventions de lecture

- `Classe::methode` désigne une méthode d'une classe PHP. Exemple : `BesoinService::transition` est la méthode `transition` de `app/Services/BesoinService.php`.
- Un chemin comme `app/Http/Controllers/BesoinController.php` part toujours du dossier racine du projet.
- Les commandes à taper dans l'invite de commandes Windows (cmd) sont présentées ainsi :

```
cd C:\dev\inps-formation-1.1.0\inps-formation
php artisan route:list
```

- Les encadrés « À retenir » résument l'essentiel ; les encadrés « Attention » signalent ce qu'il ne faut pas faire.

> **À retenir**
>
> Le code est rédigé en français (noms de classes, méthodes, variables, messages) pour être lisible par l'équipe de l'INPS. Seuls les mots imposés par PHP et Laravel restent en anglais (`function`, `return`, `belongsTo`, `validate`…).

Méthode de travail conseillée

1. Lis une section, puis **ouvre le fichier cité** dans VS Code et retrouve l'extrait.
2. Rejoue les exemples dans l'application (connecte-toi avec le bon rôle de démonstration).
3. Fais les exercices de la partie 6 dans une **copie** du projet, jamais directement sur la version utilisée.
4. Après toute modification, lance les tests (`php artisan test`) : ils te disent si tu as cassé quelque chose.

Partie 1 — Architecture générale

1.1 Vue d'ensemble

INPS Formation est une **application web monolithique** construite avec le framework **Laravel**. « Monolithique » signifie que tout est dans un seul projet : les pages (le *front*), la logique métier (le *back*), l'accès à la base de données, les exports PDF/Excel et les commandes d'exploitation. Il n'y a ni API séparée, ni application JavaScript séparée (pas de React, pas de Vue) : le serveur fabrique directement les pages HTML.

Le projet suit le modèle **MVC** (Modèle – Vue – Contrôleur), complété par une couche de **services métier** :

- le **contrôleur** reçoit la requête du navigateur, vérifie les droits, valide les données et appelle un service ;
- le **service** applique les règles métier (statuts autorisés, calculs, contrôles) dans une transaction ;
- le **modèle** représente une table MySQL et ses relations ;
- la **vue** (Blade) fabrique la page HTML renvoyée au navigateur.

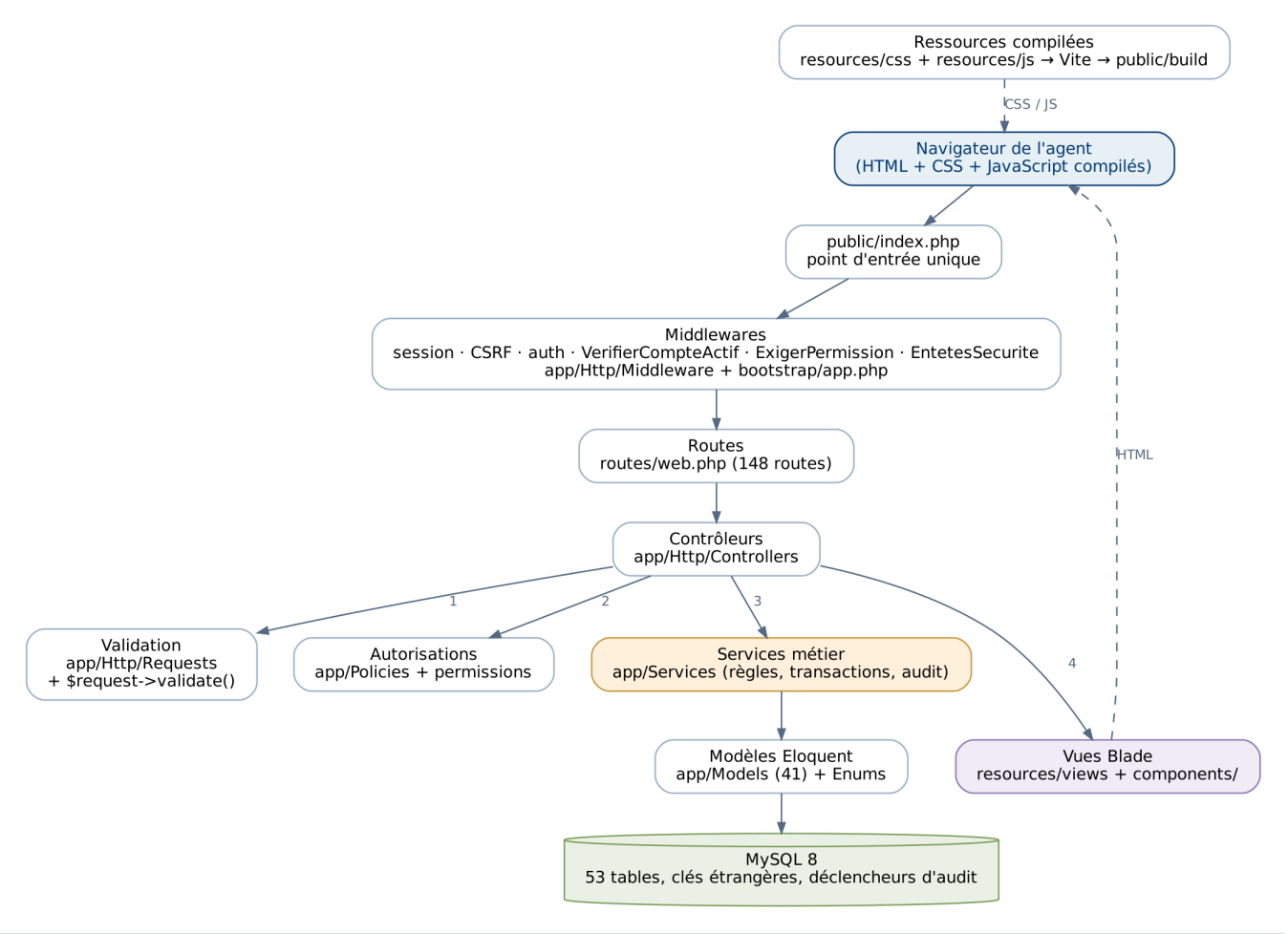


Figure 1 — Les couches de l'application et leurs dossiers.

À retenir

Règle d'or du projet : **les contrôleurs restent courts et les règles métier vivent dans `app/Services`**. Quand tu cherches « pourquoi l'application refuse telle action », ouvre d'abord le service du module (par exemple `app/Services/BesoinService.php`).

1.2 Les technologies et leur rôle

Technologie	Version dans le projet	Rôle dans INPS Formation
PHP	8.3 minimum (testé 8.3.6 ; tu utilises 8.4.6)	Le langage dans lequel tout le côté serveur est écrit.
Laravel	13.37.0 (verrouillée dans `composer.lock`)	Le framework : routage, sécurité (sessions, CSRF, mots de passe), ORM Eloquent, validation, migrations, notifications, planificateur, commandes Artisan.
MySQL	8.0	La base de données : 53 tables (dont 8 techniques propres à Laravel : `migrations`, `sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `password_reset_tokens`), clés étrangères, contraintes d'unicité, 2 déclencheurs qui protègent le journal d'audit.
Blade	(inclus dans Laravel)	Le moteur de gabarits : fichiers `*.blade.php` dans `resources/views`, mélange de HTML et de directives (`@if`, `@foreach`, `{{ }}`), composants réutilisables `<x-...>`.
Tailwind CSS	4.x	Le système de styles. Les classes utilitaires (`flex`, `gap-3`…) et les composants maison (`.btn`, `.carte`…) sont définis dans `resources/css/app.css`.
JavaScript	sans bibliothèque	Un seul fichier, `resources/js/app.js` (91 lignes) : confirmations, boîtes de dialogue, filtre de listes, menu mobile, afficher/masquer le mot de passe.
Vite	8.x	L'outil de compilation : transforme `app.css` et `app.js` en fichiers optimisés dans `public/build/`.
Composer	2.x	Le gestionnaire de dépendances PHP : installe Laravel et les bibliothèques dans `vendor/` d'après `composer.json` / `composer.lock`.
npm (Node.js)	—	Le gestionnaire de dépendances JavaScript : installe Vite, Tailwind et la police dans `node_modules/` d'après `package.json`. Nécessaire uniquement pour **recompiler** les styles.
Dompdf	3.1	Génère les PDF (plan, attestations, convocations, émargement, rapports) à partir de vues Blade (`resources/views/pdf`).
PhpSpreadsheet	5.10	Génère les exports Excel et lit les fichiers d'import (CSV, XLSX).
PHPUnit	12.5	Exécute les 102 tests automatisés (`tests/`).
Laravel Pint	1.x	Met en forme le code PHP selon un style uniforme (`vendor/bin/pint`).
Public Sans	5.3 (police, licence OFL)	La police de l'interface, embarquée localement : aucune ressource n'est chargée depuis Internet.

Les versions exactes figurent dans `composer.json` (PHP) et `package.json` (JavaScript).

1.3 Comment les parties communiquent

Toute interaction suit le même circuit. Prenons l'exemple d'un chef de service qui clique sur « Enregistrer et soumettre » dans le formulaire de besoin.

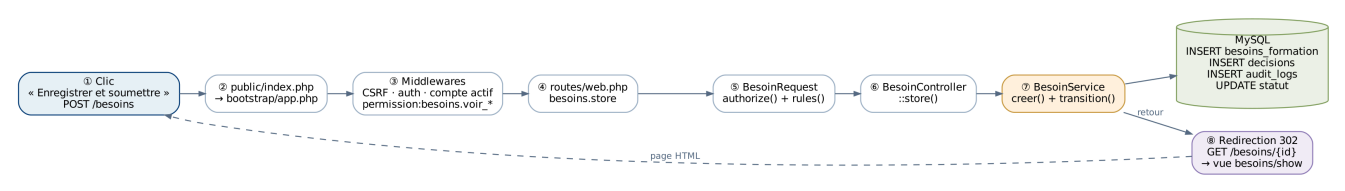


Figure 2 — Trajet d'une requête, du navigateur à MySQL et retour.

1. **Navigateur → serveur web.** Le formulaire envoie une requête HTTP `POST /besoins` contenant les champs saisis et un jeton anti-falsification (`_token`, protection CSRF).
2. **`public/index.php`** est le seul fichier PHP exposé au web. Il charge les classes (`vendor/autoload.php`) puis démarre Laravel (`bootstrap/app.php`).
3. **Middlewares.** Avant d'atteindre le code métier, la requête traverse une chaîne de filtres : session, vérification du jeton CSRF, authentification (`auth`), compte actif (`VerifierCompteActif`), permission (`ExigerPermission`), puis, au retour, ajout des en-têtes de sécurité (`EntetesSecurite`).
4. **Route.** `routes/web.php` associe `POST /besoins` à la méthode `BesoinController::store`.
5. **Validation.** La classe `BesoinRequest` vérifie les champs (année existante, service actif, longueur de l'intitulé…). En cas d'erreur, Laravel renvoie automatiquement au formulaire avec les messages.
6. **Contrôleur → service.** `BesoinController::store` appelle `BesoinService::creer` puis `BesoinService::transition`.
7. **Service → modèle → MySQL.** Le service ouvre une transaction, crée l'objet `BesoinFormation`, l'enregistre (requête `INSERT`), journalise la décision et l'audit, notifie la DRH.
8. **Réponse.** Le contrôleur renvoie une **redirection** vers la fiche ; le navigateur demande alors `GET /besoins/{id}` ; le contrôleur charge les données et la vue `resources/views/besoins/show.blade.php` produit le HTML, qui référence les fichiers CSS/JS compilés de `public/build/`.

Ce circuit est détaillé ligne par ligne dans la partie 2, avec les vraies requêtes SQL exécutées.

1.4 L'arborescence du projet

```
inps-formation/
├── app/                        ← le code de l'application (PHP)
│   ├── Console/Commands/       ← commandes « php artisan inps:… » (6 fichiers)
│   ├── Enums/                  ← listes de statuts (besoin, plan, action, session, inscription, présence,
dépense)
│   ├── Exceptions/             ← RegleMetierException : violation d'une règle métier
│   ├── Http/
│   │   ├── Controllers/        ← contrôleurs (un par écran/module) ; Admin/ pour l'administration ; Auth/
pour la connexion
│   │   ├── Middleware/         ← ExigerPermission, VerifierCompteActif, EntetesSecurite
│   │   └── Requests/           ← règles de validation des formulaires (FormRequest)
│   ├── Models/                 ← 41 modèles Eloquent (une classe par table métier)
│   ├── Notifications/          ← NotificationMetier (notification interne stockée en base)
│   ├── Policies/               ← règles d'accès objet par objet (besoins, agents, documents)
│   ├── Providers/              ← AppServiceProvider : configuration globale (mots de passe, limitations,
autorisations)
│   ├── Services/               ← la logique métier (16 services)
│   ├── Support/                ← CataloguePermissions, Identite, ProtectionAudit
│   └── helpers.php             ← fonctions d'affichage : fcfa(), date_fr(), pourcent()
├── bootstrap/app.php           ← assemblage de l'application : routes, middlewares, gestion des erreurs
├── config/                     ← configuration (lit le fichier .env) ; config/inps.php = réglages propres
```

```
au projet
├── database/
│   ├── migrations/           ← création de la structure de la base (9 fichiers)
│   ├── seeders/              ← données initiales (référentiel) et de démonstration
│   └── sql/protection_audit.sql ← déclencheurs à installer manuellement si besoin
├── docs/                     ← documentation (installation, sécurité, guides, recette…)
├── exemples/                 ← modèles de fichiers d'import (CSV)
├── public/                   ← racine web : index.php, build/ (CSS/JS compilés), images/institution/
├── resources/
│   ├── css/app.css           ← styles (Tailwind 4 + composants maison)
│   ├── js/app.js             ← comportements JavaScript
│   └── views/                ← vues Blade (par module), components/ (composants), pdf/ (gabarits PDF)
├── routes/
│   ├── web.php               ← toutes les URL de l'application (148 routes)
│   └── console.php           ← tâches planifiées (rappels quotidiens, sauvegarde)
├── storage/                  ← fichiers générés : logs/, app/private/ (documents déposés), cache des vues
├── tests/                    ← tests automatisés (Feature/ et Unit/)
├── vendor/                   ← dépendances PHP installées par Composer (ne jamais modifier)
├── node_modules/             ← dépendances JavaScript installées par npm (ne jamais modifier)
├── .env                      ← configuration locale (mots de passe, base…) — jamais partagé
├── composer.json / composer.lock ← dépendances PHP
├── package.json / package-lock.json ← dépendances JavaScript
├── phpunit.xml               ← configuration des tests (base inps_formation_test)
└── vite.config.js            ← configuration de la compilation des styles et scripts
```

Les fichiers les plus importants

Fichier	Pourquoi il compte
`routes/web.php`	La carte de l'application : chaque URL, son contrôleur et ses droits. C'est le premier fichier à ouvrir pour comprendre un écran.
`bootstrap/app.php`	Déclare les middlewares maison et la gestion centralisée des erreurs métier.
`app/Providers/AppServiceProvider.php`	Politique de mot de passe, limitation des tentatives de connexion, branchement des permissions sur le système d'autorisation de Laravel.
`app/Support/CataloguePermissions.php`	La liste des 39 permissions et la proposition des 8 rôles.
`app/Services/*.php`	Les règles métier de chaque module.
`app/Models/*.php`	La correspondance avec les tables et leurs relations.
`resources/views/components/layout.blade.php`	La coquille de toutes les pages : menu latéral (filtré selon les droits), barre du haut, messages.
`database/migrations/*.php`	La structure exacte de la base.
`.env`	Les réglages de ton installation (base, environnement, messagerie, sauvegardes).

Attention

Ne modifie jamais les dossiers `vendor/` et `node_modules/` : ils sont recréés par `composer install` et `npm ci`, toute modification serait perdue. De même, `public/build/` est régénéré par `npm run build`.

Partie 2 — Le code de A à Z

Cette partie suit l'ordre dans lequel le code est exécuté : le point d'entrée, la configuration, les routes, les middlewares, les contrôleurs, la validation, les autorisations, les services, les modèles, puis les vues, le CSS et le JavaScript. Elle se termine par un parcours complet, du clic de l'utilisateur jusqu'aux requêtes SQL réellement exécutées.

2.1 Le point d'entrée : `public/index.php`

Le serveur web (en développement : `php artisan serve` ; en production : Apache ou Nginx) envoie **toutes** les requêtes à ce fichier. C'est le fichier standard de Laravel ; il n'a pas été modifié.

`public/index.php` — lignes 1 à 20

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// Determine if the application is in maintenance mode...
if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
    require $maintenance;
}

// Register the Composer autoloader...
require __DIR__.'/../vendor/autoload.php';

// Bootstrap Laravel and handle the request...
/** @var Application $app */
$app = require_once __DIR__.'/../bootstrap/app.php';

$app->handleRequest(Request::capture());
```

Bloc par bloc :

- **Mode maintenance** : si `storage/framework/maintenance.php` existe (créé par `php artisan down`), Laravel affiche la page de maintenance et s'arrête là.
- **Chargement automatique des classes** : `vendor/autoload.php` est généré par Composer. Grâce à lui, écrire `use App\Services\BesoinService;` suffit pour que PHP trouve `app/Services/BesoinService.php` (règle « PSR-4 » déclarée dans `composer.json` : l'espace de noms `App\` correspond au dossier `app/`). C'est aussi Composer qui charge `app/helpers.php` (section `autoload.files`).
- **Démarrage** : `bootstrap/app.php` construit l'application, puis `handleRequest()` fait traverser la requête jusqu'à la réponse.

> **À retenir**
> Seul le dossier `public/` doit être accessible depuis le web. Le code (`app/`), la configuration (`.env`) et les documents déposés (`storage/app/private`) restent hors de portée des navigateurs.

2.2 L'assemblage : `bootstrap/app.php`

Ce fichier configure trois choses : les fichiers de routes, les middlewares et la gestion des erreurs.

`bootstrap/app.php` — lignes 1 à 37

```php
<?php

use App\Exceptions\RegleMetierException;
use App\Http\Middleware\EntetesSecurite;
use App\Http\Middleware\ExigerPermission;
use App\Http\Middleware\VerifierCompteActif;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Request;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->web(append: [EntetesSecurite::class, VerifierCompteActif::class]);
        $middleware->alias(['permission' => ExigerPermission::class]);
        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('accueil'));
        // Derrière un proxy inverse (HTTPS terminé en amont) : renseigner TRUSTED_PROXIES.
        $middleware->trustProxies(at: env('TRUSTED_PROXIES') ? explode(',', env('TRUSTED_PROXIES')) : []);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        // Règles métier : retour au formulaire avec un message compréhensible (gestion centralisée).
        $exceptions->render(function (RegleMetierException $e, Request $request) {
            if ($request->expectsJson()) {
                return response()->json(['message' => $e->getMessage()], 422);
            }

            return back()->withInput()->with('erreur', $e->getMessage());
        });
        $exceptions->dontReport([RegleMetierException::class]);
        $exceptions->shouldRenderJsonWhen(fn (Request $request) => $request->is('api/*') || $request-
>expectsJson());
    })->create();
```

- `withRouting` : les pages sont déclarées dans `routes/web.php`, les tâches planifiées dans `routes/console.php`. La route `/up` est un contrôle de santé fourni par Laravel (réponse 200 si l'application démarre).
- `withMiddleware` :
- `$middleware->web(append: [...])` ajoute **à toutes les pages** `EntetesSecurite` (en-têtes HTTP de sécurité) et `VerifierCompteActif` (déconnexion d'un compte désactivé, changement de mot de passe obligatoire) ;
- `alias(['permission' => ExigerPermission::class])` crée le raccourci `permission:` utilisé dans les routes ;
- `redirectGuestsTo` : un visiteur non connecté qui demande une page protégée est envoyé vers la route `login` ;
- `redirectUsersTo` : un utilisateur déjà connecté qui ouvre la page de connexion est envoyé vers `accueil` ;
- `trustProxies` : utile en production si un proxy inverse termine le HTTPS (variable `TRUSTED_PROXIES`).
- `withExceptions` : c'est ici qu'est gérée **de façon centralisée** l'exception `RegleMetierException`. Quand un service refuse une action (« Cette demande ne peut plus être modifiée… »), il lance cette exception ; Laravel renvoie alors l'utilisateur sur la page précédente (`back()`), conserve sa saisie (`withInput()`) et affiche le message (`with('erreur', …)`). C'est pourquoi les contrôleurs n'ont presque jamais de `try/catch`. `dontReport` évite d'écrire ces refus normaux dans les journaux d'erreurs.

2.3 La configuration : `.env` et `config/`

Laravel lit les réglages dans le fichier **`.env`** (un par installation, jamais partagé ni versionné), via les fichiers PHP du dossier `config/`. Exemple : `config/database.php` lit `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`.

Les réglages propres à INPS Formation sont dans `config/inps.php` :

`config/inps.php` — lignes 1 à 25

```php
<?php

/*
| Configuration technique propre à l'application (les règles métier sont dans la table
| « parametres », modifiables depuis l'écran Paramétrage).
*/
return [
    // Force HTTPS (redirections, URL générées, cookies sécurisés) — obligatoire en production.
    'forcer_https' => env('FORCE_HTTPS', env('APP_ENV') === 'production'),

    'sauvegarde' => [
        'repertoire' => env('BACKUP_PATH', storage_path('app/sauvegardes')),
        'mot_de_passe' => env('BACKUP_PASSWORD'), // chiffrement AES-256 de l'archive si défini
        'retention' => (int) env('BACKUP_RETENTION', 14),
        'mysqldump' => env('MYSQLDUMP_PATH', 'mysqldump'),
        'mysql' => env('MYSQL_CLIENT_PATH', 'mysql'),
    ],

    'securite' => [
        'tentatives_connexion' => (int) env('LOGIN_MAX_ATTEMPTS', 5), // par minute et par couple e-
mail/IP
        'mot_de_passe_longueur' => (int) env('PASSWORD_MIN_LENGTH', 12),
        'mot_de_passe_compromis' => (bool) env('PASSWORD_CHECK_PWNED', false), // nécessite un accès
Internet
        'inactivite_minutes' => (int) env('SESSION_LIFETIME', 30),
    ],
];
```

Dans le code, on lit ces valeurs avec `config('inps.securite.tentatives_connexion')`, jamais directement avec `env()` (sauf dans les fichiers de `config/`). Raison : en production, `php artisan config:cache` regroupe toute la configuration dans un fichier en cache et `env()` n'est plus relu.

Il existe un **second niveau** de réglages, modifiable sans toucher au serveur : la table `parametres` (écran *Administration › Paramètres*). Elle contient les règles métier (seuil d'achèvement, poids des retards, activation des étapes…). On la lit avec `Parametre::valeur('cle', valeurParDefaut)` :

`app/Models/Parametre.php` — lignes 29 à 45

```php
public static function valeur(string $cle, mixed $defaut = null): mixed
{
    $tous = Cache::remember('parametres.tous', 300, fn () => static::query()->get(['cle', 'valeur',
'type'])
        ->mapWithKeys(fn ($p) => [$p->cle => [$p->valeur, $p->type]])->all());

    if (! array_key_exists($cle, $tous) || $tous[$cle][0] === null || $tous[$cle][0] === '') {
        return $defaut;
    }
    [$v, $type] = $tous[$cle];

    return match ($type) {
        'int' => (int) $v,
        'decimal' => (float) $v,
        'bool' => in_array($v, ['1', 'true', 'oui'], true),
        default => $v,
    };
}
```

Le résultat est gardé 5 minutes en cache (`Cache::remember('parametres.tous', 300, …)`) pour éviter une requête SQL à chaque lecture ; `Parametre::viderCache()` est appelé après chaque modification dans l'écran de paramétrage. La conversion de type (`int`, `decimal`, `bool`) se fait selon la colonne `type`.

> **Astuce**
> Règle simple pour savoir où placer un réglage : s'il dépend du **serveur** (mot de passe MySQL, HTTPS, chemin des sauvegardes), il va dans `.env` ; s'il s'agit d'une **règle de gestion** que la DRH peut vouloir changer (seuil de 80 %, délais de rappel), il va dans la table `parametres`.

2.4 Les routes : `routes/web.php`

Une **route** associe une méthode HTTP et une URL à une méthode de contrôleur. Le fichier `routes/web.php` contient les 148 routes de l'application (liste complète en annexe A). Voici sa structure, en commençant par les pages publiques :

`routes/web.php` — lignes 40 à 67

```
// Logo et photographie institutionnels (publics : affichés sur la page de connexion)
Route::get('/identite/{type}', [IdentiteController::class, 'fichier'])->whereIn('type', ['logo',
'photo'])->name('identite.fichier');

Route::middleware('guest')->group(function () {
    Route::get('/connexion', [ConnexionController::class, 'create'])->name('login');
    Route::post('/connexion', [ConnexionController::class, 'store'])->middleware('throttle:connexion')-
>name('login.store');
    Route::get('/mot-de-passe-oublie', [MotDePasseController::class, 'demande'])-
>name('password.request');
    Route::post('/mot-de-passe-oublie', [MotDePasseController::class, 'envoyerLien'])-
>middleware('throttle:reinitialisation')->name('password.email');
    Route::get('/reinitialisation/{token}', [MotDePasseController::class, 'formulaireReinitialisation'])-
>name('password.reset');
    Route::post('/reinitialisation', [MotDePasseController::class, 'reinitialiser'])-
>middleware('throttle:reinitialisation')->name('password.update');
});

Route::middleware('auth')->group(function () {
    Route::post('/deconnexion', [ConnexionController::class, 'destroy'])->name('logout');
    Route::get('/mon-mot-de-passe', [MotDePasseController::class, 'edit'])->name('mot-de-passe.edit');
    Route::put('/mon-mot-de-passe', [MotDePasseController::class, 'update'])->name('mot-de-passe.update');

    Route::get('/', [TableauBordController::class, 'accueil'])->name('accueil');
    Route::get('/tableau-de-bord', [TableauBordController::class, 'index'])-
>middleware('permission:tableau_bord.voir')->name('tableau-bord');

    Route::get('/notifications', [NotificationController::class, 'index'])->name('notifications.index');
    Route::post('/notifications/{id}/lue', [NotificationController::class, 'lue'])-
>name('notifications.lue');
    Route::post('/notifications/tout-lire', [NotificationController::class, 'toutLire'])-
>name('notifications.tout-lire');

    Route::middleware('permission:espace.personnel')->group(function () {
        Route::get('/mon-espace', [EspaceAgentController::class, 'index'])->name('espace.index');
        Route::get('/mon-espace/attestations/{attestation}', [EspaceAgentController::class,
'attestation'])->name('espace.attestation');
    });
```

Lecture :

- `Route::get('/connexion', [ConnexionController::class, 'create'])->name('login')` : une requête `GET /connexion` appelle `ConnexionController::create`. Le **nom** `login` permet de générer l'URL partout avec `route('login')` : si l'URL change un jour, aucun lien n'est à corriger.

- `Route::middleware('guest')->group(...)` : ces pages ne sont accessibles qu'aux visiteurs **non connectés**.
- `->middleware('throttle:connexion')` : limitation du nombre de tentatives (règle définie dans `AppServiceProvider`, voir 2.6).
- `Route::middleware('auth')->group(...)` : tout ce qui suit exige d'être **connecté**.
- `Route::middleware('permission:espace.personnel')->group(...)` : en plus, ces routes exigent la permission `espace.personnel`.

Le module des besoins illustre le cumul des contrôles :

`routes/web.php` — lignes 90 à 100

```
// Module C — Besoins (cloisonnement par policy)
Route::middleware('permission:besoins.voir_tous|besoins.voir_service')->group(function () {
    Route::get('/besoins', [BesoinController::class, 'index'])->name('besoins.index');
    Route::get('/besoins/creer', [BesoinController::class, 'create'])->name('besoins.create');
    Route::post('/besoins', [BesoinController::class, 'store'])->name('besoins.store');
    Route::get('/besoins/{besoin}', [BesoinController::class, 'show'])->name('besoins.show');
    Route::get('/besoins/{besoin}/modifier', [BesoinController::class, 'edit'])->name('besoins.edit');
    Route::put('/besoins/{besoin}', [BesoinController::class, 'update'])->name('besoins.update');
    Route::delete('/besoins/{besoin}', [BesoinController::class, 'destroy'])->name('besoins.destroy');
    Route::post('/besoins/{besoin}/transition', [BesoinController::class, 'transition'])-
>name('besoins.transition');
});
```

- Le groupe exige `besoins.voir_tous` **ou** `besoins.voir_service` (le `|` signifie « au moins une des deux »).
- `{besoin}` est un **paramètre de route**. Dans le contrôleur, la signature `show(Request $r, BesoinFormation $besoin)` déclenche la *liaison implicite de modèle* (*route model binding*) : Laravel exécute `SELECT * FROM besoins_formation WHERE id = …` et injecte l'objet, ou renvoie **404** s'il n'existe pas (ou s'il est supprimé logiquement, grâce à `SoftDeletes`).
- Le contrôle fin (« ce besoin appartient-il à mon service ? ») n'est **pas** fait par la route mais par la *policy*, dans le contrôleur (section 2.8).
- Les formulaires HTML ne savent envoyer que `GET` et `POST`. Pour `PUT` et `DELETE`, la vue ajoute un champ caché avec `@method('PUT')` ; Laravel le lit et choisit la bonne route.

Pour l'administration, les routes sont préfixées et nommées en groupe :

`routes/web.php` — lignes 224 à 235

```
// Administration et paramétrage (Module N, section 5)
Route::prefix('administration')->name('admin.')->group(function () {
    Route::middleware('permission:utilisateurs.gerer')->group(function () {
        Route::get('/utilisateurs', [UtilisateurController::class, 'index'])->name('utilisateurs.index');
        Route::get('/utilisateurs/creer', [UtilisateurController::class, 'create'])-
>name('utilisateurs.create');
        Route::post('/utilisateurs', [UtilisateurController::class, 'store'])->name('utilisateurs.store');
        Route::get('/utilisateurs/{utilisateur}/modifier', [UtilisateurController::class, 'edit'])-
>name('utilisateurs.edit');
        Route::put('/utilisateurs/{utilisateur}', [UtilisateurController::class, 'update'])-
>name('utilisateurs.update');
        Route::post('/utilisateurs/{utilisateur}/activation', [UtilisateurController::class,
'activation'])->name('utilisateurs.activation');
        Route::post('/utilisateurs/{utilisateur}/reinitialiser', [UtilisateurController::class,
'reinitialiserMotDePasse'])->name('utilisateurs.reinitialiser');
    });
    Route::middleware('permission:roles.gerer')->group(function () {
```

`prefix('administration')` ajoute `/administration` devant chaque URL, et `name('admin.')` ajoute `admin.` devant chaque nom : la route déclarée `utilisateurs.index` s'appelle donc `admin.utilisateurs.index` et répond à `/administration/utilisateurs`.

> **Astuce**
> Pour retrouver le contrôleur d'une page, lis l'URL dans le navigateur puis tape `php artisan route:list --path=besoins` : la commande affiche les routes correspondantes, leur nom et leur contrôleur.

Les tâches planifiées : `routes/console.php`

`routes/console.php` — lignes 1 à 12

```php
<?php

use Illuminate\Support\Facades\Schedule;

/*
| Tâches planifiées. Sur le serveur, une seule entrée cron est nécessaire :
|   * * * * * cd /var/www/inps-formation && php artisan schedule:run >> /dev/null 2>&1
| (voir docs/INSTALLATION_PRODUCTION.md ; sous Windows : Planificateur de tâches).
*/
Schedule::command('inps:echeances')->dailyAt('06:30')->withoutOverlapping();
Schedule::command('inps:sauvegarde')->dailyAt('01:30')->withoutOverlapping()->environments(['production',
'staging']);
Schedule::command('auth:clear-resets')->daily();
```

Le **planificateur** de Laravel lance chaque jour la commande `inps:echeances` (rappels) à 6 h 30 et `inps:sauvegarde` à 1 h 30 (uniquement en production ou pré-production), et purge les jetons de réinitialisation expirés. Il ne fonctionne que si `php artisan schedule:run` est exécuté **chaque minute** par le système (tâche cron sous Linux, Planificateur de tâches sous Windows).

2.5 Les middlewares

Un **middleware** est un filtre placé avant (et/ou après) le contrôleur. Il reçoit la requête, peut la refuser ou la rediriger, sinon la passe au suivant avec `$next($request)`.

`ExigerPermission` : le contrôle d'accès des routes

`app/Http/Middleware/ExigerPermission.php` — lignes 1 à 25

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

/**
 * Contrôle d'accès côté serveur : route accessible si l'utilisateur détient AU MOINS UNE
 * des permissions listées (séparées par « | »). Refus par défaut.
 * Exemple : ->middleware('permission:besoins.voir_tous|besoins.voir_service')
 */
class ExigerPermission
{
    public function handle(Request $request, Closure $next, string $permissions): Response
    {
        $user = $request->user();
        if (! $user || ! $user->aUneDesPermissions(...explode('|', $permissions))) {
            abort(403, 'Vous ne disposez pas des droits nécessaires pour accéder à cette page.');
```

```
        }

        return $next($request);
    }
}
```

- Le texte après `permission:` arrive dans `$permissions` (par exemple `"besoins.voir_tous|besoins.voir_service"`).
- `explode('|', …)` le découpe en tableau ; l'opérateur `...` passe chaque élément comme argument à `aUneDesPermissions`.
- Si l'utilisateur n'a **aucune** des permissions, `abort(403, …)` arrête tout avec le code HTTP 403. C'est le principe du **refus par défaut** : sans permission explicite, pas d'accès.

Les pages d'erreur

Le projet ne contient pas de pages d'erreur personnalisées (il n'existe pas de dossier `resources/views/errors`). Laravel affiche donc ses pages standard, minimalistes : pour une erreur 403, le code et le message passé à `abort()` (« 403 | Vous ne disposez pas des droits nécessaires… ») ; pour les autres, un libellé en anglais (« 404 | Not Found », « 419 | Page Expired », « 500 | Server Error »). Pour des pages en français aux couleurs de l'INPS, il suffirait de créer les vues `403.blade.php`, `404.blade.php`, `419.blade.php` et `500.blade.php` dans un dossier `resources/views/errors` : c'est une amélioration possible, non réalisée.

`VerifierCompteActif` : comptes désactivés et mots de passe à changer

`app/Http/Middleware/VerifierCompteActif.php` — lignes 1 à 31

```php
<?php

namespace App\Http\Middleware;

use App\Services\Audit;
use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Symfony\Component\HttpFoundation\Response;

/** Déconnecte immédiatement un compte désactivé et impose le changement de mot de passe si requis. */
class VerifierCompteActif
{
    public function handle(Request $request, Closure $next): Response
    {
        $user = $request->user();
        if ($user && ! $user->actif) {
            Audit::enregistrer('securite.session_compte_inactif', $user, 'Session interrompue : compte désactivé');
            Auth::logout();
            $request->session()->invalidate();
            $request->session()->regenerateToken();

            return redirect()->route('login')->withErrors(['email' => 'Votre compte est désactivé. Contactez l\'administrateur.']);
        }
        if ($user && $user->doit_changer_mot_de_passe && ! $request->routeIs('mot-de-passe.*', 'logout')) {
            return redirect()->route('mot-de-passe.edit')->with('avertissement', 'Vous devez définir un nouveau mot de passe avant de continuer.');
        }

        return $next($request);
```

```
    }
}
```

Ce middleware s'exécute sur toutes les pages. Si un administrateur désactive un compte pendant que la personne est connectée, sa **prochaine** requête la déconnecte (avec trace dans le journal d'audit). Si `doit_changer_mot_de_passe` vaut vrai (mot de passe temporaire donné par l'administrateur), toutes les pages renvoient vers « Changer mon mot de passe », sauf cette page elle-même et la déconnexion.

`EntetesSecurite` : les en-têtes HTTP de sécurité

`app/Http/Middleware/EntetesSecurite.php` — lignes 1 à 31

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

/** En-têtes HTTP de sécurité (CSP stricte : aucun script ni style externe). */
class EntetesSecurite
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);
        $h = $response->headers;
        $vite = app()->environment('local') && is_file(public_path('hot')) ? ' '.trim((string)
file_get_contents(public_path('hot'))).' ws:' : '';
        $h->set('Content-Security-Policy', "default-src 'self'; script-src 'self'$vite; style-src 'self'
'unsafe-inline'$vite; img-src 'self' data:; font-src 'self'; connect-src 'self'$vite; frame-ancestors
'none'; form-action 'self'; base-uri 'self'; object-src 'none'");
        $h->set('X-Frame-Options', 'DENY');
        $h->set('X-Content-Type-Options', 'nosniff');
        $h->set('Referrer-Policy', 'same-origin');
        $h->set('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
        if (config('inps.forcer_https')) {
            $h->set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains');
        }
        if ($request->user() && ! $h->has('Content-Disposition')) {
            $h->set('Cache-Control', 'no-store, private');
        }

        return $response;
    }
}
```

Ce middleware agit **après** le contrôleur (il appelle d'abord `$next($request)` puis modifie la réponse) :

- **Content-Security-Policy (CSP)** : le navigateur n'exécute que les scripts servis par l'application elle-même (`'self'`). Un script injecté dans une page (attaque XSS) serait bloqué. C'est pour cette raison que le projet ne contient **aucun script écrit dans les vues** : tout le JavaScript est dans `resources/js/app.js`. En développement avec `npm run dev`, l'adresse du serveur Vite (fichier `public/hot`) est autorisée en plus.
- **X-Frame-Options: DENY** : interdit d'afficher l'application dans un cadre d'un autre site (anti-clickjacking).
- **Strict-Transport-Security** : impose HTTPS au navigateur, seulement si `FORCE_HTTPS` est actif.
- **Cache-Control: no-store** pour les pages d'un utilisateur connecté : le navigateur ne garde pas de copie des pages contenant des données personnelles.

Les middlewares fournis par Laravel

Le groupe `web` de Laravel 13 (appliqué à toutes les routes de `routes/web.php`) contient, dans cet ordre : `EncryptCookies`, `AddQueuedCookiesToResponse`, `StartSession`, `ShareErrorsFromSession`, `PreventRequestForgery`, `SubstituteBindings` ; le projet y ajoute `EntetesSecurite` et `VerifierCompteActif`.

Middleware	Rôle
`EncryptCookies`	Chiffre les cookies : l'utilisateur ne peut ni les lire ni les falsifier.
`StartSession`	Retrouve la session de l'utilisateur (table `sessions`, contenu chiffré si `SESSION_ENCRYPT=true`).
`ShareErrorsFromSession`	Rend la variable `$errors` (erreurs de validation) disponible dans toutes les vues.
`PreventRequestForgery`	Protection CSRF : refuse tout `POST` / `PUT` / `DELETE` sans le jeton `_token` correct → page **419 « Page Expired »**. Les vues l'insèrent avec `@csrf`.
`SubstituteBindings`	Transforme `{besoin}` dans l'URL en objet `BesoinFormation` (liaison de modèle).
`auth`	Exige un utilisateur connecté, sinon redirection vers la connexion.
`guest`	Réservé aux visiteurs non connectés.
`throttle:…`	Limitation du nombre de requêtes (tentatives de connexion, réinitialisation).

2.6 La configuration globale : `AppServiceProvider`

`app/Providers/AppServiceProvider.php` est exécuté au démarrage de chaque requête. Sa méthode `boot()` règle des comportements globaux :

`app/Providers/AppServiceProvider.php` — lignes 37 à 93

```
public function boot(): void
{
    // Protection contre l'affectation massive : seuls les champs $fillable sont acceptés ;
    // en développement, toute tentative sur un champ non autorisé lève une exception.
    Model::preventSilentlyDiscardingAttributes(! app()->isProduction());
    Model::preventLazyLoading(false);

    // Types polymorphes stables en base (indépendants des noms de classes).
    Relation::enforceMorphMap([
        'user' => User::class, 'role' => Role::class, 'agent' => Agent::class, 'besoin' =>
BesoinFormation::class,
        'plan' => PlanFormation::class, 'plan_version' => PlanVersion::class, 'action' =>
ActionFormation::class,
        'session' => SessionFormation::class, 'inscription' => Inscription::class, 'organisme' =>
Organisme::class,
        'depense' => Depense::class, 'document' => Document::class, 'attestation' => Attestation::class,
'import' => Import::class,
    ]);

    if (config('inps.forcer_https')) {
        URL::forceScheme('https');
    }

    // Politique de mot de passe (longueur paramétrable, complexité, contrôle des fuites optionnel).
    Password::defaults(function () {
        $r = Password::min(config('inps.securite.mot_de_passe_longueur', 12))->letters()->mixedCase()-
>numbers()->symbols();

        return config('inps.securite.mot_de_passe_compromis') ? $r->uncompromised() : $r;
    });

    // Limitation des tentatives de connexion et de réinitialisation.
    RateLimiter::for('connexion', fn (Request $r) => [
```

```
        Limit::perMinute(config('inps.securite.tentatives_connexion', 5))->by(mb_strtolower((string) $r->input('email')).'|'.$r->ip()),
        Limit::perMinute(30)->by($r->ip()),
    ]);
    RateLimiter::for('reinitialisation', fn (Request $r) => Limit::perMinute(3)->by($r->ip()));

    // Chaque permission est exposée comme « ability » : @can('finances.voir'), $user->can(...).
    // Refus par défaut : aucune règle ne donne tous les droits à un profil.
    Gate::after(function (User $user, string $ability, ?bool $resultat) {
        if ($resultat !== null) {
            return $resultat;
        }

        return str_contains($ability, '.') ? $user->aPermission($ability) : null;
    });

    Blade::if('permission', fn (string ...$codes) => auth()->check() && auth()->user()->aUneDesPermissions(...$codes));

    Paginator::defaultView('components.pagination');

    // Courriel de réinitialisation en français (lien à usage unique et durée limitée)
    ResetPassword::toMailUsing(fn ($user, string $token) => (new MailMessage)
        ->subject('Réinitialisation de votre mot de passe — gestion des formations')
        ->greeting('Bonjour,')
        ->line('Une demande de réinitialisation du mot de passe de votre compte a été reçue.')
        ->action('Définir un nouveau mot de passe', url(route('password.reset', ['token' => $token, 'email' => $user->getEmailForPasswordReset()], false)))
        ->line('Ce lien expire dans '.config('auth.passwords.users.expire').' minutes.')
        ->line('Si vous n\'êtes pas à l\'origine de cette demande, ignorez ce message et prévenez l\'administrateur.')
        ->salutation('Direction des Ressources humaines'));
}
```

Les points clés :

- **`Model::preventSilentlyDiscardingAttributes`** : en développement, si du code tente de remplir un champ non autorisé (absent de `$fillable`), Laravel lève une erreur au lieu de l'ignorer silencieusement. Cela aide à détecter les erreurs pendant le développement.
- **`Relation::enforceMorphMap`** : pour les tables « polymorphes » (`documents`, `decisions`, `notifications`), la colonne de type stocke un alias court et stable (`besoin`, `session`…) plutôt que le nom complet de la classe PHP. Renommer une classe ne casse donc pas les données.
- **`Password::defaults`** : la politique de mot de passe (12 caractères minimum par défaut, lettres majuscules et minuscules, chiffres, symboles). Elle est réutilisée partout avec `RegleMotDePasse::defaults()`.
- **`RateLimiter::for('connexion', …)`** : 5 tentatives par minute par couple e-mail + adresse IP, et 30 par minute par adresse IP. C'est cette règle qui produit le message « Trop de tentatives ».
- **`Gate::after`** : branche le système de permissions maison sur le système d'autorisation de Laravel. Grâce à lui, `$user->can('finances.voir')` et la directive Blade `@can('finances.voir')` fonctionnent avec les permissions de la base.
- **`Blade::if('permission', …)`** : crée la directive `@permission('code') … @endpermission` utilisée dans les vues pour afficher un bouton seulement si l'utilisateur a le droit.
- **`Paginator::defaultView`** : la pagination utilise le composant maison `resources/views/components/pagination.blade.php`.
- **`ResetPassword::toMailUsing`** : le texte en français du courriel de réinitialisation du mot de passe.

2.7 Les contrôleurs

Un **contrôleur** regroupe les actions d'un écran. Ils sont tous dans `app/Http/Controllers` et héritent de la classe de base :

`app/Http/Controllers/Controller.php` — lignes 1 à 25

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Http\Request;

abstract class Controller
{
    use AuthorizesRequests;

    /** Filtres communs (année, direction, service, domaine, type) validés. */
    protected function filtres(Request $r): array
    {
        return array_filter($r->validate([
            'annee' => ['nullable', 'integer', 'between:2000,2100'],
            'direction_id' => ['nullable', 'integer', 'exists:directions,id'],
            'service_id' => ['nullable', 'integer', 'exists:services,id'],
            'domaine_id' => ['nullable', 'integer', 'exists:domaines,id'],
            'type' => ['nullable', 'in:interne,externe'],
            'du' => ['nullable', 'date'],
            'au' => ['nullable', 'date'],
        ]), fn ($v) => $v !== null && $v !== '');
    }
}
```

- `use AuthorizesRequests;` donne aux contrôleurs la méthode `$this->authorize('action', $objet)`, qui interroge la *policy* et renvoie **403** si la réponse est non.
- `filtres()` valide les filtres communs aux écrans de pilotage (année, direction, service, domaine, type, période). `array_filter` retire les filtres vides. Le tableau de bord et les rapports l'utilisent.

Lecture complète d'un contrôleur : `BesoinController`

C'est le contrôleur du recensement des besoins ; il est représentatif de tous les autres.

`app/Http/Controllers/BesoinController.php` — lignes 20 à 40

```php
/** Module C — Recensement des besoins (cloisonnement par service via policy). */
class BesoinController extends Controller
{
    public function __construct(private BesoinService $service) {}

    public function index(Request $r): View
    {
        $this->authorize('viewAny', BesoinFormation::class);
        $f = $r->validate(['annee' => ['nullable', 'integer'], 'statut' => ['nullable',
Rule::enum(StatutBesoin::class)], 'service_id' => ['nullable', 'integer'],
            'priorite_id' => ['nullable', 'integer'], 'q' => ['nullable', 'string', 'max:100']]);
        $besoins = BesoinFormation::visiblesPar($r->user())->with(['service.direction', 'priorite',
'auteur'])
            ->when($f['annee'] ?? null, fn ($q, $v) => $q->where('annee', $v))
            ->when($f['statut'] ?? null, fn ($q, $v) => $q->where('statut', $v))
            ->when($f['service_id'] ?? null, fn ($q, $v) => $q->where('service_id', $v))
            ->when($f['priorite_id'] ?? null, fn ($q, $v) => $q->where('priorite_id', $v))
            ->when($f['q'] ?? null, fn ($q, $v) => $q->where(fn ($w) => $w->where('intitule', 'like',
'%'.addcslashes($v, '%_').'%')->orWhere('reference', 'like', '%'.addcslashes($v, '%_').'%')))
            ->latest('id')->paginate(25)->withQueryString();

        return view('besoins.index', ['besoins' => $besoins, 'f' => $f, 'annees' =>
```

```
AnneeReference::orderByDesc('annee')->pluck('annee'),
            'services' => $r->user()->aPermission('besoins.voir_tous') ? Service::actifs()->get() :
collect(), 'priorites' => Nomenclature::deType('priorite')->get()]);
    }
```

- **Injection de dépendances** : `public function __construct(private BesoinService $service) {}`. Laravel crée lui-même le `BesoinService` (et ses propres dépendances, comme `NotificationService`) et le passe au contrôleur. On n'écrit jamais `new BesoinService()`.
- **`index`** (liste) : 1. `$this->authorize('viewAny', BesoinFormation::class)` : la policy dit si l'utilisateur peut voir la liste. 2. `$r->validate([...])` valide les filtres de l'URL (`?annee=2026&statut=soumise`). `Rule::enum(StatutBesoin::class)` n'accepte que les valeurs de l'énumération. 3. `BesoinFormation::visiblesPar($r->user())` applique le **cloisonnement** : un chef de service ne voit que les besoins de son service (voir 2.10). 4. `->with(['service.direction', 'priorite', 'auteur'])` charge les relations en quelques requêtes groupées au lieu d'une requête par ligne (*eager loading*). 5. `->when($valeur, fn ...)` n'ajoute la condition que si le filtre est renseigné. 6. `addcslashes($v, '%_')` neutralise les caractères spéciaux de `LIKE` dans la recherche libre. 7. `->latest('id')->paginate(25)->withQueryString()` : tri du plus récent, 25 par page, en conservant les filtres dans les liens de pagination. 8. `return view('besoins.index', [...])` : affiche `resources/views/besoins/index.blade.php` avec les variables listées.

`app/Http/Controllers/BesoinController.php` — lignes 42 à 101

```
public function create(Request $r): View
{
    $this->authorize('create', BesoinFormation::class);

    return view('besoins.form', ['besoin' => new BesoinFormation(['annee' =>
collect(AnneeReference::ouvertes())->min() ?? now()->year + 1, 'service_id' => $r->user()->service_id])] +
$this->listes($r));
}

public function store(BesoinRequest $r): RedirectResponse
{
    $b = $this->service->creer($r->donnees(), $r->user(), $this->agentsAutorises($r));
    if ($r->boolean('soumettre')) {
        $this->service->transition($b, 'soumission', $r->user());
    }

    return redirect()->route('besoins.show', $b)->with('succes', $r->boolean('soumettre') ? 'Demande
enregistrée et soumise.' : 'Brouillon enregistré.');
}

public function show(Request $r, BesoinFormation $besoin): View
{
    $this->authorize('view', $besoin);
    $besoin->load(['service.direction', 'priorite', 'typeFormation', 'domaine', 'auteur', 'agents',
'decisions.user', 'documents.categorie']);

    return view('besoins.show', ['b' => $besoin, 'categoriesDoc' =>
Nomenclature::deType('categorie_document')->get(),
        'examenActif' => Parametre::valeur('besoins.statut_en_examen_actif', true), 'completerActif' =>
Parametre::valeur('besoins.statut_a_completer_actif', true)]);
}

public function edit(Request $r, BesoinFormation $besoin): View
{
    $this->authorize('update', $besoin);

    return view('besoins.form', ['besoin' => $besoin->load('agents')] + $this->listes($r));
}

public function update(BesoinRequest $r, BesoinFormation $besoin): RedirectResponse
{
```

```
    $this->authorize('update', $besoin);
    $this->service->modifier($besoin, $r->donnees(), $r->user(), $this->agentsAutorises($r));
    if ($r->boolean('soumettre')) {
        $this->service->transition($besoin, 'soumission', $r->user());
    }

    return redirect()->route('besoins.show', $besoin)->with('succes', 'Demande mise à jour.');
}

public function destroy(Request $r, BesoinFormation $besoin): RedirectResponse
{
    $this->authorize('delete', $besoin);
    $this->service->supprimerBrouillon($besoin);

    return redirect()->route('besoins.index')->with('succes', 'Brouillon retiré.');
}

public function transition(Request $r, BesoinFormation $besoin): RedirectResponse
{
    $d = $r->validate(['action' => ['required',
'in:soumission,mise_en_examen,demande_complement,acceptation,rejet'], 'commentaire' => ['nullable',
'string', 'max:2000']]);
    $this->authorize($d['action'] === 'soumission' ? 'update' : 'instruire', $besoin);
    $this->service->transition($besoin, $d['action'], $r->user(), $d['commentaire'] ?? null);

    return back()->with('succes', 'Décision enregistrée.');
}
```

- **`create`** prépare un besoin vide (`new BesoinFormation([...])`, non enregistré) avec des valeurs par défaut : la première année ouverte, le service de l'utilisateur. Le même gabarit `besoins.form` sert à la création et à la modification ; `$besoin->exists` permet de distinguer les deux cas dans la vue.
- **`store`** reçoit un `BesoinRequest` au lieu d'un `Request` : la validation et le contrôle d'accès du formulaire sont donc faits **avant** d'entrer dans la méthode. Il délègue tout au service, puis redirige avec un message flash (`with('succes', ...)`), affiché une seule fois sur la page suivante.
- **`show`**, **`edit`**, **`update`**, **`destroy`** commencent tous par `$this->authorize(...)` sur l'objet concerné.
- **`transition`** gère les boutons de décision (« Mettre en examen », « Accepter », « Rejeter »…). Le contrôle d'accès dépend de l'action : `update` pour une soumission (c'est l'auteur qui soumet), `instruire` pour les autres (c'est la DRH qui décide).

`app/Http/Controllers/BesoinController.php` — lignes 103 à 123

```
/** Agents nommés : limités au périmètre visible par l'utilisateur. */
private function agentsAutorises(BesoinRequest $r): array
{
    $ids = $r->validated()['agents'] ?? [];

    return $ids ? Agent::visiblesPar($r->user())->whereIn('id', $ids)->pluck('id')->all() : [];
}

private function listes(Request $r): array
{
    $u = $r->user();

    return [
        'annees' => AnneeReference::where('statut', 'ouverte')->orderBy('annee')->pluck('annee'),
        'services' => $u->aPermission('besoins.creer_tous_services') ? Service::actifs()-
>with('direction')->get() : Service::whereKey($u->service_id)->get(),
        'priorites' => Nomenclature::deType('priorite')->get(),
        'types' => Nomenclature::deType('type_formation')->get(),
        'domaines' => Domaine::actifs()->get(),
        'agents' => Agent::visiblesPar($u)->where('actif', true)->orderBy('nom')->limit(1000)->get(['id',
'matricule', 'nom', 'prenom', 'service_id']),
    ];
}
```

`agentsAutorises` est une protection importante : un utilisateur ne peut rattacher à sa demande **que** des agents qu'il a le droit de voir, même s'il modifie le formulaire à la main pour envoyer d'autres identifiants.

> **À retenir**
>
> Schéma type d'une action de contrôleur : **autoriser → valider → appeler le service → rediriger ou afficher**. Si une méthode de contrôleur dépasse une quinzaine de lignes, c'est généralement qu'une règle métier devrait être déplacée dans le service.

2.8 La validation des données

Laravel offre deux façons de valider, toutes deux utilisées dans le projet.

1. Validation directe dans le contrôleur, pour les petits formulaires :

```
$d = $r->validate(['motif' => ['required', 'string', 'max:500']]);
```

Si une règle échoue, Laravel interrompt la méthode, renvoie au formulaire, et les messages sont disponibles dans la vue (variable `$errors`, affichés par le composant `x-champ`). Les messages sont en français grâce aux fichiers du dossier `lang/`.

2. Classe `FormRequest`, pour les formulaires importants (`app/Http/Requests`) :

`app/Http/Requests/BesoinRequest.php` — lignes 1 à 47

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class BesoinRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->aPermission('besoins.creer');
    }

    public function rules(): array
    {
        return [
            'annee' => ['required', 'integer', Rule::exists('annees_reference', 'annee')],
            'service_id' => ['required', 'integer', Rule::exists('services', 'id')->where('actif', true)],
            'intitule' => ['required', 'string', 'max:255'],
            'contexte' => ['nullable', 'string', 'max:5000'],
            'competences_visees' => ['nullable', 'string', 'max:5000'],
            'objectifs' => ['nullable', 'string', 'max:5000'],
            'publics_concernes' => ['nullable', 'string', 'max:2000'],
            'nb_participants' => ['nullable', 'integer', 'min:1', 'max:5000'],
            'priorite_id' => ['nullable', 'integer', Rule::exists('nomenclatures', 'id')->where('type',
'priorite')],
            'type_formation_id' => ['nullable', 'integer', Rule::exists('nomenclatures', 'id')-
>where('type', 'type_formation')],
            'domaine_id' => ['nullable', 'integer', 'exists:domaines,id'],
            'duree_estimee_jours' => ['nullable', 'numeric', 'min:0.5', 'max:365'],
            'periode_souhaitee' => ['nullable', 'string', 'max:100'],
            'cout_previsionnel' => ['nullable', 'numeric', 'min:0', 'max:9999999999999'],
            'agents' => ['nullable', 'array', 'max:500'],
            'agents.*' => ['integer', 'exists:agents,id'],
        ];
    }
```

```
    public function attributes(): array
    {
        return ['annee' => 'année de référence', 'service_id' => 'service demandeur', 'intitule' =>
'intitulé', 'nb_participants' => 'nombre de participants',
            'cout_previsionnel' => 'coût prévisionnel', 'duree_estimee_jours' => 'durée estimée'];
    }

    public function donnees(): array
    {
        return collect($this->validated())->except('agents')->all();
    }
}
```

- **`authorize()`** : si elle renvoie `false`, la requête est refusée (403) avant même la validation.
- **`rules()`** : une liste de règles par champ. Quelques règles utilisées :
- `required` / `nullable` : obligatoire / facultatif ;
- `Rule::exists('annees_reference', 'annee')` : la valeur doit exister dans la table (empêche d'inventer une année) ;
- `Rule::exists('services', 'id')->where('actif', true)` : le service doit exister **et** être actif ;
- `Rule::exists('nomenclatures', 'id')->where('type', 'priorite')` : l'identifiant doit être une priorité, pas une autre nomenclature ;
- `max:255`, `min:1`, `numeric`, `integer` : formats et limites ;
- `'agents' => ['nullable', 'array', 'max:500']` et `'agents.*' => [...]` : un tableau et la règle de chacun de ses éléments.
- **`attributes()`** : noms lisibles des champs dans les messages (« Le champ *service demandeur* est obligatoire »).
- **`donnees()`** : méthode maison qui renvoie les données validées sans le tableau `agents` (traité à part).

Deux autres classes montrent des techniques complémentaires :

`app/Http/Requests/AgentRequest.php` — lignes 15 à 18

```
protected function prepareForValidation(): void
{
    $this->merge(['matricule' => mb_strtoupper(trim((string) $this->input('matricule')))]);
}
```

`prepareForValidation()` nettoie la saisie **avant** la validation : le matricule est mis en majuscules et débarrassé des espaces.

`app/Http/Requests/AgentRequest.php` — lignes 20 à 35

```
public function rules(): array
{
    $id = $this->route('agent')?->id;

    return [
        'matricule' => ['required', 'string', 'max:30', 'regex:/^[A-Z0-9\-\/\.]{2,30}$/',
Rule::unique('agents', 'matricule')->ignore($id)],
        'nom' => ['required', 'string', 'max:100'],
        'prenom' => ['required', 'string', 'max:150'],
        'direction_id' => ['nullable', 'integer', 'exists:directions,id'],
        'service_id' => ['nullable', 'integer', Rule::exists('services', 'id')->when($this-
>input('direction_id'), fn ($r) => $r->where('direction_id', $this->input('direction_id')))],
        'fonction' => ['nullable', 'string', 'max:190'],
        'statut_professionnel_id' => ['nullable', 'integer', Rule::exists('nomenclatures', 'id')-
>where('type', 'statut_professionnel')],
        'email_pro' => ['nullable', 'email', 'max:190'],
        'telephone_pro' => ['nullable', 'string', 'max:40', 'regex:/^[0-9+ ().-]{6,40}$/'],
```

```
        ];
    }
```

- `Rule::unique('agents', 'matricule')->ignore($id)` : le matricule doit être unique, sauf pour l'agent en cours de modification lui-même.
- `regex:` impose un format (lettres, chiffres, tirets…).
- La règle du `service_id` vérifie que le service appartient bien à la direction choisie.

> **Attention**
>
> Seules les données **validées** (`$r->validated()` ou le retour de `$r->validate()`) doivent être transmises aux services. Ne jamais passer `$r->all()` : un utilisateur malveillant pourrait ajouter des champs (par exemple `statut=acceptee`). Le test `test_cycle_de_vie_complet_d_une_demande_via_http` vérifie justement que ces champs sont ignorés.

2.9 Les autorisations : permissions et policies

L'application contrôle l'accès à **deux niveaux** :

Niveau	Question posée	Où
Permission (fonctionnalité)	« Cet utilisateur peut-il utiliser ce module ? »	Middleware `permission:` dans `routes/web.php` ; directive `@permission` dans les vues ; `$user->aPermission('code')` dans le code
Policy (objet)	« Peut-il voir / modifier **cet** enregistrement précis ? »	Classes de `app/Policies`, appelées par `$this->authorize()`

D'où viennent les permissions d'un utilisateur ?

Un utilisateur a des **rôles** (table `role_user`), chaque rôle a des **permissions** (table `permission_role`). Il n'y a **aucun** droit attribué directement à une personne, et aucun rôle « super-administrateur » qui aurait tout.

`app/Models/User.php` — lignes 58 à 72

```
/** @return array<string,bool> */
public function permissionCodes(): array
{
    if ($this->permissionsCache === null) {
        $this->permissionsCache = Permission::query()
            ->join('permission_role', 'permissions.id', '=', 'permission_role.permission_id')
            ->join('role_user', 'role_user.role_id', '=', 'permission_role.role_id')
            ->where('role_user.user_id', $this->id)
            ->pluck('permissions.code')
            ->mapWithKeys(fn ($c) => [$c => true])
            ->all();
    }

    return $this->permissionsCache;
}
```

`permissionCodes()` exécute **une seule requête** (jointure `permissions` → `permission_role` → `role_user`) et garde le résultat en mémoire pour la durée de la requête HTTP (`$permissionsCache`).

`app/Models/User.php` — lignes 80 à 87

```
public function aPermission(string $code): bool
{
    if (! $this->actif) {
        return false;
```

```
    }

    return isset($this->permissionCodes()[$code]);
}
```

Un compte désactivé (`actif = false`) n'a **aucune** permission, même si ses rôles sont toujours en base.

Une policy : `BesoinFormationPolicy`

Laravel associe automatiquement `app/Policies/BesoinFormationPolicy.php` au modèle `App\Models\BesoinFormation` grâce à la convention de nommage (nom du modèle + `Policy`). Aucune déclaration n'est nécessaire.

`app/Policies/BesoinFormationPolicy.php` — lignes 1 à 44

```php
<?php

namespace App\Policies;

use App\Enums\StatutBesoin;
use App\Models\BesoinFormation;
use App\Models\User;

/** Cloisonnement des besoins par service ; refus par défaut. */
class BesoinFormationPolicy
{
    public function viewAny(User $u): bool
    {
        return $u->aUneDesPermissions('besoins.voir_tous', 'besoins.voir_service');
    }

    public function view(User $u, BesoinFormation $b): bool
    {
        return $u->aPermission('besoins.voir_tous')
            || ($u->aPermission('besoins.voir_service') && $u->service_id && $u->service_id === $b->service_id);
    }

    public function create(User $u): bool
    {
        return $u->aPermission('besoins.creer');
    }

    /** Modification / soumission : demande modifiable, et périmètre du service (ou tous services). */
    public function update(User $u, BesoinFormation $b): bool
    {
        return $b->estModifiable() && $u->aPermission('besoins.creer')
            && ($u->aPermission('besoins.creer_tous_services') || ($u->service_id && $u->service_id === $b->service_id));
    }

    public function delete(User $u, BesoinFormation $b): bool
    {
        return $b->statut === StatutBesoin::Brouillon && $this->update($u, $b) && ($b->auteur_id === $u->id || $u->aPermission('besoins.creer_tous_services'));
    }

    public function instruire(User $u, BesoinFormation $b): bool
    {
        return $u->aPermission('besoins.instruire') && $this->view($u, $b);
    }
}
```

Chaque méthode répond par vrai ou faux :

- `view` : la DRH (`besoins.voir_tous`) voit tout ; un chef de service (`besoins.voir_service`) ne voit que les besoins **de son propre service** (`$u->service_id === $b->service_id`).
- `update` : il faut que la demande soit encore modifiable (brouillon ou « à compléter »), avoir le droit de créer des besoins, et être du même service (sauf permission `besoins.creer_tous_services`).
- `delete` : seulement un brouillon, et seulement par son auteur (ou par quelqu'un qui gère tous les services).
- `instruire` : avoir `besoins.instruire` **et** pouvoir voir la demande.

La liste et la fiche utilisent la même logique : la policy pour un objet, le *scope* `visiblesPar` pour une liste (voir 2.10). Les deux doivent rester cohérents.

2.10 Les modèles Eloquent

Un **modèle** est une classe PHP qui représente une table. Eloquent (l'ORM de Laravel) traduit les appels PHP en requêtes SQL. Exemple avec `BesoinFormation` :

`app/Models/BesoinFormation.php` — lignes 1 à 35

```php
<?php

namespace App\Models;

use App\Enums\StatutBesoin;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
use Illuminate\Database\Eloquent\Relations\MorphMany;
use Illuminate\Database\Eloquent\SoftDeletes;

class BesoinFormation extends Model
{
    use SoftDeletes;

    protected $table = 'besoins_formation';

    /** Le statut, l'auteur, la référence et la date de soumission sont gérés par le service métier. */
    protected $fillable = [
        'annee', 'service_id', 'intitule', 'contexte', 'competences_visees', 'objectifs',
        'publics_concernes', 'nb_participants', 'priorite_id', 'type_formation_id', 'domaine_id',
        'duree_estimee_jours', 'periode_souhaitee', 'cout_previsionnel',
    ];

    protected function casts(): array
    {
        return [
            'statut' => StatutBesoin::class,
            'soumis_le' => 'datetime',
            'cout_previsionnel' => 'decimal:2',
            'duree_estimee_jours' => 'decimal:1',
            'annee' => 'integer',
        ];
    }
```

- `protected $table = 'besoins_formation';` : nom de la table (sinon Laravel le déduirait de l'anglais).
- `use SoftDeletes;` : `delete()` ne supprime pas la ligne, il remplit `deleted_at`. Les requêtes ignorent ensuite automatiquement les lignes « supprimées ».
- `$fillable` : la liste blanche des champs remplissables en masse (`new BesoinFormation($donnees)`, `->fill($donnees)`). Le statut, l'auteur et la référence n'y figurent **pas** : ils ne peuvent être fixés que par le service, champ par champ (`$b->statut = ...`). C'est la protection contre l'**affectation de masse**.

- **`casts()`** : conversion automatique des colonnes. `'statut' => StatutBesoin::class` transforme la chaîne `'soumise'` de la base en objet `StatutBesoin::Soumise` ; `'cout_previsionnel' => 'decimal:2'` garde deux décimales ; `'soumis_le' => 'datetime'` donne un objet date manipulable (`->format('d/m/Y')`).

`app/Models/BesoinFormation.php` — lignes 37 à 94

```
    public function service(): BelongsTo
    {
        return $this->belongsTo(Service::class);
    }

    public function auteur(): BelongsTo
    {
        return $this->belongsTo(User::class, 'auteur_id');
    }

    public function priorite(): BelongsTo
    {
        return $this->belongsTo(Nomenclature::class, 'priorite_id');
    }

    public function typeFormation(): BelongsTo
    {
        return $this->belongsTo(Nomenclature::class, 'type_formation_id');
    }

    public function domaine(): BelongsTo
    {
        return $this->belongsTo(Domaine::class);
    }

    public function agents(): BelongsToMany
    {
        return $this->belongsToMany(Agent::class, 'besoin_agent', 'besoin_id', 'agent_id');
    }

    public function decisions(): MorphMany
    {
        return $this->morphMany(Decision::class, 'decidable')->latest('id');
    }

    public function documents(): MorphMany
    {
        return $this->morphMany(Document::class, 'documentable');
    }

    public function estModifiable(): bool
    {
        return in_array($this->statut, [StatutBesoin::Brouillon, StatutBesoin::ACompleter], true);
    }

    /** Cloisonnement par service selon les permissions. */
    public function scopeVisiblesPar(Builder $q, User $user): Builder
    {
        if ($user->aPermission('besoins.voir_tous')) {
            return $q;
        }
        if ($user->aPermission('besoins.voir_service') && $user->service_id) {
            return $q->where('service_id', $user->service_id);
        }

        return $q->whereRaw('1 = 0');
    }
}
```

Les **relations** :

Méthode	Type	Signification	SQL produit
`service()`	`belongsTo`	le besoin appartient à un service (colonne `service_id`)	`SELECT * FROM services WHERE id = ?`
`auteur()`	`belongsTo(User::class, 'auteur_id')`	clé étrangère au nom non standard, d'où le 2e argument	`SELECT * FROM users WHERE id = ?`
`agents()`	`belongsToMany(Agent::class, 'besoin_agent', 'besoin_id', 'agent_id')`	plusieurs agents, via la table pivot `besoin_agent`	jointure sur `besoin_agent`
`decisions()`	`morphMany(Decision::class, 'decidable')`	relation **polymorphe** : la table `decisions` sert aux besoins et aux versions de plan, distingués par `decidable_type`	`WHERE decidable_type = 'besoin' AND decidable_id = ?`
`documents()`	`morphMany(Document::class, 'documentable')`	idem pour les pièces jointes	

`$besoin->service` (sans parenthèses) renvoie l'objet lié ; `$besoin->service()` (avec parenthèses) renvoie la relation, sur laquelle on peut continuer une requête (`$besoin->agents()->sync([...])`).

`estModifiable()` est une petite règle métier attachée au modèle. **`scopeVisiblesPar`** est un *scope local* : on l'appelle `BesoinFormation::visiblesPar($user)` (Laravel retire le préfixe `scope`). Il ajoute `WHERE service_id = ?` pour un chef de service, rien pour la DRH, et `WHERE 1 = 0` (aucun résultat) pour quelqu'un sans permission : **refus par défaut**, encore.

Les énumérations (`app/Enums`)

`app/Enums/StatutBesoin.php` — lignes 1 à 55

```php
<?php

namespace App\Enums;

/**
 * Statuts d'une demande de besoin en formation.
 * Les codes sont stables (utilisés par la logique métier) ; les libellés sont affichés à l'écran.
 */
enum StatutBesoin: string
{
    case Brouillon = 'brouillon';
    case Soumise = 'soumise';
    case EnExamen = 'en_examen';
    case ACompleter = 'a_completer';
    case Acceptee = 'acceptee';
    case Rejetee = 'rejetee';
    case IntegreePlan = 'integree_plan';

    public function libelle(): string
    {
        return match ($this) {
            self::Brouillon => 'Brouillon',
            self::Soumise => 'Soumise',
            self::EnExamen => 'En examen',
            self::ACompleter => 'À compléter',
            self::Acceptee => 'Acceptée',
            self::Rejetee => 'Rejetée',
            self::IntegreePlan => 'Intégrée au plan',
        };
    }

    public function couleur(): string
    {
        return match ($this) {
```

```
            self::Brouillon => 'gris',
            self::Soumise => 'bleu',
            self::EnExamen => 'indigo',
            self::ACompleter => 'orange',
            self::Acceptee => 'vert',
            self::Rejetee => 'rouge',
            self::IntegreePlan => 'emeraude',
        };
    }

    /** @return array<string,string> */
    public static function options(): array
    {
        $o = [];
        foreach (self::cases() as $c) {
            $o[$c->value] = $c->libelle();
        }

        return $o;
    }
}
```

Une **énumération** PHP liste les valeurs possibles d'un statut. La base stocke le code (`'a_completer'`), l'écran affiche le libellé (`->libelle()` → « À compléter ») et la couleur du badge (`->couleur()`). `options()` fournit la liste pour un menu déroulant. Il y a 7 énumérations : `StatutBesoin`, `StatutPlanVersion`, `StatutAction`, `StatutSession`, `StatutInscription`, `StatutPresence`, `StatutDepense`.

> **Attention**
>
> Ne renomme jamais la **valeur** d'un cas d'énumération (`'a_completer'`) : elle est enregistrée dans des milliers de lignes de la base. Tu peux en revanche changer librement son **libellé**.

2.11 Les services métier

Les services (`app/Services`) contiennent les règles de gestion. Le plus pédagogique est `BesoinService`, qui gère le cycle de vie d'une demande.

`app/Services/BesoinService.php` — lignes 13 à 31

```
/**
 * Cycle de vie des demandes de besoins en formation.
 *
 * brouillon —soumettre→ soumise —examiner→ en_examen —accepter→ acceptee —(plan)→ integree_plan
 *                  ↑            └──────────────┴─demander_complement→ a_completer —soumettre┘
 *                  └ (a_completer)               └──────────── rejeter → rejetee
 */
class BesoinService
{
    /** action => [statuts de départ autorisés, statut d'arrivée, commentaire obligatoire] */
    private const TRANSITIONS = [
        'soumission' => [[S::Brouillon, S::ACompleter], S::Soumise, false],
        'mise_en_examen' => [[S::Soumise], S::EnExamen, false],
        'demande_complement' => [[S::Soumise, S::EnExamen], S::ACompleter, true],
        'acceptation' => [[S::Soumise, S::EnExamen], S::Acceptee, false],
        'rejet' => [[S::Soumise, S::EnExamen], S::Rejetee, true],
        'integration_plan' => [[S::Acceptee], S::IntegreePlan, false],
        'retrait_plan' => [[S::IntegreePlan], S::Acceptee, false],
    ];
```

La constante **`TRANSITIONS`** est une **machine à états** : pour chaque action, elle indique depuis quels statuts elle est permise, vers quel statut elle mène, et si un commentaire est obligatoire. Pour ajouter une étape au circuit, on ajoute une ligne ici (et le libellé dans `StatutBesoin`).

`app/Services/BesoinService.php` — lignes 35 à 51

```
public function creer(array $donnees, User $auteur, array $agentIds = []): BesoinFormation
{
    $this->verifierService($donnees['service_id'] ?? null, $auteur);

    return DB::transaction(function () use ($donnees, $auteur, $agentIds) {
        $b = new BesoinFormation($donnees);
        $b->reference = GenerateurReference::suivante('besoins_formation', 'reference', 'BES', (int)
$donnees['annee']);
        $b->statut = S::Brouillon;
        $b->auteur_id = $auteur->id;
        $b->save();
        $b->agents()->sync($agentIds);
        $b->decisions()->create(['action' => 'creation', 'statut_apres' => S::Brouillon->value, 'user_id'
=> $auteur->id]);
        Audit::enregistrer('besoin.creation', $b, "Création du besoin {$b->reference}");

        return $b;
    });
}
```

- `verifierService` : un utilisateur ne peut déclarer que pour son propre service (sauf `besoins.creer_tous_services`). La règle est vérifiée **dans le service**, pas seulement dans l'écran, pour qu'elle s'applique quelle que soit l'origine de l'appel (formulaire, test, import futur).
- **`DB::transaction(function () {...})`** : toutes les écritures réussissent ensemble ou aucune n'est faite. Si l'insertion de la décision échouait, le besoin ne serait pas créé à moitié.
- `GenerateurReference::suivante('besoins_formation', 'reference', 'BES', 2026)` fabrique `BES-2026-0001`, `BES-2026-0002`…
- `$b->agents()->sync($agentIds)` met la table pivot `besoin_agent` en conformité avec la liste.
- `$b->decisions()->create([...])` écrit l'historique visible sur la fiche ; `Audit::enregistrer(...)` écrit le journal d'audit.

`app/Services/BesoinService.php` — lignes 71 à 114

```
public function transition(BesoinFormation $b, string $action, User $user, ?string $commentaire = null):
BesoinFormation
{
    if (! isset(self::TRANSITIONS[$action])) {
        throw new RegleMetierException('Action inconnue.');
    }
    [$depart, $arrivee, $motifObligatoire] = self::TRANSITIONS[$action];

    if ($action === 'mise_en_examen' && ! Parametre::valeur('besoins.statut_en_examen_actif', true)) {
        throw new RegleMetierException('L\'étape « En examen » est désactivée dans le paramétrage.');
    }
    if ($action === 'demande_complement' && ! Parametre::valeur('besoins.statut_a_completer_actif', true))
{
        throw new RegleMetierException('L\'étape « À compléter » est désactivée dans le paramétrage.');
    }
    if ($action === 'soumission' && $b->annee) {
        $annee = AnneeReference::where('annee', $b->annee)->first();
        if ($annee && ! $annee->estOuverte()) {
            throw new RegleMetierException("L'année {$b->annee} est clôturée : aucune nouvelle soumission
n'est possible.");
        }
    }
    if ($motifObligatoire && trim((string) $commentaire) === '') {
        throw new RegleMetierException('Un commentaire motivé est obligatoire pour cette décision.');
    }

    return DB::transaction(function () use ($b, $action, $depart, $arrivee, $user, $commentaire) {
        $b = BesoinFormation::whereKey($b->id)->lockForUpdate()->firstOrFail();
        if (! in_array($b->statut, $depart, true)) {
```

```
            throw new RegleMetierException("Action impossible : la demande est « {$b->statut->libelle()}
».");
        }
        $avant = $b->statut;
        $b->statut = $arrivee;
        if ($action === 'soumission') {
            $b->soumis_le = now();
        }
        $b->save();
        $b->decisions()->create([
            'action' => $action, 'statut_avant' => $avant->value, 'statut_apres' => $arrivee->value,
            'commentaire' => $commentaire, 'user_id' => $user->id,
        ]);
        Audit::enregistrer('besoin.'.$action, $b, "{$b->reference} : {$avant->libelle()} → {$arrivee-
>libelle()}");
        $this->notifierTransition($b, $action);

        return $b;
    });
}
```

Points remarquables :

1. Les contrôles « de forme » (action connue, étape activée dans les paramètres, année ouverte, commentaire obligatoire) sont faits **avant** la transaction.
2. `lockForUpdate()` relit la demande en la verrouillant (`SELECT … FOR UPDATE`). Si deux personnes cliquent « Accepter » et « Rejeter » au même instant, la seconde attend la fin de la première, puis voit le nouveau statut et reçoit le message « Action impossible ». Sans ce verrou, les deux décisions pourraient s'écraser.
3. `in_array($b->statut, $depart, true)` applique la machine à états.
4. Chaque refus lance une `RegleMetierException` avec un message clair, transformé en message à l'écran par `bootstrap/app.php`.
5. `notifierTransition` prévient les bonnes personnes (DRH à la soumission, auteur lors d'une décision).

Les utilitaires transverses

Le journal d'audit — toute opération sensible appelle `Audit::enregistrer` :

`app/Services/Audit.php` — lignes 9 à 46

```
/**
 * Point d'entrée unique du journal d'audit.
 * Ne jamais transmettre de secret ni de donnée personnelle superflue dans $contexte :
 * les clés sensibles connues sont retirées automatiquement par sécurité.
 */
class Audit
{
    private const CLES_INTERDITES = ['password', 'mot_de_passe', 'password_confirmation', 'token',
'remember_token', '_token', 'motif_absence', 'secret'];

    public static function enregistrer(string $action, Model|string|null $objet = null, ?string
$description = null, array $contexte = [], ?int $userId = null): AuditLog
    {
        $request = app()->runningInConsole() && ! app()->runningUnitTests() ? null : request();
        $type = $objet instanceof Model ? $objet->getMorphClass() : $objet;
        $id = $objet instanceof Model ? $objet->getKey() : null;

        return AuditLog::create([
            'user_id' => $userId ?? auth()->id(),
            'action' => $action,
            'objet_type' => $type,
            'objet_id' => $id,
            'description' => $description ? mb_substr($description, 0, 500) : null,
            'contexte' => $contexte ? self::nettoyer($contexte) : null,
```

```
            'ip' => $request?->ip(),
            'user_agent' => $request ? mb_substr((string) $request->userAgent(), 0, 255) : 'console',
        ]);
    }

    private static function nettoyer(array $c): array
    {
        $c = Arr::except($c, self::CLES_INTERDITES);
        foreach ($c as $k => $v) {
            if (is_array($v)) {
                $c[$k] = self::nettoyer($v);
            }
        }

        return $c;
    }
```

Il enregistre qui (`user_id`), quoi (`action`, ex. `besoin.soumission`), sur quel objet (`objet_type`, `objet_id`), quand, depuis quelle adresse IP, avec un contexte JSON. Les clés sensibles (mots de passe, jetons, motifs d'absence) sont **retirées automatiquement** du contexte. La table `audit_logs` est en **ajout seul** : le modèle `AuditLog` refuse toute modification ou suppression, et deux déclencheurs MySQL font de même au niveau de la base.

Les références lisibles :

`app/Services/GenerateurReference.php` — lignes 1 à 22

```php
<?php

namespace App\Services;

use Illuminate\Support\Facades\DB;

/**
 * Génère des références lisibles et uniques : PREFIXE-ANNEE-NNNN.
 * Le calcul s'appuie sur la plus grande référence existante, sous verrou (FOR UPDATE)
 * dans la transaction appelante ; la contrainte d'unicité en base garantit l'absence de doublon.
 */
class GenerateurReference
{
    public static function suivante(string $table, string $colonne, string $prefixe, int $annee): string
    {
        $base = $prefixe.'-'.$annee.'-';
        $derniere = DB::table($table)->where($colonne, 'like', $base.'%')->lockForUpdate()->orderByDesc($colonne)->value($colonne);
        $n = $derniere ? ((int) substr($derniere, strlen($base))) + 1 : 1;

        return $base.str_pad((string) $n, 4, '0', STR_PAD_LEFT);
    }
}
```

La plus grande référence existante de l'année est lue **sous verrou** ; la contrainte d'unicité de la colonne `reference` garantit en dernier recours qu'aucun doublon n'est enregistré.

Les notifications — `NotificationService::notifier` crée une notification interne (table `notifications`, icône cloche) et, si le paramètre `notifications.email_actif` est activé, tente un e-mail dont le **résultat réel** est consigné dans `envois_email` :

`app/Services/NotificationService.php` — lignes 35 à 47

```php
/** @param iterable<User>|User $destinataires */
public function notifier(iterable|User $destinataires, string $categorie, string $titre, string $message,
?string $url = null): int
{
    $liste = $destinataires instanceof User ? collect([$destinataires]) : collect($destinataires);
```

```
        $liste = $liste->filter(fn (User $u) => $u->actif)->unique('id');
        $n = new NotificationMetier($categorie, $titre, $message, $url);
        foreach ($liste as $u) {
            $u->notify($n);
            $this->envoyerEmail($u, $titre, $message, $url);
        }

        return $liste->count();
    }
```

Les fonctions d'affichage (`app/helpers.php`) : `fcfa(2500000)` → « 2 500 000 FCFA », `date_fr($date)` → « 21/10/2026 », `pourcent(83.3)` → « 83,3 % ». Utilise-les dans les vues pour un affichage homogène.

Les autres services

Service	Rôle
`PlanService`	Plans annuels : création, intégration des besoins acceptés, lignes, soumission au circuit de validation, décisions par étape, verrouillage, révision, comparaison prévu/réalisé.
`ActionService`	Actions et sessions : création, séances journalières, modification, report, annulation, changements de statut.
`InscriptionService`	Inscriptions : capacité, liste d'attente, doublons, chevauchement de dates, convocations, désistements, remplacements.
`PresenceService`	Saisie des présences, validation du relevé, corrections, calcul du taux de présence et de l'achèvement.
`AttestationService`	Conditions d'émission, émission, annulation des attestations.
`DepenseService`	Dépenses : contrôles, corrections tracées, synthèse financière d'une action.
`DocumentService`	Dépôt sécurisé des pièces jointes (type réel vérifié, nom aléatoire, empreinte SHA-256).
`ImportService`	Import d'agents et d'historique depuis CSV/Excel, en deux temps (analyse puis confirmation).
`IndicateurService`	Indicateurs du tableau de bord.
`RapportService`	Les 13 rapports statistiques.
`ExportService`	Génération des fichiers Excel et PDF.
`SauvegardeService`	Sauvegarde et restauration (base + documents).
`NotificationService`, `Audit`, `GenerateurReference`	Utilitaires transverses (ci-dessus).

Ils sont détaillés module par module dans la partie 4.

2.12 Les vues Blade

Les pages sont produites par des **vues Blade** : des fichiers `*.blade.php` dans `resources/views`, rangés par module (`besoins/`, `plans/`, `sessions/`…). Un contrôleur appelle `view('besoins.show', [...])` : le point remplace le séparateur de dossier, donc `resources/views/besoins/show.blade.php`.

La syntaxe essentielle

Syntaxe	Rôle	Exemple dans le projet
`{{ $variable }}`	Affiche une valeur **échappée** (les caractères `<`, `>`, `"` sont neutralisés : protection contre l'injection de code, dite XSS)	`{{ $b->intitule }}`
`{!! $html !!}`	Affiche **sans** échappement — à n'utiliser que pour du HTML produit par l'application, jamais pour une saisie utilisateur	uniquement dans `x-icone`, pour insérer les tracés SVG définis dans le composant lui-même
`@if … @elseif … @else … @endif`	Condition	`@if ($annees->isEmpty())`
`@foreach ($liste as $x) … @endforeach`	Boucle	`@foreach ($agents as $a)`
`@csrf`	Champ caché contenant le jeton anti-falsification ; **obligatoire** dans tout formulaire `POST`	tous les formulaires
`@method('PUT')`	Simule `PUT` / `DELETE`	formulaire de modification
`@can('update', $b) … @endcan`	Affiche si la policy autorise	boutons « Modifier », « Soumettre »
`@permission('finances.voir') … @endpermission`	Affiche si l'utilisateur a la permission (directive maison)	blocs financiers
`@php … @endphp`	Petit bloc PHP (à limiter : la logique va dans le contrôleur ou le service)	préparation d'une liste
`<x-nom attribut="…" />`	Utilise un **composant** de `resources/views/components/nom.blade.php`	`<x-champ>`, `<x-carte>`
`:attribut="$expr"`	Passe une expression PHP à un composant (le `:` signifie « évaluer »)	`:valeur="$besoin->annee"`
`old('champ', $defaut)`	Valeur saisie avant une erreur de validation	dans `x-champ`

La coquille commune : `components/layout.blade.php`

Toutes les pages connectées commencent par `<x-layout :titre="...">` et finissent par `</x-layout>`. Le composant fournit le menu latéral, la barre du haut et la zone de messages ; le contenu de la page est inséré à la place de `{{ $slot }}`.

`resources/views/components/layout.blade.php` — lignes 1 à 55

```
@props(['titre' => null])
@php
    $u = auth()->user();
    $p = fn (...$c) => $u->aUneDesPermissions(...$c);
    $couleur = \App\Models\Parametre::valeur('institution.couleur_principale', '#003B73');
    $couleur = preg_match('/^#[0-9a-fA-F]{6}$/', (string) $couleur) ? $couleur : '#003B73';
    $sigle = \App\Models\Parametre::valeur('institution.sigle', 'INPS');
    $nomInstitution = \App\Models\Parametre::valeur('institution.nom', 'Institut National de Prévoyance
Sociale');
    $nonLues = $u->unreadNotifications()->count();
    $logo = \App\Support\Identite::url('logo');
    // [libellé, route, motif de route active, autorisé, icône]
    $menu = [
        'Pilotage' => [
            ['Tableau de bord', 'tableau-bord', 'tableau-bord', $p('tableau_bord.voir'), 'tableau'],
            ['Mon espace', 'espace.index', 'espace.*', $p('espace.personnel'), 'espace'],
        ],
```

```
        'Formations' => [
            ['Besoins', 'besoins.index', 'besoins.*', $p('besoins.voir_tous', 'besoins.voir_service'),
'besoins'],
            ['Plans annuels', 'plans.index', 'plans.*', $p('plans.voir'), 'plans'],
            ['Actions', 'actions.index', 'actions.*', $p('actions.voir'), 'actions'],
            ['Sessions', 'sessions.index', 'sessions.*|presences.*|inscriptions.*|evaluations.*',
$p('actions.voir'), 'sessions'],
            ['Attestations', 'attestations.index', 'attestations.*', $p('attestations.emettre'),
'attestations'],
        ],
        'Référentiels' => [
            ['Personnel', 'agents.index', 'agents.*', $p('agents.voir_tous', 'agents.voir_service'),
'personnel'],
            ['Importations', 'imports.index', 'imports.*', $p('agents.importer'), 'importations'],
            ['Organismes', 'organismes.index', 'organismes.*', $p('organismes.voir'), 'organismes'],
            ['Formateurs', 'formateurs.index', 'formateurs.*', $p('organismes.voir'), 'formateurs'],
        ],
        'Suivi' => [
            ['Finances', 'depenses.index', 'depenses.*', $p('finances.voir'), 'finances'],
            ['Rapports', 'rapports.index', 'rapports.*', $p('rapports.voir'), 'rapports'],
        ],
        'Administration' => [
            ['Paramètres', 'admin.parametres.index', 'admin.parametres.*', $p('parametres.gerer'),
'parametres'],
            ['Référentiels', ['admin.referentiels.index', 'directions'], 'admin.referentiels.*',
$p('parametres.gerer'), 'referentiels'],
            ['Circuits de validation', 'admin.circuits.index', 'admin.circuits.*', $p('parametres.gerer'),
'circuits'],
            ['Utilisateurs', 'admin.utilisateurs.index', 'admin.utilisateurs.*', $p('utilisateurs.gerer'),
'utilisateurs'],
            ['Rôles et permissions', 'admin.roles.index', 'admin.roles.*', $p('roles.gerer'), 'roles'],
            ['Journal d\'audit', 'admin.audit.index', 'admin.audit.*', $p('audit.voir'), 'audit'],
            ['Système et sauvegardes', 'admin.systeme.index', 'admin.systeme.*', $p('systeme.superviser'),
'systeme'],
        ],
    ];
    // Fil d'Ariane : rubrique active déduite du menu
    $rubrique = null;
    foreach ($menu as $section => $liens) {
        foreach ($liens as $l) {
            if ($l[3] && request()->routeIs(...explode('|', $l[2]))) {
                $rubrique = [$section, $l[0], is_array($l[1]) ? route($l[1][0], $l[1][1]) : route($l[1])];
            }
        }
    }
    $initiales = collect(preg_split('/\s+/', trim($u->name)))->filter(fn ($m) => preg_match('/^\p{L}/u',
$m))->take(2)->map(fn ($m) => mb_strtoupper(mb_substr($m, 0, 1)))->implode('');
    $roles = $u->roles->pluck('nom')->implode(', ');
@endphp
```

- Le **menu** est un tableau PHP : `[libellé, route, motif de route active, autorisé ?, icône]`. La 4e colonne appelle `$p('permission')` : une entrée n'apparaît **que si l'utilisateur détient la permission**. Pour ajouter une entrée de menu, on ajoute une ligne dans ce tableau.
- `request()->routeIs('besoins.*')` détermine l'entrée active (surlignée) et le **fil d'Ariane**.
- La couleur principale vient du paramètre `institution.couleur_principale` ; une valeur qui n'est pas un code couleur valide est remplacée par `#003B73` (le test `test_couleur_institutionnelle_par_defaut_et_valeur_invalide_neutralisee` le vérifie).
- `$nonLues` alimente la pastille rouge de la cloche des notifications.

`resources/views/components/layout.blade.php` — lignes 56 à 68

```
<!DOCTYPE html>
<html lang="fr" style="--couleur-principale: {{ $couleur }}">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <meta name="robots" content="noindex, nofollow">
    <meta name="theme-color" content="#003B73">
    <title>{{ $titre ? $titre.' — ' : '' }}INPS Formation</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body>
<a href="#contenu" class="sr-only focus:not-sr-only focus:fixed focus:top-3 focus:left-3 focus:z-[60] bg-
white px-4 py-2 rounded border border-filet">Aller au contenu</a>
<div class="coquille">
```

`@vite([...])` insère les balises `<link>` et `<script>` vers les fichiers compilés de `public/build/` (en lisant `public/build/manifest.json`). C'est la ligne qui provoque l'erreur « Vite manifest not found » si `npm run build` n'a jamais été exécuté.

> **À retenir**
>
> Masquer un bouton dans la vue **n'est pas** une sécurité : c'est du confort. La sécurité est assurée côté serveur (middleware `permission:` sur la route, `authorize()` dans le contrôleur). Les deux doivent toujours aller ensemble.

Les composants réutilisables

Le dossier `resources/views/components` contient 15 composants. Les plus utilisés :

Composant	Rôle
`x-layout`	Coquille des pages connectées (menu, barre du haut, messages).
`x-invite`	Coquille des pages publiques (connexion, mot de passe oublié).
`x-entete`	Titre de page, sous-titre et zone de boutons.
`x-carte`	Panneau blanc avec titre et zone d'actions.
`x-champ`	Champ de formulaire complet : libellé, valeur, aide, message d'erreur, accessibilité.
`x-badge`	Pastille de statut (couleur et libellé tirés de l'énumération).
`x-action-form`	Bouton qui envoie une action `POST` / `DELETE` protégée (avec confirmation facultative).
`x-dialogue-motif`	Boîte de dialogue demandant un motif (rejet, report, annulation…).
`x-documents`	Liste et dépôt des pièces jointes d'un objet.
`x-decisions`	Historique des décisions d'un besoin ou d'une version de plan.
`x-messages`	Messages de succès, d'erreur, d'avertissement, erreurs de validation.
`x-kpi`, `x-vide`, `x-pagination`, `x-icone`	Indicateur chiffré, état vide, pagination, pictogrammes SVG.

Le composant de champ montre comment un composant reçoit des paramètres (`@props`) et produit un HTML accessible :

`resources/views/components/champ.blade.php` — lignes 1 à 40

```
{{-- Champ de formulaire accessible : libellé visible, valeur ancienne, aide et erreur reliées au champ. -
-}}
@props(['nom', 'libelle', 'type' => 'text', 'valeur' => null, 'options' => null, 'requis' => false, 'aide'
=> null, 'vide' => '— Sélectionner —', 'attrs' => [], 'basculer' => false])
@php
    $id = 'ch_'.str_replace(['[', ']', '.'], '_', $nom);
    $cle = str_replace(['[', ']'], ['.', ''], $nom);
```

```
    $v = old($cle, $valeur instanceof \BackedEnum ? $valeur->value : ($valeur instanceof
\DateTimeInterface ? $valeur->format('Y-m-d') : $valeur));
    $enErreur = $errors->has($cle);
    $decrit = trim(($aide ? $id.'_aide ' : '').($enErreur ? $id.'_err' : ''));
@endphp
<div {{ $attributes->only('class') }}>
    @if ($type === 'checkbox')
        <input type="hidden" name="{{ $nom }}" value="0">
        <label class="inline-flex items-center gap-2.5 font-normal cursor-pointer"><input type="checkbox"
id="{{ $id }}" name="{{ $nom }}" value="1" @checked($v)> {{ $libelle }}</label>
    @else
        <label for="{{ $id }}">{{ $libelle }}@if ($requis) <span class="text-erreur" aria-hidden="true">*
</span>@endif</label>
        @if ($type === 'select')
            <select id="{{ $id }}" name="{{ $nom }}" @required($requis) @if ($enErreur) aria-
invalid="true" @endif @if ($decrit) aria-describedby="{{ $decrit }}" @endif>
                @if ($vide !== false)<option value="">{{ $vide }}</option>@endif
                @foreach ($options as $k => $o)
                    <option value="{{ $k }}" @selected((string) $v === (string) $k)>{{ $o }}</option>
                @endforeach
            </select>
        @elseif ($type === 'textarea')
            <textarea id="{{ $id }}" name="{{ $nom }}" rows="{{ $attrs['rows'] ?? 3 }}"
@required($requis) @if ($enErreur) aria-invalid="true" @endif @if ($decrit) aria-describedby="{{
$decrit }}" @endif>{{ $v }}</textarea>
        @elseif ($type === 'password' && $basculer)
            <div class="champ-mdp">
                <input type="password" id="{{ $id }}" name="{{ $nom }}" value="" @required($requis)
                       @foreach ($attrs as $a => $av) {{ $a }}="{{ $av }}" @endforeach @if ($enErreur)
aria-invalid="true" @endif @if ($decrit) aria-describedby="{{ $decrit }}" @endif>
                <button type="button" data-basculer-mdp="{{ $id }}" aria-controls="{{ $id }}" aria-
pressed="false" aria-label="Afficher le mot de passe">
                    <x-icone nom="oeil" class="h-[18px] w-[18px] icone-afficher" /><x-icone nom="oeil-
barre" class="h-[18px] w-[18px] icone-masquer hidden" />
                </button>
            </div>
        @else
            <input type="{{ $type }}" id="{{ $id }}" name="{{ $nom }}" value="{{ $type === 'password' ? ''
: $v }}" @required($requis)
                   @foreach ($attrs as $a => $av) {{ $a }}="{{ $av }}" @endforeach @if ($enErreur)
aria-invalid="true" @endif @if ($decrit) aria-describedby="{{ $decrit }}" @endif>
        @endif
    @endif
    @if ($aide)<p class="aide" id="{{ $id }}_aide">{{ $aide }}</p>@endif
    @error($cle)<p class="erreur-champ" id="{{ $id }}_err"><x-icone nom="erreur" class="h-3.5 w-3.5 mt-
0.5" />{{ $message }}</p>@enderror
</div>
```

- `@props([...])` déclare les paramètres et leurs valeurs par défaut.
- `$cle` convertit un nom de champ en notation de validation (`agents[0]` → `agents.0`) pour retrouver le bon message d'erreur.
- `old($cle, …)` réaffiche la saisie après une erreur ; un mot de passe n'est jamais réaffiché.
- `aria-invalid` et `aria-describedby` relient le champ à son message d'erreur pour les lecteurs d'écran.
- `@error($cle) … @enderror` n'affiche le message que si ce champ est en erreur.

Un formulaire complet, celui de la déclaration de besoin, s'écrit alors en peu de lignes :

`resources/views/besoins/form.blade.php` — lignes 1 à 18

```
<x-layout :titre="$besoin->exists ? 'Modifier la demande' : 'Nouvelle demande'">
    <x-entete :titre="$besoin->exists ? 'Modifier la demande '.$besoin->reference : 'Déclarer un besoin en
formation'" />
    @if ($annees->isEmpty())
        <div class="mb-4 rounded-md border border-amber-200 bg-amber-50 px-4 py-3 text-sm">Aucune année de
référence n'est ouverte au recensement. L'administrateur fonctionnel doit en ouvrir une (Administration
&gt; Référentiels &gt; Années).</div>
```

```
    @endif
    <form method="POST" action="{{ $besoin->exists ? route('besoins.update', $besoin) :
route('besoins.store') }}" class="space-y-5">
        @csrf @if ($besoin->exists) @method('PUT') @endif
        <x-carte titre="Identification">
            <div class="grille-form">
                <x-champ nom="annee" libelle="Année de référence" type="select" :valeur="$besoin->annee"
:options="$annees->combine($annees)" :requis="true" :vide="false" />
                <x-champ nom="service_id" libelle="Service demandeur" type="select" :valeur="$besoin-
>service_id" :options="$services->mapWithKeys(fn ($s) => [$s->id => $s->nom])" :requis="true" />
                <x-champ class="md:col-span-2" nom="intitule" libelle="Intitulé du besoin"
:valeur="$besoin->intitule" :requis="true" />
                <x-champ nom="domaine_id" libelle="Domaine de compétence" type="select" :valeur="$besoin-
>domaine_id" :options="$domaines->pluck('nom', 'id')" />
                <x-champ nom="type_formation_id" libelle="Type de formation souhaité" type="select"
:valeur="$besoin->type_formation_id" :options="$types->pluck('libelle', 'id')" />
                <x-champ nom="priorite_id" libelle="Niveau de priorité" type="select" :valeur="$besoin-
>priorite_id" :options="$priorites->pluck('libelle', 'id')" />
                <x-champ nom="periode_souhaitee" libelle="Période souhaitée" :valeur="$besoin-
>periode_souhaitee" aide="Exemple : 2e trimestre, avant juin…" />
            </div>
        </x-carte>
```

Les deux boutons de fin de formulaire portent le même nom `soumettre` avec deux valeurs (0 ou 1) : c'est ainsi que `BesoinController::store` sait s'il doit seulement enregistrer le brouillon ou aussi le soumettre (`$r->boolean('soumettre')`).

`resources/views/besoins/form.blade.php` — lignes 45 à 49

```
<div class="flex flex-wrap gap-2">
    <button class="btn btn-secondaire" name="soumettre" value="0">Enregistrer le brouillon</button>
    <button class="btn btn-primaire" name="soumettre" value="1" data-confirm="Soumettre la demande à la
DRH ? Elle ne sera plus modifiable sauf demande de compléments.">Enregistrer et soumettre</button>
    <a class="btn btn-secondaire" href="{{ $besoin->exists ? route('besoins.show', $besoin) :
route('besoins.index') }}">Annuler</a>
</div>
```

Les boutons d'action d'une fiche utilisent `x-action-form`, qui fabrique un petit formulaire protégé :

`resources/views/components/action-form.blade.php` — lignes 1 à 8

```
{{-- Bouton déclenchant une action POST protégée CSRF, avec confirmation éventuelle. --}}
@props(['url', 'libelle', 'methode' => 'POST', 'confirmer' => null, 'classe' => 'btn btn-secondaire btn-
petit', 'champs' => []])
<form method="POST" action="{{ $url }}" class="inline" @if ($confirmer) data-confirm="{{ $confirmer }}"
@endif>
    @csrf
    @if (strtoupper($methode) !== 'POST') @method($methode) @endif
    @foreach ($champs as $k => $v)<input type="hidden" name="{{ $k }}" value="{{ $v }}">@endforeach
    <button type="submit" class="{{ $classe }}">{{ $libelle }}</button>
</form>
```

Les gabarits PDF

Les PDF sont aussi des vues Blade (`resources/views/pdf/`), transformées par Dompdf dans `ExportService::pdf`. Les fichiers commençant par `_` (`_entete`, `_pied`, `_style`) sont des morceaux inclus avec `@include('pdf._entete')`. Dompdf ne comprend qu'un CSS simple : pas de Tailwind dans ces vues, uniquement le style de `pdf/_style.blade.php`.

2.13 Le CSS : Tailwind 4 et les composants maison

Tout le style est dans `resources/css/app.css`. Il commence par les **jetons** de la charte graphique :

`resources/css/app.css` — lignes 9 à 46

```
@import 'tailwindcss';
@import '@fontsource-variable/public-sans';

@source '../../vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php';
@source '../../storage/framework/views/*.php';
@source '../views';
@source '../../app';

@theme {
    --font-sans: 'Public Sans Variable', 'Public Sans', 'Segoe UI', system-ui, -apple-system, Arial, sans-
serif;

    /* La couleur principale reste paramétrable (Administration > Paramètres) */
    --color-primaire: var(--couleur-principale, #003B73);
    --color-primaire-fonce: color-mix(in oklab, var(--couleur-principale, #003B73) 78%, black);
    --color-primaire-clair: #EEF4FA;
    --color-inps: #003B73;
    --color-inps-2: #0056A3;
    --color-inps-50: #EEF4FA;
    --color-inps-100: #D9E6F3;

    --color-encre: #172235;
    --color-encre-2: #475467;
    --color-encre-3: #667085;
    --color-fond: #F3F5F8;
    --color-fond-2: #F8FAFC;
    --color-filet: #DCE2EA;
    --color-filet-fort: #C5CFDB;

    --color-ok: #1E7B4F;
    --color-ok-fond: #EAF5EF;
    --color-alerte: #8A5300;
    --color-alerte-fond: #FDF3E1;
    --color-erreur: #B42318;
    --color-erreur-fond: #FDECEA;

    --radius-controle: 6px;
    --radius-panneau: 8px;
}
```

- `@import 'tailwindcss';` charge Tailwind ; `@import '@fontsource-variable/public-sans';` charge la police depuis `node_modules` (elle est copiée dans `public/build` à la compilation).
- `@source` indique à Tailwind **où chercher les classes utilisées** (vues, code PHP). Une classe utilisée seulement ailleurs ne serait pas générée.
- `@theme { … }` définit les couleurs et rayons : chaque `--color-xxx` crée automatiquement les classes `bg-xxx`, `text-xxx`, `border-xxx`. Exemple : `--color-encre-3` permet d'écrire `text-encre-3` dans une vue.
- `--color-primaire` lit la variable `--couleur-principale` posée par `layout.blade.php` à partir du paramètre de l'INPS : changer la couleur dans l'écran *Paramètres* change l'interface sans recompiler.

Ensuite, la couche `@layer components` (à partir de la ligne 99) définit les classes maison : `.btn`, `.btn-primaire`, `.btn-secondaire`, `.btn-danger`, `.carte`, `.grille-form`, `.statut`, `.message`… Les vues combinent ces classes et les utilitaires Tailwind (`flex`, `gap-2`, `mt-4`, `md:grid-cols-3`…). Le fichier se termine par les règles d'accessibilité (`prefers-reduced-motion` : animations coupées si l'utilisateur l'a demandé à son système) et d'impression (`@media print`).

Attention

Après **toute** modification de `resources/css/app.css`, de `resources/js/app.js` **ou de classes Tailwind dans une vue**, il faut relancer `npm run build` (ou laisser tourner `npm run dev` pendant le développement). Sinon, la nouvelle classe n'existe pas dans `public/build` et n'a aucun effet.

2.14 Le JavaScript : `resources/js/app.js`

Le projet n'utilise aucune bibliothèque JavaScript. Le fichier écoute les événements **au niveau du document** et réagit aux attributs `data-*` présents dans le HTML. Avantages : aucun script dans les vues (compatible avec la CSP stricte), et un comportement s'ajoute à n'importe quel élément en lui donnant simplement l'attribut.

`resources/js/app.js` — lignes 1 à 45

```
/*
 * JavaScript minimal (aucune dépendance) — compatible avec la CSP stricte (aucun script en ligne).
 *  - data-confirm : demande de confirmation avant les opérations sensibles ;
 *  - data-dialog-open / data-dialog-close : ouverture des boîtes de dialogue <dialog> ;
 *  - data-filtre-liste : filtre texte d'une liste de cases à cocher ;
 *  - data-tout-cocher : sélection groupée ;
 *  - état de chargement des boutons à la soumission (évite les doubles envois) ;
 *  - menu latéral : tiroir sur mobile, réduction en icônes sur bureau (préférence mémorisée) ;
 *  - affichage / masquage du mot de passe.
 */
document.addEventListener('submit', (e) => {
    const form = e.target;
    const msg = form.dataset.confirm || e.submitter?.dataset.confirm;
    if (msg && !window.confirm(msg)) {
        e.preventDefault();
        return;
    }
    if (form.method.toLowerCase() === 'post' && !form.hasAttribute('data-sans-chargement')) {
        const b = e.submitter || form.querySelector('[type=submit]');
        if (b) {
            setTimeout(() => { b.disabled = true; b.dataset.texte = b.textContent; b.textContent =
'Traitement…'; }, 0);
        }
    }
});

document.addEventListener('click', (e) => {
    const o = e.target.closest('[data-dialog-open]');
    if (o) {
        document.getElementById(o.dataset.dialogOpen)?.showModal();
    }
    const c = e.target.closest('[data-dialog-close]');
    if (c) {
        c.closest('dialog')?.close();
    }
    if (e.target.closest('[data-menu-ouvrir]')) {
        document.body.classList.add('menu-ouvert');
        document.querySelector('#menu-lateral a, #menu-lateral button')?.focus();
    }
    if (e.target.closest('[data-menu-fermer]')) {
        document.body.classList.remove('menu-ouvert');
    }
    if (e.target.closest('[data-menu-reduire]')) {
        const reduit = document.documentElement.classList.toggle('menu-reduit');
        try { localStorage.setItem('inps.menuReduit', reduit ? '1' : '0'); } catch (err) { /* stockage
indisponible */ }
    }
```

Attribut HTML	Effet
`data-confirm="Texte ?"`	Demande une confirmation avant d'envoyer le formulaire.
(tout formulaire `POST`)	Désactive le bouton et affiche « Traitement… » pour éviter un double envoi ; `data-sans-chargement` désactive ce comportement.
`data-dialog-open="id"` / `data-dialog-close`	Ouvre / ferme une boîte de dialogue `<dialog>`.
`data-menu-ouvrir`, `data-menu-fermer`	Ouvre / ferme le menu latéral sur mobile.
`data-menu-reduire`	Réduit le menu en icônes sur ordinateur ; le choix est mémorisé (`localStorage`).
`data-basculer-mdp="id"`	Affiche / masque le mot de passe.
`data-filtre-liste="#liste"` + `data-filtre-texte`	Filtre une liste de cases à cocher pendant la frappe.
`data-tout-cocher=".selecteur"`	Coche / décoche toutes les cases visibles.

2.15 La compilation : Vite

`vite.config.js` — lignes 1 à 19

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';
import tailwindcss from '@tailwindcss/vite';

// Aucune police ni ressource distante : l'application fonctionne sans accès Internet.
export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css', 'resources/js/app.js'],
            refresh: true,
        }),
        tailwindcss(),
    ],
    server: {
        watch: {
            ignored: ['**/storage/framework/views/**'],
        },
    },
});
```

- `laravel({ input: [...] })` : les deux fichiers d'entrée.
- `tailwindcss()` : le module qui génère le CSS de Tailwind.
- `npm run build` produit `public/build/manifest.json` et `public/build/assets/app-XXXX.css` / `app-XXXX.js`. Le suffixe `XXXX` change à chaque modification : le navigateur ne peut pas garder une ancienne version en cache.
- `npm run dev` lance un serveur de développement qui recompile instantanément et recharge la page à chaque enregistrement ; il crée le fichier `public/hot` (supprimé à l'arrêt). Pendant que `npm run dev` tourne, `php artisan serve` doit tourner aussi, dans une autre fenêtre.

Les dépendances JavaScript sont déclarées dans `package.json` ; elles servent **uniquement** à compiler. En production, seul le dossier `public/build` est nécessaire : Node.js n'a pas besoin d'être installé sur le serveur si l'on copie un `public/build` déjà compilé (c'est le cas de l'archive livrée).

2.16 Authentification, rôles, permissions et sécurité

La connexion

Formulaire : `resources/views/auth/connexion.blade.php`. Traitement : `ConnexionController::store`.

`app/Http/Controllers/Auth/ConnexionController.php` — lignes 24 à 44

```
public function store(Request $request): RedirectResponse
{
    $cred = $request->validate([
        'email' => ['required', 'string', 'email', 'max:190'],
        'password' => ['required', 'string', 'max:255'],
    ]);
    $empreinte = substr(hash('sha256', mb_strtolower($cred['email'])), 0, 16); // aucun identifiant en
clair dans le journal

    if (! Auth::attempt(['email' => $cred['email'], 'password' => $cred['password'], 'actif' => true],
false)) {
        $existant = User::where('email', $cred['email'])->first();
        Audit::enregistrer('securite.connexion_echec', null, 'Échec de connexion',
['empreinte_identifiant' => $empreinte, 'compte_inactif' => (bool) ($existant && ! $existant->actif)],
$existant?->id);

        return back()->withInput($request->only('email'))->withErrors(['email' => 'Identifiants incorrects
ou compte désactivé.']);
    }
    $request->session()->regenerate(); // protection contre la fixation de session
    $u = $request->user();
    $u->forceFill(['derniere_connexion_le' => now(), 'derniere_connexion_ip' => $request->ip()])->save();
    Audit::enregistrer('securite.connexion', $u, 'Connexion réussie');

    return redirect()->intended(route('accueil'));
}
```

1. Validation simple des deux champs.
2. `Auth::attempt([...,'actif' => true])` : Laravel cherche l'utilisateur par e-mail **et** `actif = 1`, puis compare le mot de passe saisi avec l'empreinte stockée (algorithme *bcrypt* : la base ne contient jamais le mot de passe en clair). Le second argument `false` signifie : pas de « se souvenir de moi ».
3. En cas d'échec : une trace est écrite dans l'audit avec une **empreinte** de l'e-mail (pas l'e-mail lui-même) et le message reste volontairement vague (« Identifiants incorrects ou compte désactivé ») pour ne pas révéler si un compte existe.
4. En cas de succès : `session()->regenerate()` change l'identifiant de session (protection contre la *fixation de session*), la date et l'IP de dernière connexion sont enregistrées, puis `redirect()->intended(...)` renvoie vers la page initialement demandée, ou vers `accueil`.

La route de connexion est limitée par `throttle:connexion` (5 essais par minute pour un même e-mail depuis une même adresse IP). Au-delà, Laravel répond « Trop de tentatives » ; pendant le développement, `php artisan cache:clear` remet le compteur à zéro.

La page `accueil` (`TableauBordController::accueil`) oriente chaque profil vers son écran principal : tableau de bord, besoins, finances ou espace personnel.

Mots de passe

- **Création du premier compte** : il n'existe aucun compte par défaut. Le premier administrateur technique est créé en console avec `php artisan inps:creer-admin` (saisie masquée, politique de mot de passe appliquée).

- **Réinitialisation par l'administrateur** (`UtilisateurController::reinitialiserMotDePasse`) : un mot de passe temporaire de 16 caractères est généré, affiché **une seule fois** à l'administrateur, toutes les sessions de l'utilisateur sont fermées et `doit_changer_mot_de_passe` est mis à vrai.
- **Changement par l'utilisateur** (`MotDePasseController::update`) : l'ancien mot de passe est vérifié (`current_password`), le nouveau doit être différent et respecter la politique ; les autres sessions ouvertes de l'utilisateur sont supprimées.
- **« Mot de passe oublié » par e-mail** : implémenté avec le mécanisme standard de Laravel (lien à usage unique, durée limitée). Il ne fonctionne que si un serveur SMTP est configuré dans `.env` ; sans SMTP, la page invite à contacter l'administrateur. L'application n'affirme jamais avoir envoyé un e-mail qu'elle n'a pas pu remettre.

Gestion des comptes et des rôles

`app/Http/Controllers/Admin/UtilisateurController.php` contient trois protections contre l'**élévation de privilèges** :

`app/Http/Controllers/Admin/UtilisateurController.php` — lignes 146 à 155

```
private function verifierRolesAttribuables(User $acteur, array $roleIds): void
{
    if (! $roleIds || $acteur->aPermission('roles.gerer')) {
        return;
    }
    $sensibles = Role::whereIn('id', $roleIds)->whereHas('permissions', fn ($q) => $q->whereIn('code',
self::PERMISSIONS_ADMIN))->pluck('nom');
    if ($sensibles->isNotEmpty()) {
        abort(403, 'Seul un détenteur de la permission « Gérer les rôles » peut attribuer ou retirer un
rôle d\'administration ('.$sensibles->implode(', ').').');
    }
}
```

Un gestionnaire de comptes qui n'a pas `roles.gerer` ne peut pas attribuer un rôle contenant des permissions d'administration.

`app/Http/Controllers/Admin/UtilisateurController.php` — lignes 157 à 167

```
private function protegerDernierAdmin(User $u, array $nouveauxRoles): void
{
    $admin = Role::where('code', 'admin_technique')->first();
    if (! $admin || ! $u->roles()->whereKey($admin->id)->exists() || in_array($admin->id, $nouveauxRoles,
true)) {
        return;
    }
    $autres = $admin->users()->where('users.actif', true)->where('users.id', '!=', $u->id)->count();
    if ($autres === 0) {
        abort(422, 'Impossible : ce compte est le dernier administrateur technique actif.');
    }
}
```

On ne peut pas retirer son rôle au **dernier** administrateur technique actif : l'application ne peut pas se retrouver sans administrateur.

Dans `RoleController::update`, un administrateur ne peut pas modifier un rôle **qu'il détient lui-même** (il ne peut donc pas s'ajouter des droits), et chaque modification des permissions d'un rôle est tracée (permissions ajoutées et retirées) :

`app/Http/Controllers/Admin/RoleController.php` — lignes 35 à 48

```
public function update(Request $r, Role $role): RedirectResponse
{
    if ($r->user()->roles()->whereKey($role->id)->exists()) {
```

```
        return back()->with('erreur', 'Vous ne pouvez pas modifier un rôle que vous détenez (prévention de
l\'auto-attribution de droits).');
    }
    $d = $r->validate(['nom' => ['required', 'string', 'max:150'], 'description' => ['nullable', 'string',
'max:1000'], 'permissions' => ['nullable', 'array'], 'permissions.*' => ['integer',
'exists:permissions,id']]);
    $avant = $role->permissions()->pluck('code')->all();
    $role->update(['nom' => $d['nom'], 'description' => $d['description'] ?? null]);
    $role->permissions()->sync($d['permissions'] ?? []);
    $apres = $role->permissions()->pluck('code')->all();
    Audit::enregistrer('role.permissions', $role, "Modification des permissions du rôle {$role->code}",
['ajoutees' => array_values(array_diff($apres, $avant)), 'retirees' => array_values(array_diff($avant,
$apres))]);

    return back()->with('succes', 'Permissions mises à jour.');
}
```

Le journal d'audit infalsifiable

Le modèle `AuditLog` refuse toute modification au niveau PHP :

`app/Models/AuditLog.php` — lignes 28 à 32

```
protected static function booted(): void
{
    static::updating(fn () => throw new \LogicException('Le journal d\'audit n\'est pas modifiable.'));
    static::deleting(fn () => throw new \LogicException('Le journal d\'audit n\'est pas supprimable.'));
}
```

et la migration crée deux **déclencheurs MySQL** qui refusent tout `UPDATE` ou `DELETE` sur la table, même en SQL direct :

`database/migrations/2026_01_01_000600_create_finances_documents_audit_tables.php` — lignes 94 à 106

```
// Protection du journal contre les modifications ordinaires (MySQL)
// Si le compte MySQL de l'application n'a pas le droit de créer des déclencheurs
// (binlog actif sans SUPER), la migration continue : l'administrateur de base exécute
// alors database/sql/protection_audit.sql (voir docs/SECURITE.md). La commande
// « php artisan inps:sante » signale l'absence des déclencheurs.
if (DB::getDriverName() === 'mysql') {
    try {
        DB::unprepared("CREATE TRIGGER audit_logs_no_update BEFORE UPDATE ON audit_logs FOR EACH ROW
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Journal d audit non modifiable'");
        DB::unprepared("CREATE TRIGGER audit_logs_no_delete BEFORE DELETE ON audit_logs FOR EACH ROW
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Journal d audit non supprimable'");
    } catch (QueryException $e) {
        fwrite(STDERR, "AVERTISSEMENT : déclencheurs de protection du journal d'audit non créés (".$e-
>getCode()."). Exécuter database/sql/protection_audit.sql avec un compte administrateur MySQL.\n");
    }
```

Certains serveurs MySQL refusent la création de déclencheurs à un compte ordinaire quand la journalisation binaire est active (c'est le cas par défaut de MySQL 8). La migration continue alors avec un avertissement ; il faut ensuite exécuter `database/sql/protection_audit.sql` avec le compte `root`, ou autoriser temporairement l'opération (`SET GLOBAL log_bin_trust_function_creators = 1;`) puis lancer `php artisan inps:proteger-audit`. La commande `php artisan inps:sante` indique si la protection est active.

Les documents déposés

`app/Services/DocumentService.php` — lignes 39 à 76

```
public function deposer(Model $objet, UploadedFile $fichier, array $meta, User $user): Document
{
    $ext = strtolower($fichier->getClientOriginalExtension());
    $mimeReel = (new \finfo(FILEINFO_MIME_TYPE))->file($fichier->getRealPath()) ?: 'application/octet-
```

```
stream';
    if (! isset(self::TYPES_AUTORISES[$ext]) || ! in_array($mimeReel, self::TYPES_AUTORISES[$ext], true))
{
        throw new RegleMetierException("Type de fichier refusé (extension « {$ext} », contenu détecté «
{$mimeReel} »).");
    }
    if ($fichier->getSize() > self::tailleMaxKo() * 1024) {
        throw new RegleMetierException('Fichier trop volumineux (maximum '.self::tailleMaxKo().' Ko).');
    }
    $chemin = 'documents/'.now()->format('Y/m').'/'.Str::uuid()->toString().'.'.$ext;
    Storage::disk('prive')->putFileAs(dirname($chemin), $fichier, basename($chemin));

    try {
        return DB::transaction(function () use ($objet, $fichier, $meta, $user, $chemin, $mimeReel) {
            $d = new Document([
                'titre' => $meta['titre'] ?: $fichier->getClientOriginalName(),
                'categorie_id' => $meta['categorie_id'] ?? null,
                'confidentiel' => (bool) ($meta['confidentiel'] ?? false),
            ]);
            $d->documentable()->associate($objet);
            $d->nom_original = mb_substr(basename($fichier->getClientOriginalName()), 0, 255);
            $d->chemin = $chemin;
            $d->mime = $mimeReel;
            $d->taille = $fichier->getSize();
            $d->sha256 = hash_file('sha256', Storage::disk('prive')->path($chemin));
            $d->depose_par = $user->id;
            $d->save();
            Audit::enregistrer('document.depot', $d, "Dépôt du document « {$d->titre} »", ['rattachement'
=> $objet->getMorphClass().'#'.$objet->getKey(), 'confidentiel' => $d->confidentiel]);

            return $d;
        });
    } catch (\Throwable $e) {
        Storage::disk('prive')->delete($chemin); // pas de fichier orphelin

        throw $e;
    }
}
```

- L'extension **et** le contenu réel du fichier (détecté par `finfo`, indépendamment du nom) doivent correspondre à un type autorisé : un exécutable renommé en `.pdf` est refusé.
- Le fichier est rangé sur le disque **privé** (`storage/app/private/documents/AAAA/MM/`) sous un nom aléatoire (UUID) ; il n'est jamais accessible par une URL directe.
- Une empreinte SHA-256 est conservée pour le contrôle d'intégrité.
- Si l'enregistrement en base échoue, le fichier copié est supprimé (pas de fichier orphelin).
- Le téléchargement passe par `DocumentController::telecharger`, qui vérifie la policy (`DocumentPolicy`) : il faut pouvoir voir l'objet auquel le document est rattaché, et détenir `documents.confidentiels` pour un document confidentiel (chaque téléchargement confidentiel est journalisé).

Synthèse des protections

Menace	Protection dans le projet	Où
Accès sans être connecté	middleware `auth` sur toutes les routes métier	`routes/web.php`
Accès à un module non autorisé	middleware `permission:` (refus par défaut)	`ExigerPermission`
Accès à l'enregistrement d'un autre service	policies + scopes `visiblesPar`	`app/Policies`, modèles
Falsification de requête (CSRF)	jeton `@csrf` vérifié par `PreventRequestForgery`	toutes les vues de formulaire

Menace	Protection dans le projet	Où
Injection SQL	requêtes Eloquent paramétrées ; `addcslashes` pour `LIKE`	partout
Injection de script (XSS)	échappement `{{ }}` + CSP stricte sans script en ligne	vues, `EntetesSecurite`
Affectation de masse	`$fillable` ; données validées uniquement	modèles, contrôleurs
Force brute sur la connexion	`throttle:connexion`	`AppServiceProvider`
Mots de passe faibles	politique `Password::defaults`	`AppServiceProvider`
Vol de session	régénération à la connexion, cookies chiffrés, `HttpOnly`, `SameSite=Lax`, `Secure` en production, durée 30 min	`ConnexionController`, `config/session.php`
Fichier malveillant	type réel vérifié, disque privé, nom aléatoire	`DocumentService`
Formules dans les exports Excel	les cellules commençant par `=`, `+`, `-`, `@` sont neutralisées	`ExportService::excel`
Effacement des traces	audit en ajout seul + déclencheurs MySQL	`AuditLog`, migration
Élévation de privilèges	rôles d'administration réservés, rôle détenu non modifiable, dernier admin protégé	contrôleurs `Admin/`
Affichage dans un cadre (clickjacking)	`X-Frame-Options: DENY`, `frame-ancestors 'none'`	`EntetesSecurite`
Fuite d'informations en cas d'erreur	`APP_DEBUG=false` en production ; message générique	`.env`

Les tests `tests/Feature/AuthentificationTest.php`, `tests/Feature/AutorisationTest.php` et `tests/Feature/AuditSecuriteTest.php` vérifient ces protections à chaque exécution de la suite de tests.

2.17 Parcours complet : du clic à la base de données

Rassemblons tout sur un cas réel : **un chef de service déclare un besoin de formation et le soumet à la DRH**.

Étape 1 — L'utilisateur ouvre le formulaire

Il clique sur *Besoins › Nouvelle demande*. Le lien pointe vers `route('besoins.create')`, soit `GET /besoins/creer`.

- Middlewares : session, `auth` (connecté ?), `VerifierCompteActif`, puis `permission:besoins.voir_tous|besoins.voir_service` (le chef de service a `besoins.voir_service`).
- Contrôleur : `BesoinController::create` → `authorize('create', BesoinFormation::class)` → la policy vérifie `besoins.creer`.
- `listes()` prépare les menus déroulants : années ouvertes, services autorisés (seulement le sien pour un chef de service), priorités, types, domaines, agents de son service.
- Vue : `resources/views/besoins/form.blade.php`, dans la coquille `x-layout`.

Étape 2 — Il remplit et clique sur « Enregistrer et soumettre »

Le navigateur envoie `POST /besoins` avec les champs du formulaire, `soumettre=1` et le jeton `_token`. Le JavaScript (`app.js`) demande d'abord la confirmation (`data-confirm`), puis désactive le bouton pour éviter un double envoi.

Étape 3 — Le serveur traite la requête

1. `PreventRequestForgery` compare `_token` au jeton de la session : sinon, erreur 419.
2. `ExigerPermission` vérifie la permission du groupe de routes.
3. `BesoinRequest::authorize()` vérifie `besoins.creer` ; `BesoinRequest::rules()` valide chaque champ. Si l'intitulé manque, retour immédiat au formulaire avec le message « Le champ intitulé est obligatoire. » et la saisie conservée.
4. `BesoinController::store` appelle `BesoinService::creer` (statut *Brouillon*), puis, comme `soumettre=1`, `BesoinService::transition($b, 'soumission', ...)` (statut *Soumise*).
5. Redirection `302` vers `GET /besoins/{id}` avec le message flash « Demande enregistrée et soumise. ».

Étape 4 — Les requêtes SQL réellement exécutées

Les requêtes ci-dessous n'ont pas été écrites à la main : elles ont été **enregistrées** pendant l'exécution réelle de ce scénario (dans une base de test, avec le mécanisme `DB::listen` de Laravel). Les valeurs des paramètres ont été réinsérées pour la lisibilité.

```
-- POST /besoins (enregistrer et soumettre)
/* 01 */ select `permissions`.`code` from `permissions` inner join `permission_role` on `permissions`.`id`
= `permission_role`.`permission_id` inner join `role_user` on `role_user`.`role_id` =
`permission_role`.`role_id` where `role_user`.`user_id` = 1;
/* 02 */ select count(*) as `aggregate` from `annees_reference` where `annee` = 2026;
/* 03 */ select count(*) as `aggregate` from `services` where `id` = 1 and `actif` = 1;
/* 04 */ select `reference` from `besoins_formation` where `reference` like 'BES-2026-%' order by
`reference` desc limit 1 for update;
/* 05 */ insert into `besoins_formation` (`annee`, `service_id`, `intitule`, `nb_participants`,
`reference`, `statut`, `auteur_id`, `updated_at`, `created_at`) values (2026, 1, 'Excel avancé pour le
suivi des dossiers', 6, 'BES-2026-0001', 'brouillon', 1, '2026-10-21 07:51:46', '2026-10-21 07:51:46');
/* 06 */ select * from `besoin_agent` where `besoin_agent`.`besoin_id` = 1;
/* 07 */ insert into `decisions` (`action`, `statut_apres`, `user_id`, `decidable_id`, `decidable_type`,
`created_at`) values ('creation', 'brouillon', 1, 1, 'besoin', '2026-10-21 07:51:46');
/* 08 */ insert into `audit_logs` (`user_id`, `action`, `objet_type`, `objet_id`, `description`,
`contexte`, `ip`, `user_agent`, `created_at`) values (1, 'besoin.creation', 'besoin', 1, 'Création du
besoin BES-2026-0001', NULL, '127.0.0.1', 'Symfony', '2026-10-21 07:51:46');
/* 09 */ select * from `annees_reference` where `annee` = 2026 limit 1;
/* 10 */ select * from `besoins_formation` where `besoins_formation`.`id` = 1 and
`besoins_formation`.`deleted_at` is null limit 1 for update;
/* 11 */ update `besoins_formation` set `statut` = 'soumise', `soumis_le` = '2026-10-21 07:51:46',
`besoins_formation`.`updated_at` = '2026-10-21 07:51:46' where `id` = 1;
/* 12 */ insert into `decisions` (`action`, `statut_avant`, `statut_apres`, `commentaire`, `user_id`,
`decidable_id`, `decidable_type`, `created_at`) values ('soumission', 'brouillon', 'soumise', NULL, 1, 1,
'besoin', '2026-10-21 07:51:46');
/* 13 */ insert into `audit_logs` (`user_id`, `action`, `objet_type`, `objet_id`, `description`,
`contexte`, `ip`, `user_agent`, `created_at`) values (1, 'besoin.soumission', 'besoin', 1, 'BES-2026-0001
: Brouillon → Soumise', NULL, '127.0.0.1', 'Symfony', '2026-10-21 07:51:46');
/* 14 */ select * from `users` where `actif` = 1 and exists (select * from `roles` inner join `role_user`
on `roles`.`id` = `role_user`.`role_id` where `users`.`id` = `role_user`.`user_id` and exists (select *
from `permissions` inner join `permission_role` on `permissions`.`id` = `permission_role`.`permission_id`
where `roles`.`id` = `permission_role`.`role_id` and `code` = 'besoins.instruire'));

-- GET /besoins/1 (redirection vers la fiche)
/* 15 */ select * from `besoins_formation` where `id` = 1 and `besoins_formation`.`deleted_at` is null
limit 1;
/* 16 */ select * from `services` where `services`.`id` in (1);
/* 17 */ select * from `directions` where `directions`.`id` in (1);
/* 18 */ select * from `users` where `users`.`id` in (1);
/* 19 */ select `agents`.*, `besoin_agent`.`besoin_id` as `pivot_besoin_id`, `besoin_agent`.`agent_id` as
`pivot_agent_id` from `agents` inner join `besoin_agent` on `agents`.`id` = `besoin_agent`.`agent_id`
where `besoin_agent`.`besoin_id` in (1);
/* 20 */ select * from `decisions` where `decisions`.`decidable_id` in (1) and
`decisions`.`decidable_type` = 'besoin' order by `id` desc;
/* 21 */ select * from `users` where `users`.`id` in (1);
/* 22 */ select * from `documents` where `documents`.`documentable_id` in (1) and
`documents`.`documentable_type` = 'besoin' and `documents`.`deleted_at` is null;
```

```
/* 23 */ select * from `nomenclatures` where `type` = 'categorie_document' and `actif` = 1 order by
`ordre` asc, `libelle` asc;
/* 24 */ select `cle`, `valeur`, `type` from `parametres`;
/* 25 */ select count(*) as `aggregate` from `notifications` where `notifications`.`notifiable_type` =
'user' and `notifications`.`notifiable_id` = 1 and `notifications`.`notifiable_id` is not null and
`read_at` is null;
/* 26 */ select `roles`.*, `role_user`.`user_id` as `pivot_user_id`, `role_user`.`role_id` as
`pivot_role_id`, `role_user`.`attribue_le` as `pivot_attribue_le`, `role_user`.`attribue_par` as
`pivot_attribue_par` from `roles` inner join `role_user` on `roles`.`id` = `role_user`.`role_id` where
`role_user`.`user_id` = 1;
```

Lecture, requête par requête :

N°	Origine dans le code	Rôle
01	`User::permissionCodes` (appelé par le middleware `permission:`)	Charge toutes les permissions de l'utilisateur en une requête ; le résultat est réutilisé pendant toute la requête HTTP.
02–03	Règles `Rule::exists(...)` de `BesoinRequest`	L'année existe ; le service existe et est actif.
04	`GenerateurReference::suivante`	Plus grande référence `BES-2026-…`, lue **sous verrou** (`FOR UPDATE`).
05	`$b->save()` dans `BesoinService::creer`	Insertion du besoin, statut `brouillon`. Seuls les champs `$fillable` remplis et ceux fixés par le service figurent dans la requête.
06	`$b->agents()->sync($agentIds)`	Lecture de la table pivot avant mise à jour (aucun agent nommé ici, donc aucun ajout).
07	`$b->decisions()->create(...)`	Historique : « création ». Remarque `decidable_type = 'besoin'` : c'est l'alias défini par `enforceMorphMap`.
08	`Audit::enregistrer('besoin.creation', ...)`	Journal d'audit.
09	`BesoinService::transition`	L'année de la demande est-elle encore ouverte ?
10	`lockForUpdate()`	Relecture de la demande verrouillée.
11	`$b->save()`	Passage au statut `soumise`, date de soumission. Seules les colonnes **modifiées** sont envoyées.
12–13	décision + audit	Historique « soumission » (brouillon → soumise) et journal.
14	`NotificationService::utilisateursAvecPermission('besoins.instruire')`	Recherche des destinataires de la notification. Dans ce scénario de test, aucun compte DRH n'existait, donc aucune notification n'a été insérée ; dans l'application réelle, une ligne `INSERT INTO notifications` suit pour chaque instructeur.
15	liaison de modèle `{besoin}`	Chargement de la demande pour la fiche (le filtre `deleted_at is null` vient de `SoftDeletes`).
16–22	`$besoin->load([...])` dans `BesoinController::show`	Une requête **par relation**, quel que soit le nombre de lignes : c'est l'intérêt du chargement anticipé.
23	`Nomenclature::deType('categorie_document')`	Liste des catégories pour le dépôt de document.
24	`Parametre::valeur(...)`	Lecture de tous les paramètres (puis mise en cache).
25	`layout.blade.php`	Nombre de notifications non lues (pastille de la cloche).
26	`layout.blade.php` (`$u->roles`)	Rôles affichés dans le menu du compte.

Les ordres `BEGIN` / `COMMIT` des transactions n'apparaissent pas dans ce relevé, mais les requêtes 04 à 08, puis 10 à 14, s'exécutent chacune dans une transaction. Dans l'application réelle, s'ajoutent aussi la lecture et l'écriture de la session (table `sessions`) et le chargement de l'utilisateur connecté (`select * from users where id = ...`) ; le scénario de test les remplace par une session en mémoire.

Étape 5 — L'affichage de la fiche

`resources/views/besoins/show.blade.php` reçoit `$b` et affiche :

- l'en-tête avec le badge de statut (`<x-badge :statut="$b->statut" />` → libellé et couleur depuis `StatutBesoin`) ;
- les boutons autorisés uniquement (`@can('update', $b)`, `@can('instruire', $b)`) : pour le chef de service, la demande soumise n'est plus modifiable, donc les boutons « Modifier » et « Soumettre » disparaissent ;
- l'historique des décisions (`<x-decisions>`), les pièces jointes (`<x-documents>`) ;
- le message flash vert, affiché par `<x-messages>` dans la coquille.

Étape 6 — Ce que voit la DRH

L'instructeur reçoit une notification (cloche), ouvre la fiche, et voit le bloc « Instruction de la demande » avec les boutons « Mettre en examen », « Accepter », « Demander des compléments », « Rejeter ». Chacun envoie `POST /besoins/{id}/transition` avec `action=...` et, si nécessaire, un `commentaire` saisi dans une boîte de dialogue (`x-dialogue-motif`). Le même `BesoinService::transition` applique la machine à états.

> **À retenir**
>
> Pour comprendre **n'importe quel écran**, refais ce parcours : URL → `php artisan route:list --path=...` → méthode du contrôleur → policy et validation → service → modèle → vue. En cinq fichiers, tu as la totalité du comportement.

Partie 3 — La base de données

3.1 Les migrations : la structure de la base sous forme de code

La base n'est jamais créée « à la main ». Sa structure est décrite par des **migrations**, des fichiers PHP exécutés dans l'ordre de leur nom par `php artisan migrate`. Laravel note chaque migration appliquée dans la table `migrations` : une migration ne s'exécute qu'une fois.

Fichier (`database/migrations/`)	Contenu
`0001_01_01_000001_create_cache_table.php`	Tables `cache`, `cache_locks` (cache de Laravel en base : compteurs de tentatives, paramètres).
`0001_01_01_000002_create_jobs_table.php`	Tables `jobs`, `job_batches`, `failed_jobs` (files d'attente, non utilisées : `QUEUE_CONNECTION=sync`).
`2026_01_01_000100_create_organisation_tables.php`	Directions, services, domaines, nomenclatures, années de référence, paramètres.
`2026_01_01_000200_create_agents_users_rbac_tables.php`	Agents, utilisateurs, sessions, réinitialisation des mots de passe, rôles, permissions et tables de liaison.
`2026_01_01_000300_create_besoins_plans_tables.php`	Besoins, agents concernés, circuits et étapes de validation, plans, versions, lignes, décisions.
`2026_01_01_000400_create_organismes_actions_sessions_tables.php`	Organismes, contacts, conventions, formateurs, actions, sessions, changements de session, séances.
`2026_01_01_000500_create_participation_tables.php`	Inscriptions et leurs événements, présences et corrections, évaluations (satisfaction, acquis, impact), appréciations des prestataires, attestations.
`2026_01_01_000600_create_finances_documents_audit_tables.php`	Dépenses, documents, notifications, rappels envoyés, envois d'e-mails, journal d'audit (et ses déclencheurs), imports.
`2026_10_20_000700_identite_visuelle_inps.php`	Version 1.1.0 : couleur #003B73 et paramètre de la photographie du siège.

Chaque migration a deux méthodes : `up()` (appliquer) et `down()` (annuler, avec `php artisan migrate:rollback`). Voici la plus simple, qui crée les directions et les services :

`database/migrations/2026_01_01_000100_create_organisation_tables.php` — lignes 12 à 30

```
public function up(): void
{
    Schema::create('directions', function (Blueprint $table) {
        $table->id();
        $table->string('code', 30)->unique();
        $table->string('nom', 190);
        $table->boolean('actif')->default(true)->index();
        $table->timestamps();
    });

    Schema::create('services', function (Blueprint $table) {
        $table->id();
        $table->foreignId('direction_id')->constrained('directions')->restrictOnDelete();
        $table->string('code', 30)->unique();
        $table->string('nom', 190);
        $table->boolean('actif')->default(true)->index();
        $table->timestamps();
        $table->index(['direction_id', 'actif']);
    });
```

Instruction	Effet en MySQL
`$table->id()`	Colonne `id` BIGINT non signé, auto-incrémentée, **clé primaire**.
`$table->string('code', 30)->unique()`	`VARCHAR(30)` + index d'unicité : deux directions ne peuvent pas avoir le même code.
`$table->boolean('actif')->default(true)->index()`	`TINYINT(1)` valant 1 par défaut, indexé pour accélérer les filtres.
`$table->foreignId('direction_id')->constrained('directions')`	Colonne `direction_id` + **clé étrangère** vers `directions.id` : impossible d'enregistrer un service pointant vers une direction inexistante.
`->restrictOnDelete()`	Impossible de supprimer une direction qui a encore des services.
`->cascadeOnDelete()`	Supprimer le parent supprime les enfants (utilisé pour des détails sans existence propre : séances d'une session, lignes d'un import…).
`->nullOnDelete()`	Supprimer le parent met la colonne à `NULL` (ex. un domaine supprimé laisse les besoins sans domaine).
`$table->timestamps()`	Colonnes `created_at` et `updated_at`, remplies automatiquement par Eloquent.
`$table->softDeletes()`	Colonne `deleted_at` pour la suppression logique.
`$table->index(['direction_id', 'actif'])`	Index composé pour une recherche fréquente.
`$table->morphs('documentable')`	Deux colonnes `documentable_type` + `documentable_id` (relation polymorphe) et leur index.

Attention

Ne modifie jamais une migration déjà appliquée sur une base qui contient des données réelles : Laravel ne la rejouera pas, et les autres installations seraient incohérentes. Pour changer la structure, crée **une nouvelle migration** (voir la partie 6), comme cela a été fait pour la version 1.1.0 avec `2026_10_20_000700_identite_visuelle_inps.php`.

3.2 Les tables et leur rôle

La base comprend **53 tables**. Le catalogue complet (chaque colonne, son type, ses clés) est en annexe C. Vue d'ensemble par domaine :

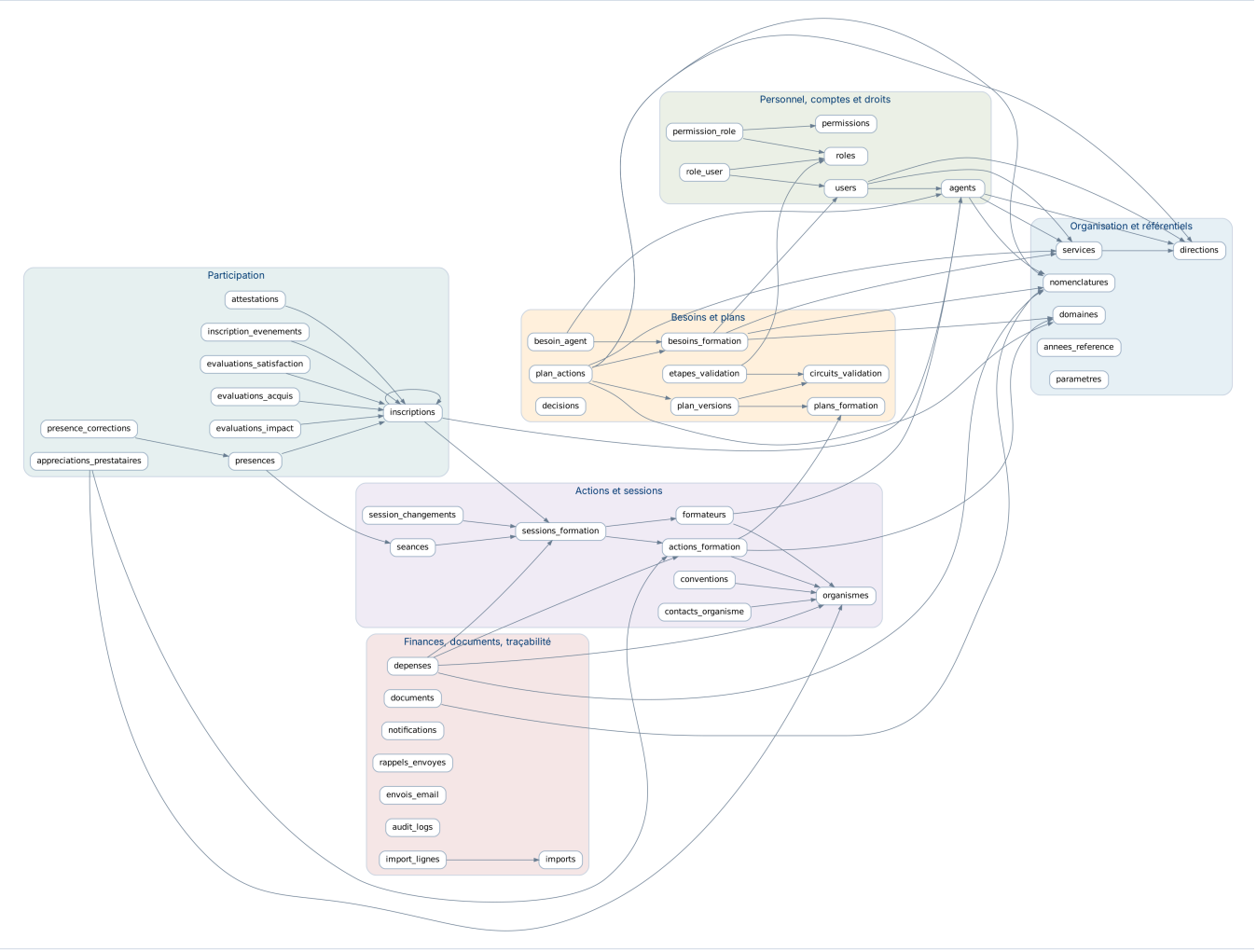


Figure 3 — Les tables regroupées par domaine et leurs clés étrangères (généré depuis la base). Les tables polymorphes (`documents`, `decisions`, `notifications`) sont reliées par type + identifiant, sans clé étrangère.

Domaine	Tables	Rôle
Organisation et référentiels	`directions`, `services`, `domaines`, `nomenclatures`, `annees_reference`, `parametres`	Structure de l'INPS ; listes paramétrables (priorités, types de formation, statuts professionnels, axes stratégiques, catégories de dépenses et de documents — toutes dans `nomenclatures`, distinguées par la colonne `type`) ; années ouvertes au recensement ; paramètres métier.
Personnel, comptes, droits	`agents`, `users`, `roles`, `permissions`, `permission_role`, `role_user`	Le référentiel du personnel est **séparé** des comptes de connexion : un agent n'a pas forcément de compte ; un compte peut être lié à un agent (`users.agent_id`, unique).
Besoins et plans	`besoins_formation`, `besoin_agent`, `circuits_validation`, `etapes_validation`, `plans_formation`, `plan_versions`, `plan_actions`, `decisions`	Recensement, plan annuel versionné, circuit de validation configurable, historique des décisions.
Actions et sessions	`organismes`, `contacts_organisme`, `conventions`, `formateurs`, `actions_formation`, `sessions_formation`, `session_changements`, `seances`	Offre de formation et organisation concrète.

Domaine	Tables	Rôle
Participation	`inscriptions`, `inscription_evenements`, `presences`, `presence_corrections`, `evaluations_satisfaction`, `evaluations_acquis`, `evaluations_impact`, `appreciations_prestataires`, `attestations`	Qui participe, présences par séance, résultats, évaluations, attestations.
Finances, documents, traçabilité	`depenses`, `documents`, `notifications`, `rappels_envoyes`, `envois_email`, `audit_logs`, `imports`, `import_lignes`	Coûts, pièces jointes, notifications internes, anti-doublon des rappels, suivi des e-mails, journal d'audit, historique des imports.
Techniques (Laravel)	`migrations`, `sessions`, `cache`, `cache_locks`, `password_reset_tokens`, `jobs`, `job_batches`, `failed_jobs`	Fonctionnement interne du framework.

La chaîne métier principale

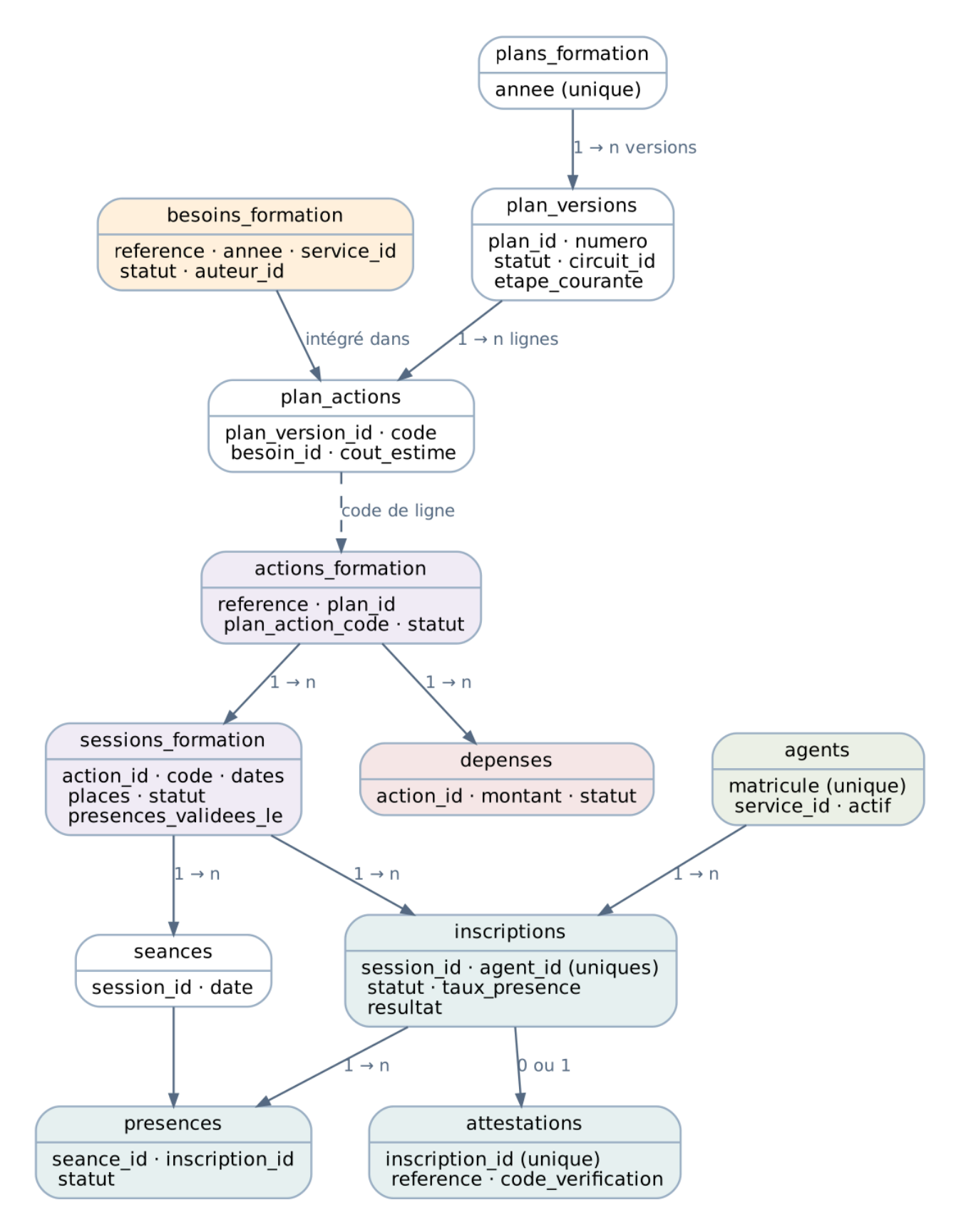


Figure 4 — Du besoin à l'attestation : les tables principales et leurs liens.

1. Un **besoin** accepté est intégré au plan : une ligne `plan_actions` est créée avec `besoin_id`.

2. Un **plan** (un par année : `plans_formation.annee` est unique) a plusieurs **versions** ; chaque version a ses **lignes**, identifiées par un **code** (`PA2026-001`) conservé d'une version à l'autre.
3. Une **action** de formation est rattachée à un plan et à un code de ligne (`plan_id` + `plan_action_code`) : c'est ce lien qui permet la comparaison prévu / réalisé.
4. Une action a des **sessions** ; une session a des **séances** (une par jour ouvré, générées automatiquement) et des **inscriptions**.
5. Une **inscription** relie une session et un agent (le couple est **unique** : un agent ne peut pas être inscrit deux fois à la même session) ; elle a une **présence** par séance, au plus une **attestation**.
6. Les **dépenses** sont rattachées à l'action (et facultativement à une session).

Les contraintes qui protègent les données

Les règles importantes sont garanties **par la base elle-même**, pas seulement par le code PHP :

Contrainte	Où	Ce qu'elle empêche
`UNIQUE(session_id, agent_id)`	`inscriptions`	Double inscription d'un agent à une session.
`UNIQUE(seance_id, inscription_id)`	`presences`	Deux présences pour la même personne à la même séance.
`UNIQUE(inscription_id)`	`attestations`, `evaluations_satisfaction`, `evaluations_acquis`	Deux attestations (ou deux évaluations) pour une même inscription.
`UNIQUE(annee)`	`plans_formation`	Deux plans pour la même année.
`UNIQUE(plan_version_id, code)` et `UNIQUE(plan_version_id, besoin_id)`	`plan_actions`	Deux lignes de même code, ou un besoin intégré deux fois, dans une version.
`UNIQUE(reference)`	besoins, actions, attestations	Références en double.
`UNIQUE(matricule)`	`agents`	Deux agents avec le même matricule.
`UNIQUE(cle)`	`rappels_envoyes`	Envoi en double d'un même rappel automatique.
clés étrangères `RESTRICT`	ex. `inscriptions.agent_id`, `besoins_formation.auteur_id`	Suppression d'un agent ou d'un utilisateur qui a un historique.
déclencheurs	`audit_logs`	Toute modification ou suppression du journal.

3.3 Les modèles Eloquent

Chaque table métier a son modèle dans `app/Models` (41 modèles). Laravel déduit le nom de la table du nom du modèle **en anglais** (`User` → `users`) ; comme les noms sont en français, la plupart des modèles précisent `protected $table`. Exemples :

Modèle	Table	Remarque
`BesoinFormation`	`besoins_formation`	`SoftDeletes`
`PlanFormation`, `PlanVersion`, `PlanAction`	`plans_formation`, `plan_versions`, `plan_actions`	
`ActionFormation`	`actions_formation`	

Modèle	Table	Remarque
`SessionFormation`	`sessions_formation`	le nom `sessions` est déjà pris par la table des sessions de connexion
`Inscription`, `Presence`, `Seance`	`inscriptions`, `presences`, `seances`	`$fillable` vide pour `Inscription` : tout est fixé par le service
`Document`	`documents`	`SoftDeletes`, colonnes `chemin` et `sha256` masquées (`$hidden`)
`AuditLog`	`audit_logs`	pas de `updated_at`, modification interdite

Les relations d'un modèle « carrefour », `SessionFormation` :

`app/Models/SessionFormation.php` — lignes 12 à 65

```
class SessionFormation extends Model
{
    protected $table = 'sessions_formation';

    protected $fillable = [
        'date_debut', 'date_fin', 'heure_debut', 'heure_fin', 'duree_heures', 'lieu', 'places',
        'liste_attente_active', 'formateur_id', 'bilan_pedagogique',
    ];

    /** Statuts d'inscription qui occupent une place. */
    public const STATUTS_OCCUPANT_PLACE = ['inscrit', 'convoque', 'confirme'];

    protected function casts(): array
    {
        return [
            'statut' => StatutSession::class,
            'date_debut' => 'date',
            'date_fin' => 'date',
            'places' => 'integer',
            'liste_attente_active' => 'boolean',
            'presences_validees_le' => 'datetime',
            'duree_heures' => 'decimal:1',
        ];
    }

    public function action(): BelongsTo
    {
        return $this->belongsTo(ActionFormation::class, 'action_id');
    }

    public function formateur(): BelongsTo
    {
        return $this->belongsTo(Formateur::class);
    }

    public function inscriptions(): HasMany
    {
        return $this->hasMany(Inscription::class, 'session_id');
    }

    public function seances(): HasMany
    {
        return $this->hasMany(Seance::class, 'session_id')->orderBy('date')->orderBy('heure_debut');
    }

    public function changements(): HasMany
    {
        return $this->hasMany(SessionChangement::class, 'session_id')->latest('id');
    }

    public function documents(): MorphMany
    {
```

```
        return $this->morphMany(Document::class, 'documentable');
    }
```

Les méthodes métier qui suivent dans ce modèle (`placesOccupees`, `placesRestantes`, `presencesValidees`, `participantsActifs`) sont réutilisées par les services et les vues. La constante `STATUTS_OCCUPANT_PLACE` définit en un seul endroit quels statuts d'inscription occupent une place : inscrit, convoqué, confirmé.

3.4 Les seeders : les données initiales

Un **seeder** insère des données. Il y en a trois (`database/seeders`) :

Seeder	Contenu	En production ?
`DatabaseSeeder`	Appelle seulement `ReferentielInitialSeeder` (commande `php artisan db:seed`).	Oui
`ReferentielInitialSeeder`	Les 39 permissions, les 8 rôles proposés, les nomenclatures génériques, les paramètres par défaut, un circuit de validation proposé **inactif**. Aucune donnée de personnel, aucun compte.	Oui
`DemoSeeder`	Jeu de démonstration **fictif** (directions, agents « DEMO… », comptes `demo.<rôle>@exemple.invalid`, plan, sessions…). Exige la variable `DEMO_PASSWORD`.	**Interdit** (il s'arrête en production)

`ReferentielInitialSeeder` est **idempotent** : on peut le relancer après une mise à jour. Il crée les permissions nouvelles (`updateOrCreate`) mais ne touche pas aux rôles existants, pour ne pas écraser les ajustements faits par l'administrateur :

`database/seeders/ReferentielInitialSeeder.php` — lignes 20 à 33

```
public function run(): void
{
    foreach (CataloguePermissions::PERMISSIONS as $code => [$module, $libelle, $sensible]) {
        Permission::updateOrCreate(['code' => $code], ['module' => $module, 'libelle' => $libelle,
'sensible' => $sensible]);
    }
    foreach (CataloguePermissions::ROLES as $code => [$nom, $desc, $perms]) {
        $role = Role::where('code', $code)->first();
        if (! $role) {
            $role = new Role(['code' => $code, 'nom' => $nom, 'description' => $desc]);
            $role->systeme = true;
            $role->save();
            $role->permissions()->sync(Permission::whereIn('code', $perms)->pluck('id'));
        }
    }
```

3.5 Consulter, ajouter, modifier et supprimer des données

Règle n° 1 : passer par l'application

Toute modification de données métier doit passer par **l'interface** ou par les **services**, jamais par une requête SQL directe. Le service applique les règles (statuts autorisés, contrôles, calculs), écrit l'historique et le journal d'audit. Une modification SQL directe contournerait tout cela et pourrait rendre les données incohérentes (par exemple un besoin « accepté » sans décision d'acceptation, ou un taux de présence qui ne correspond plus aux présences).

Consulter en SQL (lecture seule)

Pour une recherche ponctuelle, le client MySQL suffit. Sous Windows, il n'est pas dans le `PATH` ; on l'appelle avec son chemin complet :

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p --default-character-set=utf8mb4
inps_formation
```

Exemples de requêtes **de lecture** utiles :

```sql
-- Nombre de besoins par statut pour 2026
SELECT statut, COUNT(*) FROM besoins_formation WHERE annee = 2026 AND deleted_at IS NULL GROUP BY statut;

-- Historique de formation d'un agent
SELECT s.code, a.intitule, s.date_debut, i.statut, i.taux_presence, i.resultat
FROM inscriptions i
JOIN sessions_formation s ON s.id = i.session_id
JOIN actions_formation a ON a.id = s.action_id
JOIN agents ag ON ag.id = i.agent_id
WHERE ag.matricule = 'DEMO0006'
ORDER BY s.date_debut DESC;

-- Qui a fait quoi aujourd'hui (journal d'audit)
SELECT created_at, user_id, action, description FROM audit_logs WHERE created_at >= CURDATE() ORDER BY id
DESC;

-- Les rôles et permissions d'un utilisateur
SELECT r.code AS role, p.code AS permission
FROM users u JOIN role_user ru ON ru.user_id = u.id JOIN roles r ON r.id = ru.role_id
JOIN permission_role pr ON pr.role_id = r.id JOIN permissions p ON p.id = pr.permission_id
WHERE u.email = 'demo.chef_service@exemple.invalid';
```

Un outil graphique comme **MySQL Workbench** (installé avec MySQL sous Windows) permet aussi de parcourir les tables. Utilise-le en lecture.

Manipuler avec Eloquent depuis un script

Pour tester une requête Eloquent ou appeler un service hors de l'interface (sur une base de **développement**), on peut écrire un petit script qui démarre Laravel. Crée par exemple `C:\dev\essais\essai.php` (hors du projet) :

```php
<?php
// Démarre l'application Laravel puis exécute le code de test
require 'C:/dev/inps-formation-1.1.0/inps-formation/vendor/autoload.php';
$app = require 'C:/dev/inps-formation-1.1.0/inps-formation/bootstrap/app.php';
$app->make(Illuminate\Contracts\Console\Kernel::class)->bootstrap();

// --- Lecture ---
echo App\Models\BesoinFormation::where('statut', 'soumise')->count(), PHP_EOL;
$u = App\Models\User::where('email', 'demo.chef_service@exemple.invalid')->first();
echo $u->name, ' / service ', $u->service_id, PHP_EOL;
echo implode(', ', array_keys($u->permissionCodes())), PHP_EOL;
$a = App\Models\Agent::where('matricule', 'DEMO0006')->first();
echo $a->nom_complet, ' : ', $a->inscriptions()->count(), ' inscription(s)', PHP_EOL;
$s = App\Models\SessionFormation::first();
echo $s->code, ' places restantes ', $s->placesRestantes(), PHP_EOL;
echo App\Models\BesoinFormation::visiblesPar($u)->count(), ' besoin(s) visibles par le chef', PHP_EOL;

// --- Écriture : toujours par le service, qui applique les règles ---
$service = app(App\Services\BesoinService::class);
$b = $service->creer(['annee' => 2026, 'service_id' => $u->service_id, 'intitule' => 'Essai depuis la
console'], $u);
echo $b->reference, ' ', $b->statut->libelle(), PHP_EOL;
try {
    $service->transition($b, 'acceptation', $u);
} catch (App\Exceptions\RegleMetierException $e) {
    echo 'Refus : ', $e->getMessage(), PHP_EOL;
}
```

puis lance `php C:\dev\essais\essai.php`. Ce script a été exécuté sur une copie de la base de démonstration ; il a produit :

```
0
Démo — Chef de service / service 1
agents.voir_service, besoins.voir_service, besoins.creer, actions.voir, presences.saisir
EXEMPLE Oumar (démo) : 2 inscription(s)
ACT-2026-0001-S01 places restantes 6
4 besoin(s) visibles par le chef
BES-2026-0005 Brouillon
Refus : Action impossible : la demande est « Brouillon ».
```

La dernière ligne illustre la machine à états : on ne peut pas accepter un brouillon. Remarque aussi que le **service** ne vérifie pas les permissions de l'utilisateur pour cette transition : c'est le rôle du contrôleur (`authorize('instruire', ...)`). Un script appelant directement un service doit donc être réservé aux développeurs, sur une base de développement.

Astuce

L'outil interactif **Tinker** de Laravel n'est pas installé dans ce projet. Si tu souhaites l'utiliser sur ton poste de développement, la commande est `composer require --dev laravel/tinker`, puis `php artisan tinker`. Cette installation n'a pas pu être testée dans l'environnement de préparation de ce document ; le script ci-dessus fonctionne sans elle.

Supprimer

L'application **ne supprime presque rien**, par choix de traçabilité :

Donnée	Comportement
Agent	Désactivation logique (`actif = 0`, date et motif), jamais de suppression.
Utilisateur	Désactivation ; ses sessions sont fermées.
Besoin	Seul un **brouillon** peut être retiré (suppression logique, `deleted_at`).
Document	Retrait logique (`deleted_at`) ; le fichier reste sur le disque privé.
Session	Annulation avec motif ; les inscriptions passent au statut « annulé ».
Attestation	Annulation avec motif ; la ligne reste.
Journal d'audit	Ni modification ni suppression possible.
Ligne de plan	Supprimable uniquement dans une version modifiable (brouillon ou rejetée).

Aucune purge automatique n'est programmée : les durées de conservation sont des paramètres « à valider par l'INPS » (groupe *Conservation*).

3.6 Commandes utiles sur la base

Commande	Effet
`php artisan migrate`	Applique les migrations en attente (à lancer après chaque mise à jour du code).
`php artisan migrate:status`	Liste les migrations et indique si elles sont appliquées.
`php artisan migrate:rollback`	Annule le dernier lot de migrations (⚠ peut supprimer des colonnes et leurs données).
`php artisan db:seed`	Charge le référentiel initial (sans risque, idempotent).

Commande	Effet
`php artisan migrate:fresh --seed`	**Supprime toutes les tables** puis recrée la base vide : **à n'utiliser que sur une base de développement ou de test**, jamais sur des données réelles.
`php artisan db:show` / `php artisan db:table agents`	Résumé de la base / structure d'une table.

Partie 4 — Les fonctionnalités, module par module

Chaque module est présenté de la même façon : **fichiers concernés**, **fonctionnement technique**, **parcours des données**, **règles métier**, **erreurs possibles et correction**. Les messages d'erreur cités sont ceux du code (`RegleMetierException` ou validation). Dans le code, certains modules portent une lettre (B à L, N) issue du cahier des charges ; ces lettres sont reprises ici quand elles figurent dans le code.

4.1 Connexion, comptes et accueil

Fichiers	`app/Http/Controllers/Auth/ConnexionController.php`, `app/Http/Controllers/Auth/MotDePasseController.php`, `app/Http/Controllers/Admin/UtilisateurController.php`, `app/Http/Controllers/TableauBordController.php` (méthode `accueil`), vues `resources/views/auth/*`, `resources/views/admin/utilisateurs/*`, `resources/views/components/invite.blade.php`
Tables	`users`, `role_user`, `sessions`, `password_reset_tokens`, `audit_logs`
Permissions	`utilisateurs.gerer`, `roles.gerer`

Fonctionnement. Détaillé dans la section 2.16. Points complémentaires :

- Un compte peut être lié à une fiche agent (`users.agent_id`, unique) : c'est ce lien qui donne accès à l'« Espace agent ».
- Le **service** et la **direction** du compte (`users.service_id`, `users.direction_id`) déterminent le cloisonnement (un chef de service voit son service).
- Désactiver un compte (`UtilisateurController::activation`) ferme immédiatement ses sessions.

Erreurs fréquentes

Symptôme	Cause	Correction
« Identifiants incorrects ou compte désactivé. »	Mauvais mot de passe, ou compte désactivé (`users.actif = 0`).	Vérifier l'adresse ; un administrateur réactive le compte ou génère un mot de passe temporaire.
« Trop de tentatives… »	Limitation `throttle:connexion` (5 par minute).	Attendre une minute. En développement : `php artisan cache:clear`.
Toutes les pages renvoient vers « Changer mon mot de passe »	`doit_changer_mot_de_passe = 1` (mot de passe temporaire).	Choisir un nouveau mot de passe conforme.
Page 419 « Page Expired »	Jeton CSRF expiré (session de 30 min dépassée) ou formulaire ouvert avant une reconnexion.	Recharger la page puis renvoyer.
« Impossible : ce compte est le dernier administrateur technique actif. »	Tentative de retirer le dernier administrateur.	Créer d'abord un autre administrateur.
Aucun administrateur et plus d'accès	—	`php artisan inps:creer-admin` en console.

4.2 Tableau de bord

Fichiers	`app/Http/Controllers/TableauBordController.php`, `app/Services/IndicateurService.php`, `resources/views/tableau-bord/index.blade.php`, `resources/views/components/kpi.blade.php`
Permission	`tableau_bord.voir` (blocs financiers : `finances.voir`)

Fonctionnement. `TableauBordController::index` valide les filtres (`filtres()` de la classe de base), puis `IndicateurService::tableauDeBord($f)` calcule tous les indicateurs **en base** (aucun chiffre stocké ni inventé). Les définitions sont écrites en tête du service :

`app/Services/IndicateurService.php` — lignes 20 à 32

```
/**
 * Indicateurs du tableau de bord, calculés exclusivement à partir de la base.
 *
 * Définitions (voir docs/GUIDE_UTILISATEUR.md § Indicateurs) :
 *  - inscription retenue : inscription au statut inscrit, convoqué ou confirmé (hors attente,
désistement, remplacement, annulation) ;
 *  - participation : inscription retenue ayant au moins une présence (présent, retard ou partiel) ;
 *  - agent formé : agent UNIQUE ayant au moins une inscription au résultat « achevée » ;
 *  - taux de participation = participations / inscriptions retenues des sessions dont le relevé est
validé ;
 *  - taux d'achèvement = inscriptions achevées / inscriptions retenues des sessions dont le relevé est
validé ;
 *  - budget prévisionnel = Σ coûts estimés de la version validée du plan de l'année (à défaut : Σ coûts
prévisionnels des actions) ;
 *  - engagé = Σ dépenses engagées + payées ; dépensé = Σ dépenses payées (dépenses annulées exclues).
 */
class IndicateurService
```

Parcours des données. Filtres du formulaire (GET) → `IndicateurService::actions($f)` et `inscriptions($f)` construisent deux requêtes de base filtrées → comptages groupés (`GROUP BY statut`, `GROUP BY type`), sommes des dépenses, taux → tableau PHP `$k` → vue.

Règles métier. Un agent formé est compté **une seule fois** (`distinct()->count('agent_id')`) ; les taux ne portent que sur les sessions dont le relevé de présence est **validé** ; les dépenses annulées sont exclues.

Erreurs possibles. Un indicateur à 0 alors que des formations ont eu lieu signifie souvent que les relevés de présence n'ont pas été **validés** (le résultat individuel n'est calculé qu'à la validation) ; « — » s'affiche quand un taux n'a pas de base de calcul.

4.3 Référentiel du personnel et importations (module B)

Fichiers	`app/Http/Controllers/AgentController.php`, `app/Http/Requests/AgentRequest.php`, `app/Policies/AgentPolicy.php`, `app/Models/Agent.php`, `app/Http/Controllers/ImportController.php`, `app/Services/ImportService.php`, `app/Models/Import.php`, `app/Models/ImportLigne.php`, vues `resources/views/agents/*`, `resources/views/imports/*`, modèles CSV dans `exemples/`
Tables	`agents`, `imports`, `import_lignes` (et `actions_formation`, `sessions_formation`, `inscriptions` pour l'historique)
Permissions	`agents.voir_tous`, `agents.voir_service`, `agents.gerer`, `agents.importer`, `agents.exporter`

Fonctionnement — fiche agent. Création et modification par formulaire validé (`AgentRequest`) ; l'agent n'est **jamais supprimé** :

`app/Http/Controllers/AgentController.php` — lignes 86 à 96

```
public function activation(Request $r, Agent $agent): RedirectResponse
{
    $d = $r->validate(['actif' => ['required', 'boolean'], 'motif' => ['required_if:actif,0', 'nullable',
'string', 'max:255']]);
    $agent->actif = (bool) $d['actif'];
    $agent->date_desactivation = $agent->actif ? null : now();
    $agent->motif_desactivation = $agent->actif ? null : $d['motif'];
    $agent->save();
    Audit::enregistrer($agent->actif ? 'agent.reactivation' : 'agent.desactivation', $agent, ($agent-
>actif ? 'Réactivation' : 'Désactivation')." de l'agent {$agent->matricule}");

    return back()->with('succes', $agent->actif ? 'Agent réactivé.' : 'Agent désactivé (ses données et son
historique sont conservés).');
}
```

L'export Excel (`AgentController::export`) applique le même cloisonnement que la liste (`Agent::visiblesPar`).

Fonctionnement — import en deux temps. `ImportService` lit un CSV ou un fichier Excel, **analyse** chaque ligne sans rien écrire dans le référentiel, présente le résultat (lignes valides, en erreur, en doublon), puis **confirme** l'écriture des seules lignes valides dans une transaction unique (tout ou rien) :

`app/Services/ImportService.php` — lignes 27 à 46

```
/**
 * Importation contrôlée (agents, historique des formations) en deux temps :
 *  1. analyser() : lecture, contrôle des colonnes, validation ligne à ligne, détection des doublons
 *     (dans le fichier et en base) et des références inconnues ; rien n'est écrit dans le référentiel ;
 *  2. confirmer() : écriture des seules lignes valides dans UNE transaction (tout ou rien).
 * Aucune donnée existante n'est modifiée ni supprimée. Le fichier téléversé n'est pas conservé.
 */
class ImportService
{
    public const COLONNES = [
        'agents' => [
            'obligatoires' => ['matricule', 'nom', 'prenom'],
            'facultatives' => ['direction_code', 'service_code', 'fonction', 'statut_professionnel',
'email_pro', 'telephone_pro'],
        ],
        'historique' => [
            'obligatoires' => ['matricule', 'intitule', 'type', 'date_debut', 'date_fin', 'resultat'],
            'facultatives' => ['domaine_code', 'organisme', 'lieu', 'duree_heures'],
        ],
    ];
```

`app/Services/ImportService.php` — lignes 93 à 125

```
public function confirmer(Import $import, User $user): Import
{
    if ($import->statut !== 'analyse') {
        throw new RegleMetierException('Cet import a déjà été traité.');
    }
    if ($import->lignes_valides === 0) {
        throw new RegleMetierException('Aucune ligne valide à importer.');
    }
    try {
        DB::transaction(function () use ($import, $user) {
            $n = 0;
            $lignes = $import->lignes()->where('statut', 'valide')->get();
            if ($import->type === 'agents') {
                foreach ($lignes as $l) {
                    $this->creerAgent($l->donnees);
                    $l->update(['statut' => 'importee']);
                    $n++;
                }
```

```
            } else {
                $n = $this->creerHistorique($lignes, $user);
            }
            $import->update(['statut' => 'termine', 'lignes_importees' => $n, 'termine_le' => now(),
'message' => "$n ligne(s) importée(s)."]);
            Audit::enregistrer('import.confirmation', $import, "Import {$import->type} confirmé",
['importees' => $n]);
        });
    } catch (Throwable $e) {
        // Transaction annulée : aucune ligne n'a été écrite. L'utilisateur corrige le fichier puis
relance.
        report($e);
        $import->update(['statut' => 'echoue', 'message' => 'Échec — aucune donnée écrite (annulation
complète). Détail : '.Str::limit($e->getMessage(), 300)]);
        Audit::enregistrer('import.echec', $import, "Échec de l'import {$import->type}");
    }

    return $import->fresh();
}
```

Parcours des données. Fichier téléversé → `ImportService::lire` (détection de l'encodage : un CSV enregistré par Excel en Windows-1252 est converti en UTF-8 ; séparateur `;` ou `,` détecté) → contrôle des colonnes obligatoires → validation ligne par ligne (`validerAgent` / `validerHistorique`) → lignes stockées dans `import_lignes` avec leur statut et leurs messages → écran de contrôle → confirmation → création des agents (ou des actions, sessions et inscriptions historiques) → statut `termine`. Le fichier téléversé n'est pas conservé.

Règles métier. Matricule unique ; service cohérent avec la direction ; aucune donnée existante n'est modifiée par un import ; 5 000 lignes maximum par fichier.

Erreurs possibles et correction

Message	Correction
« Colonnes obligatoires absentes : … Utilisez le modèle fourni. »	Télécharger le modèle depuis l'écran *Importations* (ou `exemples/modele-import-agents.csv`) et respecter les en-têtes.
« Format non reconnu : utilisez CSV (séparateur « ; » ou « , ») ou Excel (.xlsx). »	Enregistrer le fichier en `.csv` ou `.xlsx`.
« Le fichier dépasse 5000 lignes… »	Découper le fichier.
Ligne « doublon »	Matricule déjà présent en base ou deux fois dans le fichier : corriger le fichier.
« Matricule absent ou invalide (2 à 30 caractères : lettres, chiffres, -/ .). »	Corriger le matricule dans le fichier.
Accents illisibles dans la liste	Fichier dans un encodage autre que UTF-8 ou Windows-1252 : réenregistrer en « CSV UTF-8 ».

4.4 Recensement des besoins (module C)

Fichiers	`app/Http/Controllers/BesoinController.php`, `app/Http/Requests/BesoinRequest.php`, `app/Policies/BesoinFormationPolicy.php`, `app/Services/BesoinService.php`, `app/Models/BesoinFormation.php`, `app/Models/Decision.php`, `app/Enums/StatutBesoin.php`, vues `resources/views/besoins/*`, `resources/views/components/decisions.blade.php`
Tables	`besoins_formation`, `besoin_agent`, `decisions`, `documents`, `notifications`, `audit_logs`
Permissions	`besoins.voir_tous`, `besoins.voir_service`, `besoins.creer`, `besoins.creer_tous_services`, `besoins.instruire`

Fonctionnement et parcours des données. Détaillés aux sections 2.7 à 2.11 et 2.17 (parcours complet avec les requêtes SQL).

Cycle de vie.

```
brouillon ──soumission──► soumise ──mise_en_examen──► en_examen ──acceptation──► acceptee ──(intégration au plan)──► integree_plan
    ▲                        │                          │                          ▲
    │                        └──────── acceptation / rejet / demande_complement ──────┘
    │
    └──── soumission ◄── a_completer ◄── demande_complement          rejet ──► rejetee        (retrait du plan) ◄──────┘
```

Règles métier. Déclaration pour son propre service uniquement (sauf `besoins.creer_tous_services`) ; modification possible seulement en brouillon ou « à compléter » ; commentaire **obligatoire** pour un rejet ou une demande de compléments ; plus de soumission sur une année **clôturée** ; les étapes « En examen » et « À compléter » peuvent être désactivées dans les paramètres ; seul un brouillon peut être retiré.

Erreurs possibles et correction

Message	Correction
« Vous ne pouvez déclarer des besoins que pour votre propre service. »	Le compte n'a pas le bon `service_id`, ou il faut la permission `besoins.creer_tous_services`.
« Cette demande ne peut plus être modifiée dans son état actuel (…). »	Seuls les brouillons et les demandes « à compléter » sont modifiables.
« Un commentaire motivé est obligatoire pour cette décision. »	Saisir le motif dans la boîte de dialogue.
« L'année 2026 est clôturée : aucune nouvelle soumission n'est possible. »	Ouvrir l'année dans *Administration › Référentiels › Années*, si c'est voulu.
« Action impossible : la demande est « … ». »	La demande a changé de statut entre-temps (autre utilisateur) : recharger la fiche.
Formulaire : « Aucune année de référence n'est ouverte au recensement »	Créer/ouvrir une année dans les référentiels.

4.5 Plans annuels et circuits de validation (module D)

Fichiers	`app/Http/Controllers/PlanController.php`, `app/Http/Requests/PlanLigneRequest.php`, `app/Services/PlanService.php`, `app/Models/PlanFormation.php`, `app/Models/PlanVersion.php`, `app/Models/PlanAction.php`, `app/Models/CircuitValidation.php`, `app/Models/EtapeValidation.php`, `app/Http/Controllers/Admin/CircuitController.php`, `app/Enums/StatutPlanVersion.php`, vues `resources/views/plans/*`, `resources/views/admin/circuits/index.blade.php`, PDF `resources/views/pdf/plan.blade.php`
Tables	`plans_formation`, `plan_versions`, `plan_actions`, `circuits_validation`, `etapes_validation`, `decisions`
Permissions	`plans.voir`, `plans.gerer`, `plans.valider`, `plans.reviser` ; circuits : `parametres.gerer`

Fonctionnement. Un plan par année ; chaque plan a des **versions** numérotées ; seule une version **brouillon** ou **rejetée** est modifiable. La soumission exige un **circuit de validation actif** (étapes ordonnées, chacune confiée à un rôle) :

`app/Services/PlanService.php` — lignes 166 à 190

```
public function soumettre(PlanVersion $v, User $user): PlanVersion
{
    $this->verifierModifiable($v);
    if ($v->actions()->count() === 0) {
        throw new RegleMetierException('Le plan ne contient aucune action : il ne peut pas être soumis.');
    }
    $circuit = CircuitValidation::actifPour('plan');
    if (! $circuit || $circuit->etapes->isEmpty()) {
        throw new RegleMetierException('Aucun circuit de validation actif n\'est configuré pour les plans.
Configurez-le dans Paramétrage > Circuits de validation.');
    }

    return DB::transaction(function () use ($v, $user, $circuit) {
        $avant = $v->statut;
        $v->statut = V::Soumise;
        $v->circuit_id = $circuit->id;
        $v->etape_courante = $circuit->etapes->first()->ordre;
        $v->soumise_le = now();
        $v->save();
        $v->decisions()->create(['action' => 'soumission', 'statut_avant' => $avant->value, 'statut_apres'
=> V::Soumise->value, 'user_id' => $user->id]);
        Audit::enregistrer('plan.soumission', $v, "Soumission du plan {$v->plan->annee} v{$v->numero}");
        $this->notifierEtape($v);

        return $v;
    });
}
```

Les décisions suivent les étapes ; la validation finale **verrouille** la version et remplace l'ancienne version de référence :

`app/Services/PlanService.php` — lignes 192 à 207

```
/** L'utilisateur peut-il décider à l'étape courante ? (permission + rôle de l'étape + séparation des
tâches) */
public function peutDecider(PlanVersion $v, User $user): bool
{
    if ($v->statut !== V::Soumise || ! $user->aPermission('plans.valider')) {
        return false;
    }
    $etape = $v->etapeCouranteModele();
    if (! $etape || ! $user->roles()->where('roles.id', $etape->role_id)->exists()) {
        return false;
    }
    if (Parametre::valeur('plans.interdire_auto_validation', true) && $v->cree_par === $user->id) {
        return false;
    }

    return true;
}
```

Pour pouvoir décider, il faut **à la fois** la permission `plans.valider`, **le rôle de l'étape en cours**, et — si le paramètre `plans.interdire_auto_validation` est actif (par défaut) — ne pas être l'auteur de la version (séparation des tâches).

Parcours des données. Création du plan (`creerPlan` : plan + version 1 brouillon) → intégration des besoins acceptés (`integrerBesoins` : une ligne `plan_actions` par besoin, code `PA2026-001`…, le besoin passe à `integree_plan`, avertissement si doublon probable) → ajout/modification de lignes → soumission (`statut = soumise`, `etape_courante` = 1re étape, notification aux validateurs) → décision à chaque étape (`decisions`) → validation finale (`statut = validee`, `verrouillee_le`) → éventuelle **révision** (`reviser` : nouvelle version brouillon, copie des lignes avec les mêmes codes, motif obligatoire).

Comparaison prévu / réalisé (`PlanService::comparaison`) : pour chaque ligne de la version validée, on cherche les actions de formation portant le même `plan_id` et le même `plan_action_code`, puis on additionne les dépenses payées et on compte les agents ayant achevé.

Erreurs possibles et correction

Message	Correction
« Aucun circuit de validation actif n'est configuré pour les plans… »	*Administration › Circuits de validation* : ajouter au moins une étape au circuit, puis l'activer. Le circuit fourni par le référentiel initial est **inactif** à dessein (à valider par l'INPS).
« Le plan ne contient aucune action : il ne peut pas être soumis. »	Intégrer des besoins ou ajouter des lignes.
« La version N est « Validée » : elle n'est pas modifiable. Ouvrez une révision si nécessaire. »	Bouton « Ouvrir une révision » (permission `plans.reviser`).
« Vous n'êtes pas habilité à décider à cette étape du circuit de validation. »	Vérifier : permission `plans.valider`, rôle de l'étape, et que la personne n'est pas l'auteur de la version.
« Un plan existe déjà pour l'année … »	Un seul plan par année : ouvrir le plan existant.
« … ignoré : seul un besoin accepté peut être intégré … »	Le besoin doit être au statut « Acceptée ».

4.6 Actions de formation et sessions (module E)

Fichiers	`app/Http/Controllers/ActionController.php`, `app/Http/Controllers/SessionController.php`, `app/Http/Requests/ActionRequest.php`, `app/Http/Requests/SessionRequest.php`, `app/Services/ActionService.php`, `app/Models/ActionFormation.php`, `app/Models/SessionFormation.php`, `app/Models/Seance.php`, `app/Models/SessionChangement.php`, `app/Enums/StatutAction.php`, `app/Enums/StatutSession.php`, vues `resources/views/actions/*`, `resources/views/sessions/*`, PDF `pdf/emargement`, `pdf/convocations`
Tables	`actions_formation`, `sessions_formation`, `seances`, `session_changements`
Permissions	`actions.voir`, `actions.gerer` (convocations PDF : `inscriptions.gerer`)

Fonctionnement. Une action (référence `ACT-2026-0001`) peut être rattachée à une ligne de plan. Une session (code `ACT-2026-0001-S01`) génère automatiquement **une séance par jour ouvré** :

`app/Services/ActionService.php` — lignes 203 à 214

```
/** Une séance par jour ouvré (lundi–vendredi) entre les dates ; ajustable ensuite manuellement. */
public function genererSeances(SessionFormation $s): void
{
    $jours = collect(CarbonPeriod::create($s->date_debut, $s->date_fin))->filter(fn ($d) => ! $d->isWeekend());
    if ($jours->isEmpty()) {
        $jours = collect([$s->date_debut->copy()]);
    }
    $i = 1;
    foreach ($jours as $d) {
        Seance::create(['session_id' => $s->id, 'date' => $d->toDateString(), 'libelle' => 'Jour '.$i++, 'heure_debut' => $s->heure_debut, 'heure_fin' => $s->heure_fin]);
    }
}
```

Les changements de statut d'une session sont encadrés :

`app/Services/ActionService.php` — lignes 177 à 201

```
public function changerStatutSession(SessionFormation $s, S $nouveau, User $user): SessionFormation
{
    $autorises = [
        S::Planifiee->value => [S::Confirmee, S::EnCours],
        S::Confirmee->value => [S::EnCours, S::Planifiee],
        S::EnCours->value => [S::Terminee],
        S::Reportee->value => [S::Planifiee],
    ];
    if (! in_array($nouveau, $autorises[$s->statut->value] ?? [], true)) {
        throw new RegleMetierException("Passage de « {$s->statut->libelle()} » à « {$nouveau->libelle()} »
non autorisé. Utilisez les fonctions de report ou d'annulation si nécessaire.");
    }
    if ($nouveau === S::Terminee && ! $s->presencesValidees()) {
        throw new RegleMetierException('Validez d\'abord le relevé de présence avant de clôturer la
session.');
    }
    $s->changements()->create(['type' => 'changement_statut', 'ancien_statut' => $s->statut->value,
'nouveau_statut' => $nouveau->value, 'user_id' => $user->id]);
    $s->statut = $nouveau;
    $s->save();
    if ($nouveau === S::EnCours && $s->action->statut === StatutAction::Planifiee) {
        $s->action->statut = StatutAction::EnCours;
        $s->action->save();
    }
    Audit::enregistrer('session.statut', $s, "Session {$s->code} : {$nouveau->libelle()}");

    return $s;
}
```

Règles métier. Rien n'est effacé : chaque report, modification de dates, annulation et changement de statut est écrit dans `session_changements` avec l'auteur et le motif. Une session ne peut pas être « terminée » tant que le relevé de présence n'est pas validé ; une action ne peut être « réalisée » que si au moins une session est terminée ; l'annulation d'une session passe ses inscriptions au statut « annulé » ; le report n'est possible que si aucune présence n'a été saisie.

Erreurs possibles et correction

Message	Correction
« Validez d'abord le relevé de présence avant de clôturer la session. »	Saisir toutes les présences puis « Valider le relevé ».
« Passage de « … » à « … » non autorisé… »	Respecter l'ordre planifiée → confirmée → en cours → terminée ; utiliser « Reporter » ou « Annuler » pour les autres cas.
« Des présences ont déjà été saisies pour cette session : le report n'est pas possible. »	La session a commencé : la clore normalement et créer une nouvelle session si nécessaire.
« Le nombre de places ne peut pas être inférieur au nombre d'inscrits occupant une place (N). »	Désinscrire d'abord des participants ou garder une capacité suffisante.
« Une action ne peut être déclarée réalisée que si au moins une session est terminée. »	Terminer une session.
« La ligne du plan indiquée n'existe pas dans ce plan. »	Vérifier le code de ligne (ex. `PA2026-003`).

4.7 Organismes et formateurs (module F)

Fichiers	`app/Http/Controllers/OrganismeController.php`, `app/Http/Controllers/FormateurController.php`, `app/Models/Organisme.php`, `app/Models/ContactOrganisme.php`, `app/Models/Convention.php`, `app/Models/Formateur.php`, `app/Models/AppreciationPrestataire.php`, vues `resources/views/organismes/*`, `resources/views/formateurs/*`
Tables	`organismes`, `contacts_organisme`, `conventions`, `formateurs`, `appreciations_prestataires`
Permissions	`organismes.voir`, `organismes.gerer`, `organismes.contrats` (conventions, identifiant fiscal, documents de l'organisme) ; appréciations : `organismes.gerer` ou `evaluations.saisir`

Fonctionnement. Fiches organismes avec contacts ; les **données contractuelles** (conventions, identifiant fiscal) ne sont affichées et modifiables qu'avec `organismes.contrats` : `OrganismeController::show` ne les charge pas sinon, et `donnees()` retire l'identifiant fiscal de la saisie d'un utilisateur non habilité. Un formateur est **interne** (lié à un agent) ou **externe** (lié à un organisme). Les appréciations (note de 1 à 5) alimentent le rapport « Prestataires sollicités ».

Erreurs possibles. Une convention en double pour un même organisme est refusée (contrainte `UNIQUE(organisme_id, reference)`) ; un formateur désactivé n'est plus proposé dans les sessions.

4.8 Inscriptions et participants (module G)

Fichiers	`app/Http/Controllers/InscriptionController.php`, `app/Services/InscriptionService.php`, `app/Models/Inscription.php`, `app/Models/InscriptionEvenement.php`, `app/Enums/StatutInscription.php`, vues `resources/views/sessions/show.blade.php`, `resources/views/inscriptions/show.blade.php`
Tables	`inscriptions`, `inscription_evenements`, `notifications`
Permission	`inscriptions.gerer`

Fonctionnement. Inscription individuelle ou collective (tout un service), avec contrôle de capacité **sous verrou** pour éviter les dépassements en cas d'accès simultanés :

`app/Services/InscriptionService.php` — lignes 23 à 80

```
/**
 * @return array{inscrits: list<string>, attente: list<string>, refus: list<string>}
 */
public function inscrire(SessionFormation $session, array $agentIds, User $user): array
{
    $r = ['inscrits' => [], 'attente' => [], 'refus' => []];

    DB::transaction(function () use ($session, $agentIds, $user, &$r) {
        $s = SessionFormation::whereKey($session->id)->lockForUpdate()->firstOrFail();
        $this->verifierSessionOuverte($s);

        foreach (Agent::whereIn('id', array_unique($agentIds))->get() as $agent) {
            $nom = "{$agent->matricule} {$agent->nom_complet}";
            if (! $agent->actif) {
                $r['refus'][] = "$nom : agent inactif.";

                continue;
            }
            $existante = Inscription::where('session_id', $s->id)->where('agent_id', $agent->id)->first();
            if ($existante && ! in_array($existante->statut, [I::Desiste, I::Annule], true)) {
                $r['refus'][] = "$nom : déjà inscrit (« {$existante->statut->libelle()} »).";

                continue;
            }
```

```
            if ($conflit = $this->chevauchement($agent->id, $s)) {
                $r['refus'][] = "$nom : déjà inscrit à la session {$conflit} sur des dates qui se
chevauchent.";

                continue;
            }
            $placeDispo = $s->placesOccupees() < $s->places;
            if (! $placeDispo && ! $s->liste_attente_active) {
                $r['refus'][] = "$nom : capacité d'accueil atteinte ({$s->places} places).";

                continue;
            }
            $i = $existante ?? new Inscription;
            $i->session_id = $s->id;
            $i->agent_id = $agent->id;
            $i->inscrit_par = $user->id;
            $i->desiste_le = null;
            $i->motif_desistement = null;
            if ($placeDispo) {
                $i->statut = I::Inscrit;
                $i->rang_attente = null;
                $r['inscrits'][] = $nom;
            } else {
                $i->statut = I::ListeAttente;
                $i->rang_attente = (int) Inscription::where('session_id', $s->id)->where('statut',
I::ListeAttente->value)->max('rang_attente') + 1;
                $r['attente'][] = $nom;
            }
            $i->save();
            $i->evenements()->create(['evenement' => $existante ? 'reinscription' : 'inscription',
'details' => $i->statut->libelle(), 'user_id' => $user->id]);
        }
        Audit::enregistrer('inscription.ajout', $s, "Inscriptions sur {$s->code}", ['inscrits' =>
count($r['inscrits']), 'attente' => count($r['attente']), 'refus' => count($r['refus'])]);
    });

    return $r;
}
```

Statuts : `inscrit`, `liste_attente`, `convoque`, `confirme`, `desiste`, `remplace`, `annule`. Seuls **inscrit, convoqué, confirmé** occupent une place. Chaque événement (inscription, convocation, confirmation, désistement, remplacement, promotion depuis la liste d'attente) est écrit dans `inscription_evenements`.

Règles métier. Pas de double inscription ; pas d'agent inactif ; pas de **chevauchement de dates** avec une autre session active du même agent ; liste d'attente si la session est pleine et que l'option est activée ; un désistement libère la place et **promeut** le premier de la liste d'attente ; un remplacement exige un motif ; tout est fermé une fois le relevé de présence validé ou la session terminée/annulée.

Erreurs possibles. Les refus ne bloquent pas toute l'inscription collective : ils sont listés dans le message « Points d'attention » (« déjà inscrit », « agent inactif », « dates qui se chevauchent », « capacité d'accueil atteinte »). « Le relevé de présence est validé : le désistement n'est plus possible. » signifie que la session est clôturée sur le plan des présences.

4.9 Présences (module H)

Fichiers	`app/Http/Controllers/PresenceController.php`, `app/Services/PresenceService.php`, `app/Models/Presence.php`, `app/Models/PresenceCorrection.php`, `app/Enums/StatutPresence.php`, vue `resources/views/presences/edit.blade.php`
Tables	`presences`, `presence_corrections`, `inscriptions` (taux et résultat)
Permissions	`presences.saisir`, `presences.valider`, `presences.motifs` (motifs d'absence, données sensibles)

Fonctionnement. Saisie par séance (présent, retard, partiel, absent) ; validation du relevé quand **toutes** les présences sont saisies ; calcul du résultat individuel à la validation :

`app/Services/PresenceService.php` — lignes 122 à 140

```
public function calculerResultat(Inscription $i): Inscription
{
    $nbSeances = $i->session->seances()->count();
    $somme = 0.0;
    foreach ($i->presences()->get() as $p) {
        $somme += $this->poids($p->statut);
    }
    $taux = $nbSeances ? round(100 * $somme / $nbSeances, 2) : 0;
    $acheve = $taux >= (float) Parametre::valeur('regles.seuil_achevement', 80);
    if ($acheve && Parametre::valeur('regles.acquis_requis_pour_achevement', false)) {
        $acheve = (bool) $i->evaluationAcquis?->reussi;
    }
    $i->taux_presence = $taux;
    $i->resultat = $acheve ? 'achevee' : 'non_achevee';
    $i->resultat_calcule_le = now();
    $i->save();

    return $i;
}
```

Exemple : 3 séances, présent – partiel – présent → (1 + 0,5 + 1) / 3 = 83,33 % ≥ 80 % → achevée. Avec présent – absent – partiel → 50 % → non achevée. Le seuil (`regles.seuil_achevement`), les poids (`regles.poids_retard`, `regles.poids_partiel`) et l'exigence de réussite aux acquis sont des **paramètres** modifiables.

Règles métier. Après validation, une présence ne se modifie plus que par **correction** motivée (`PresenceService::corriger`), historisée dans `presence_corrections`, et le résultat de l'agent est recalculé. Les motifs d'absence ne sont acceptés et affichés qu'avec `presences.motifs` (`PresenceController::update` les retire sinon).

Erreurs possibles. « Relevé incomplet : N présence(s) restent à saisir avant validation. » → compléter chaque séance ; « Le relevé est validé : utilisez la procédure de correction. » → bouton « Corriger » ; « Le motif de la correction est obligatoire. ».

4.10 Évaluations et attestations (module I)

Fichiers	`app/Http/Controllers/EvaluationController.php`, `app/Http/Controllers/AttestationController.php`, `app/Services/AttestationService.php`, modèles `EvaluationSatisfaction`, `EvaluationAcquis`, `EvaluationImpact`, `Attestation`, vues `resources/views/evaluations/*`, `resources/views/attestations/index.blade.php`, PDF `resources/views/pdf/attestation.blade.php`
Tables	`evaluations_satisfaction`, `evaluations_acquis`, `evaluations_impact`, `attestations`
Permissions	`evaluations.saisir`, `evaluations.resultats` (acquis individuels), `attestations.emettre`

Fonctionnement. Trois évaluations : satisfaction (notes de 1 à 5), acquis (méthode, score, réussite — accès restreint), impact différé (niveau de mise en pratique de 1 à 4, rappel automatique après le délai paramétré). Le bilan d'une session agrège les moyennes. Une attestation n'est délivrée que si les conditions sont remplies :

`app/Services/AttestationService.php` — lignes 18 à 43

```
/** @return list<string> conditions non remplies (vide = émission possible) */
public function conditionsNonRemplies(Inscription $i): array
```

```
{
    $manque = [];
    $i->loadMissing(['session', 'evaluationSatisfaction', 'evaluationAcquis', 'attestation']);
    if (in_array($i->statut, [StatutInscription::Desiste, StatutInscription::Remplace,
StatutInscription::Annule, StatutInscription::ListeAttente], true)) {
        $manque[] = "L'inscription n'est pas active ({$i->statut->libelle()}).";
    }
    if (! $i->session->presencesValidees()) {
        $manque[] = 'Le relevé de présence de la session n\'est pas validé.';
    }
    if ($i->resultat !== 'achevee') {
        $manque[] = 'La formation n\'est pas achevée pour cet agent (taux de présence : '.($i-
>taux_presence !== null ? $i->taux_presence.' %' : 'non calculé').').';
    }
    if (Parametre::valeur('regles.attestation_exige_satisfaction', false) && ! $i->evaluationSatisfaction)
{
        $manque[] = 'L\'évaluation de satisfaction n\'a pas été renseignée.';
    }
    if (Parametre::valeur('regles.attestation_exige_acquis', false) && ! $i->evaluationAcquis?->reussi) {
        $manque[] = 'L\'évaluation des acquis n\'est pas renseignée ou n\'est pas réussie.';
    }
    if ($i->attestation && $i->attestation->estValide()) {
        $manque[] = 'Une attestation valide existe déjà ('.$i->attestation->reference.').';
    }

    return $manque;
}
```

L'attestation reçoit une référence (`ATT-2026-0001`) et un **code de vérification** aléatoire (32 caractères hexadécimaux) imprimé sur le PDF. Une attestation annulée (motif obligatoire) ne peut plus être téléchargée (réponse 410).

Erreurs possibles. « Attestation non délivrable : … » suivi de la liste exacte des conditions manquantes (relevé non validé, formation non achevée, évaluation manquante si exigée, attestation déjà valide).

4.11 Suivi financier (module J)

Fichiers	`app/Http/Controllers/DepenseController.php`, `app/Services/DepenseService.php`, `app/Models/Depense.php`, `app/Enums/StatutDepense.php`, vues `resources/views/depenses/*`
Tables	`depenses`
Permissions	`finances.voir`, `finances.gerer`, `finances.exporter`

Fonctionnement. Dépenses rattachées à une action (et éventuellement à une session), statuts **engagée**, **payée**, **annulée**. Toute correction exige un motif et l'audit conserve les valeurs **avant/après** des champs financiers :

`app/Services/DepenseService.php` — lignes 37 à 52

```
public function corriger(Depense $dep, array $d, string $motif): Depense
{
    if (trim($motif) === '') {
        throw new RegleMetierException('Le motif de la correction est obligatoire.');
    }
    $this->controler($dep->action, $d);

    return DB::transaction(function () use ($dep, $d, $motif) {
        $dep->fill($d);
        $diff = Audit::differences($dep, self::CHAMPS_SUIVIS);
        $dep->save();
        Audit::enregistrer('finance.depense_correction', $dep, "Correction d'une dépense ({$dep->action-
>reference})", ['motif' => mb_substr($motif, 0, 300), 'modifications' => $diff]);
```

```
        return $dep;
    });
}
```

`app/Services/DepenseService.php` — lignes 67 à 89

```
/**
 * Synthèse financière d'une action.
 *  prévisionnel = cout_previsionnel de l'action ; engagé = engagées + payées ; payé = payées
 *  coût par participant = payé / agents uniques ayant achevé (null si aucun)
 */
public function synthese(ActionFormation $a): array
{
    $engage = $a->coutEngage();
    $paye = $a->coutPaye();
    $formes = Inscription::whereHas('session', fn ($q) => $q->where('action_id', $a->id))-
>where('resultat', 'achevee')->distinct()->count('agent_id');
    $participants = Inscription::whereHas('session', fn ($q) => $q->where('action_id', $a->id))
        ->whereIn('statut', SessionFormation::STATUTS_OCCUPANT_PLACE)->distinct()->count('agent_id');

    return [
        'previsionnel' => (float) $a->cout_previsionnel,
        'engage' => $engage,
        'paye' => $paye,
        'ecart' => (float) $a->cout_previsionnel - $paye,
        'agents_formes' => $formes,
        'participants' => $participants,
        'cout_par_participant' => $formes ? round($paye / $formes, 0) : null,
    ];
}
```

Règles métier. Montant strictement positif ; date de paiement obligatoire pour une dépense payée et postérieure à l'engagement ; la session doit appartenir à l'action ; coût par participant = payé / agents uniques ayant achevé. Ce module **suit** les coûts ; il ne remplace pas la comptabilité de l'INPS.

Erreurs possibles. « La date de paiement est obligatoire pour une dépense payée. » ; « La session indiquée n'appartient pas à cette action. » ; « Le motif de la correction est obligatoire. ».

4.12 Documents (module K)

Fichiers	`app/Http/Controllers/DocumentController.php`, `app/Services/DocumentService.php`, `app/Policies/DocumentPolicy.php`, `app/Models/Document.php`, composant `resources/views/components/documents.blade.php`
Tables / stockage	`documents` ; fichiers dans `storage/app/private/documents/AAAA/MM/`
Permissions	celles de l'objet de rattachement (besoin, plan, action, session, organisme, dépense) + `documents.confidentiels`

Fonctionnement. Voir 2.16. Les types de rattachement autorisés sont listés dans `Document::RATTACHEMENTS` ; la route `POST /documents/{type}/{id}` n'accepte que ces types.

Erreurs possibles. « Type de fichier refusé (extension « … », contenu détecté « … »). » → le contenu ne correspond pas à l'extension (fichier renommé, ou format non autorisé) ; « Fichier trop volumineux (maximum 10240 Ko). » → paramètre `documents.taille_max_ko`, mais aussi limites PHP `upload_max_filesize` et `post_max_size` du fichier `php.ini`.

4.13 Rapports et exports (module L)

Fichiers	`app/Http/Controllers/RapportController.php`, `app/Services/RapportService.php`, `app/Services/ExportService.php`, vues `resources/views/rapports/*`, `resources/views/pdf/rapport.blade.php`
Permission	`rapports.voir` ; les rapports contenant des montants exigent en plus `finances.voir`

Fonctionnement. 13 rapports déclarés dans une constante :

`app/Services/RapportService.php` — lignes 19 à 33

```
public const RAPPORTS = [
    'formations_annee' => 'Formations par année et par statut',
    'agents_formes' => 'Agents formés par direction et service',
    'agents_sans_formation' => 'Agents n\'ayant suivi aucune formation sur la période',
    'domaines' => 'Répartition par domaine de compétence',
    'interne_externe' => 'Formations internes et externes',
    'jours_formation' => 'Nombre de jours de formation',
    'budget' => 'Budget prévisionnel et réalisé par action',
    'cout_moyen' => 'Coût moyen par agent formé',
    'realisation_plan' => 'Taux de réalisation du plan annuel',
    'presence_achevement' => 'Taux de présence et d\'achèvement par session',
    'evaluations' => 'Résultats des évaluations de satisfaction',
    'prestataires' => 'Prestataires sollicités',
    'reportees_annulees' => 'Sessions reportées ou annulées',
];
```

`RapportService::generer($code, $filtres)` renvoie toujours la même structure : `titre`, `colonnes`, `lignes`, `montants` (index des colonnes monétaires), `note` (règle de calcul). La vue écran, l'impression, le PDF et l'Excel réutilisent ce même résultat : les chiffres sont identiques sur tous les supports. Chaque export est journalisé (`export.rapport`). Les rapports financiers (`budget`, `cout_moyen`, `realisation_plan`, `prestataires`) sont masqués et refusés (403) sans `finances.voir`.

Ajouter un rapport : une entrée dans `RAPPORTS`, une méthode de calcul dans `RapportService` appelée par `generer`, et c'est tout — l'écran, le PDF et l'Excel suivent.

4.14 Notifications et échéances

Fichiers	`app/Services/NotificationService.php`, `app/Notifications/NotificationMetier.php`, `app/Http/Controllers/NotificationController.php`, `app/Console/Commands/Echeances.php`, `routes/console.php`, vue `resources/views/notifications/index.blade.php`
Tables	`notifications`, `rappels_envoyes`, `envois_email`

Fonctionnement. Notifications internes (cloche) envoyées par les services lors des événements (soumission, décision, convocation, validation en attente…). Chaque jour, la commande `inps:echeances` génère les rappels (sessions à venir, convocations à préparer, sessions non clôturées, évaluations à faire, demandes à compléter depuis plus de 7 jours, date limite de recensement, clôture annuelle, bilan annuel). Chaque rappel n'est émis **qu'une fois** grâce à une clé unique :

`app/Console/Commands/Echeances.php` — lignes 100 à 108

```
private function une(string $cle, callable $envoi): void
{
    try {
        RappelEnvoye::create(['cle' => mb_substr($cle, 0, 190)]);
    } catch (QueryException) {
        return; // déjà envoyé
```

```
    }
    $this->total += (int) $envoi();
}
```

La contrainte `UNIQUE(cle)` de `rappels_envoyes` fait échouer l'insertion d'un rappel déjà envoyé ; l'exception est interceptée et l'envoi est ignoré.

Erreurs possibles. Aucun rappel n'arrive : le planificateur ne tourne pas (`php artisan inps:sante` le signale) — sous Windows, créer une tâche planifiée qui exécute `php artisan schedule:run` chaque minute, ou lancer `php artisan inps:echeances` manuellement pour tester. Les e-mails ne partent pas : `MAIL_MAILER=log` (aucun serveur SMTP) ou paramètre `notifications.email_actif` désactivé ; le résultat de chaque tentative est visible dans *Administration › Système*.

Sécurité. `NotificationController::lue` n'ouvre que les notifications de l'utilisateur connecté et ne redirige que vers une adresse **interne** à l'application (pas de redirection ouverte).

4.15 Espace agent

Fichiers	`app/Http/Controllers/EspaceAgentController.php`, vue `resources/views/espace/index.blade.php`
Permission	`espace.personnel`

Un agent dont le compte est lié à sa fiche (`users.agent_id`) voit **uniquement ses** formations et télécharge **ses** attestations valides. `EspaceAgentController::attestation` vérifie que l'attestation appartient bien à l'agent connecté et qu'elle n'est pas annulée ; sinon, 403.

4.16 Administration et paramétrage (module N)

Écran	Contrôleur	Permission	Rôle
Paramètres	`Admin/ParametreController`	`parametres.gerer`	Identité (nom, sigle, couleur, logo, photographie), règles de présence/achèvement/attestation, étapes des besoins, notifications, exports, conservation. Champs validés selon leur type ; cache vidé après enregistrement.
Référentiels	`Admin/ReferentielController`	`parametres.gerer`	Directions, services, domaines, années, nomenclatures. Pas de suppression : désactivation.
Circuits de validation	`Admin/CircuitController`	`parametres.gerer`	Étapes ordonnées (rôle par étape) ; un seul circuit actif par objet ; un circuit utilisé par une version en cours de validation n'est pas modifiable.
Utilisateurs	`Admin/UtilisateurController`	`utilisateurs.gerer`	Comptes, rôles, activation, mot de passe temporaire (protections décrites en 2.16).
Rôles et permissions	`Admin/RoleController`	`roles.gerer`	Création de rôles, attribution des permissions (cases à cocher), suppression d'un rôle non attribué.
Journal d'audit	`Admin/AuditController`	`audit.voir`, `audit.exporter`	Consultation filtrée et export.
Système et sauvegardes	`Admin/SystemeController`	`systeme.superviser`	Contrôles de configuration, liste des sauvegardes, sauvegarde immédiate, suivi des e-mails.

`app/Http/Controllers/Admin/ParametreController.php` — lignes 22 à 65

```
public function update(Request $r): RedirectResponse
{
    $params = Parametre::all()->keyBy('cle');
    $regles = [
        'logo' => ['nullable', 'file', 'max:1024', 'mimes:png,jpg,jpeg'],
        'photo' => ['nullable', 'file', 'max:6144', 'mimes:jpg,jpeg,png,webp',
'dimensions:min_width=1200,min_height=700'],
    ];
    foreach ($params as $p) {
        $regles['p.'.str_replace('.', '__', $p->cle)] = match ($p->type) {
            'int' => ['nullable', 'integer', 'min:0', 'max:100000'],
            'decimal' => ['nullable', 'numeric', 'min:0', 'max:100'],
            'bool' => ['nullable', 'in:0,1'],
            'file' => ['prohibited'],
            default => ['nullable', 'string', 'max:2000'],
        };
    }
    $d = $r->validate($regles);
    $modifies = [];
    foreach ($params as $p) {
        if ($p->type === 'file') {
            continue;
        }
        $cle = 'p.'.str_replace('.', '__', $p->cle);
        $val = data_get($d, $cle);
        $val = $p->type === 'bool' ? ($val === '1' ? '1' : '0') : ($val === null ? null : (string) $val);
        if ($val !== $p->valeur) {
            $p->valeur = $val;
            $p->save();
            $modifies[] = $p->cle;
        }
    }
    foreach (['logo' => 'institution.logo', 'photo' => 'institution.photo_connexion'] as $champ => $cle) {
        if ($r->hasFile($champ) && isset($params[$cle])) {
            $nom = $champ.'-'.Str::random(12).'.'.$r->file($champ)->extension();
            Storage::disk('prive')->putFileAs('identite', $r->file($champ), $nom);
            $params[$cle]->update(['valeur' => $nom]);
            $modifies[] = $cle;
        }
    }
    Parametre::viderCache();
    Audit::enregistrer('parametres.modification', 'parametre', 'Modification des paramètres', ['cles' =>
$modifies]);

    return back()->with('succes', count($modifies).' paramètre(s) modifié(s).');
}
```

Chaque paramètre est validé selon sa colonne `type`. Les noms de champs utilisent `__` à la place du point (`p.regles__seuil_achevement`) parce que le point a un sens particulier dans les noms de champs de Laravel.

4.17 Sauvegarde et restauration

Fichiers	`app/Services/SauvegardeService.php`, `app/Console/Commands/Sauvegarder.php`, `app/Console/Commands/Restaurer.php`, `config/inps.php`, `docs/SAUVEGARDE_RESTAURATION.md`
Commandes	`php artisan inps:sauvegarde`, `php artisan inps:restaurer chemin.zip [--base=…] [--sans-fichiers] [--force]`

Fonctionnement. `mysqldump` exporte la base, les documents de `storage/app/private` sont ajoutés, un **manifeste** contient l'empreinte SHA-256 de chaque fichier, et l'archive ZIP est **chiffrée en AES-256** si `BACKUP_PASSWORD` est défini. Les identifiants MySQL sont passés par un fichier d'options temporaire (jamais

sur la ligne de commande, où ils seraient visibles). Seules les `BACKUP_RETENTION` dernières archives sont conservées. La restauration vérifie le manifeste et **chaque empreinte** avant de toucher à la base, demande de retaper le nom de la base pour confirmer, puis réinstalle les déclencheurs d'audit.

Sous Windows. `mysqldump` et `mysql` ne sont pas dans le `PATH` : renseigner dans `.env` `MYSQLDUMP_PATH="C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe"` et `MYSQL_CLIENT_PATH="C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"`. L'extension PHP `zip` doit être active.

Erreurs possibles. « Commande mysqldump en échec : … » → chemin de `mysqldump` incorrect ou droits MySQL insuffisants ; « Extraction impossible (mot de passe BACKUP_PASSWORD incorrect ?). » ; « Contrôle d'intégrité échoué pour « … » : restauration interrompue. » → archive altérée : utiliser une autre sauvegarde.

4.18 Identité visuelle (version 1.1.0)

Fichiers	`app/Support/Identite.php`, `app/Http/Controllers/IdentiteController.php`, `public/images/institution/` (logo, photographie), `resources/views/components/invite.blade.php`, `resources/views/components/layout.blade.php`
Route	`GET /identite/{logo|photo}` (publique, lecture seule)

`Identite::chemin` cherche d'abord le fichier déposé dans l'application (paramètres `institution.logo` / `institution.photo_connexion`, disque privé), sinon un fichier de `public/images/institution/`. `Identite::url` ajoute à l'adresse une empreinte du fichier : quand le logo change, le navigateur recharge la nouvelle version. `Identite::dataUri` fournit le logo aux PDF.

Partie 5 — Installation et exécution sous Windows

5.1 Les outils et leur rôle

Outil	Rôle	Quand l'utiliser
PHP (`php.exe`)	Exécute le code de l'application et les commandes Artisan.	Toujours.
Composer	Lit `composer.json` / `composer.lock` et télécharge les bibliothèques PHP dans `vendor/`.	À l'installation et après chaque mise à jour du code (`composer install`).
Node.js + npm	Lit `package.json` / `package-lock.json` et installe les outils de compilation dans `node_modules/`.	Seulement si tu modifies le CSS, le JavaScript ou les classes des vues (`npm ci`, `npm run build`). L'archive livrée contient déjà `public/build`.
Vite	Compile `resources/css/app.css` et `resources/js/app.js` vers `public/build/`.	Lancé par `npm run build` (une fois) ou `npm run dev` (en continu pendant le développement).
Artisan (`php artisan …`)	La console de Laravel : serveur de développement, migrations, caches, commandes de l'INPS.	Tous les jours. `php artisan list` affiche toutes les commandes.
Fichier `.env`	La configuration de **ton** installation (base, mots de passe, environnement).	Créé une fois (copie de `.env.example`), modifié avec le Bloc-notes.
MySQL Server 8.0	La base de données, installée comme **service Windows** nommé `MySQL80`.	Doit être démarré avant l'application.

Le fichier `.env` ligne par ligne (valeurs de développement)

Variable	Rôle	Valeur en développement	En production
`APP_ENV`	Environnement	`local`	`production`
`APP_KEY`	Clé de chiffrement (sessions, cookies)	générée par `php artisan key:generate`	générée une fois, **ne jamais la changer** ensuite (les sessions et données chiffrées deviendraient illisibles)
`APP_DEBUG`	Affiche le détail des erreurs	`true`	**`false`** obligatoirement
`APP_URL`	Adresse de l'application	`http://localhost:8000`	l'adresse HTTPS réelle
`DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Connexion MySQL	`127.0.0.1`, `3306`, `inps_formation`, ton compte	compte dédié, mot de passe robuste
`SESSION_LIFETIME`	Déconnexion après inactivité (minutes)	`30`	`30`
`MAIL_MAILER`	Envoi d'e-mails	`log` (aucun envoi : écrit dans les journaux)	`smtp` + paramètres du serveur de l'INPS
`BACKUP_PASSWORD`	Chiffrement des sauvegardes	vide	**obligatoire**

Variable	Rôle	Valeur en développement	En production
`MYSQLDUMP_PATH`, `MYSQL_CLIENT_PATH`	Chemins des outils MySQL pour les sauvegardes	chemins complets sous Windows (voir 4.17)	
`FORCE_HTTPS`	Impose HTTPS	`false`	`true`

Attention

Après toute modification de `.env`, exécute `php artisan optimize:clear` : sinon une ancienne configuration en cache peut rester active. Le fichier `.env` contient des mots de passe : il ne doit jamais être envoyé par e-mail, copié dans une archive partagée ou placé dans le dossier `public/`.

5.2 Installation complète (nouvelle machine)

La procédure détaillée est dans `docs/INSTALLATION_WINDOWS.md`. En résumé, dans l'invite de commandes :

```
cd C:\dev\inps-formation-1.1.0\inps-formation
copy .env.example .env
notepad .env
composer install
php artisan key:generate
php artisan migrate --seed
php artisan inps:creer-admin
php artisan serve
```

Commande	Ce qu'elle fait
`copy .env.example .env` puis `notepad .env`	Crée la configuration locale et l'ouvre pour renseigner la base de données.
`composer install`	Installe Laravel et les bibliothèques dans `vendor/` (quelques minutes la première fois).
`php artisan key:generate`	Écrit une clé aléatoire dans `APP_KEY`.
`php artisan migrate --seed`	Crée les 53 tables puis charge le référentiel initial (permissions, rôles, paramètres).
`php artisan inps:creer-admin`	Crée le premier administrateur technique (saisie masquée du mot de passe).
`php artisan serve`	Lance le serveur de développement sur http://127.0.0.1:8000.

5.3 Démarrer et arrêter au quotidien

Démarrer

1. MySQL (si l'ordinateur a redémarré) — invite de commandes **en administrateur** : `bat net start MySQL80`
2. L'application — invite de commandes normale : `bat cd C:\dev\inps-formation-1.1.0\inps-formation php artisan serve`
3. Navigateur : http://127.0.0.1:8000

Arrêter : `Ctrl + C` dans la fenêtre de `php artisan serve`. MySQL peut rester démarré ; pour l'arrêter : `net stop MySQL80` (administrateur).

Démarrer MySQL automatiquement avec Windows : touche Windows + R, taper `services.msc`, ouvrir le service **MySQL80**, choisir « Type de démarrage : Automatique ». Ou, en administrateur : `sc config MySQL80 start= auto` (l'espace après `start=` est obligatoire).

Utiliser un autre port si le 8000 est occupé : `php artisan serve --port=8001`.

Pendant le développement de l'interface : une deuxième fenêtre avec `npm run dev` recompile le CSS et le JavaScript à chaque enregistrement. Penser à arrêter `npm run dev` puis à lancer `npm run build` avant de livrer.

5.4 Mettre à jour le projet

C'est la procédure que tu as suivie pour passer en 1.1.0 :

1. Arrêter l'application (`Ctrl + C`).
2. Décompresser la nouvelle version dans un **nouveau** dossier.
3. Copier l'ancien `.env` et le dossier `storage\app\private` (documents déposés) dans le nouveau dossier.
4. Dans le nouveau dossier :

```
composer install
php artisan migrate
php artisan optimize:clear
php artisan serve
```

5. Si la mise à jour modifie le CSS ou le JavaScript et que l'archive ne contient pas `public/build`, ajouter `npm ci --ignore-scripts` puis `npm run build`.

`php artisan migrate` n'applique **que** les migrations nouvelles ; tes données sont conservées. En cas de doute, fais d'abord une sauvegarde (partie 6).

5.5 Les commandes Artisan à connaître

Commande	Usage
`php artisan serve`	Serveur de développement.
`php artisan route:list` (`--path=besoins`, `--name=plans`)	Liste des routes : URL, nom, contrôleur, middlewares.
`php artisan migrate`, `migrate:status`	Appliquer / voir l'état des migrations.
`php artisan optimize:clear`	Vide **tous** les caches (configuration, routes, vues, événements). Premier réflexe après une mise à jour ou une modification de `.env`.
`php artisan view:clear`	Vide le cache des vues Blade compilées.
`php artisan cache:clear`	Vide le cache applicatif (paramètres, compteurs de tentatives de connexion).
`php artisan config:cache`, `route:cache`, `view:cache`	En **production** uniquement : met en cache pour accélérer.
`php artisan down` / `php artisan up`	Mode maintenance (page d'attente) pendant une intervention.
`php artisan test`	Lance les tests automatisés.
`php artisan inps:sante`	Contrôle de santé (base, migrations, protection de l'audit, administrateur, stockage, sauvegardes, planificateur).
`php artisan inps:creer-admin`	Crée un administrateur technique.

Commande	Usage
`php artisan inps:echeances`	Génère les rappels d'échéances (normalement automatique, chaque jour).
`php artisan inps:sauvegarde` / `inps:restaurer`	Sauvegarde / restauration.
`php artisan inps:proteger-audit`	Installe les déclencheurs qui protègent le journal d'audit.
`php artisan schedule:list`	Liste des tâches planifiées.

5.6 Diagnostiquer les erreurs courantes

Méthode

1. **Lis le message en entier**, notamment la ligne `SQLSTATE[...]` ou le nom du fichier.
2. Si la page affiche « Erreur 500 » sans détail (ou en production), ouvre le journal : `storage\logs\laravel-AAAA-MM-JJ.log` (le plus récent). L'erreur y est écrite avec la pile d'appels ; la première ligne qui commence par `app\` indique le fichier de **ton** code concerné.
3. Vide les caches : `php artisan optimize:clear`.
4. Vérifie l'état général : `php artisan inps:sante`.

Les erreurs que tu peux rencontrer (dont celles déjà rencontrées pendant l'installation)

Message ou symptôme	Cause	Correction
`'mysql' n'est pas reconnu en tant que commande interne…`	Le client MySQL n'est pas dans le `PATH` de Windows.	Utiliser le chemin complet `"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"`, ou ajouter ce dossier `bin` au `PATH`.
`SQLSTATE[HY000] [2002] Aucune connexion n'a pu être établie car l'ordinateur cible l'a expressément refusée`	Le service MySQL est **arrêté** (souvent après un redémarrage du PC).	`net start MySQL80` en administrateur.
`SQLSTATE[HY000] [1045] Access denied for user …`	Identifiant ou mot de passe MySQL faux dans `.env`.	Corriger `DB_USERNAME` / `DB_PASSWORD`, puis `php artisan optimize:clear`.
`SQLSTATE[HY000] [1049] Unknown database 'inps_formation'`	La base n'existe pas.	La créer (`CREATE DATABASE …`, voir `docs/INSTALLATION_WINDOWS.md`).
`could not find driver`	Extension PHP `pdo_mysql` désactivée.	Activer `extension=pdo_mysql` dans `php.ini`.
`Could not open input file: artisan`	La commande est lancée hors du dossier du projet.	`cd C:\dev\inps-formation-1.1.0\inps-formation` d'abord.
`Vite manifest not found at: … public\build\manifest.json`	Les ressources n'ont jamais été compilées.	`npm ci --ignore-scripts` puis `npm run build`.
`No application encryption key has been specified.`	`APP_KEY` vide.	`php artisan key:generate`.
`Table '…' doesn't exist` ou colonne inconnue après une mise à jour	Migrations non appliquées.	`php artisan migrate`.

Message ou symptôme	Cause	Correction
Avertissement « déclencheurs de protection du journal d'audit non créés » pendant `migrate`	MySQL refuse la création de déclencheurs au compte applicatif (journalisation binaire active).	Avec `root` : `mysql -u root -p inps_formation < database\sql\protection_audit.sql` ; ou `SET GLOBAL log_bin_trust_function_creators = 1;` puis `php artisan inps:proteger-audit`.
Page **419** « Page Expired »	Jeton CSRF invalide (session expirée, formulaire ancien).	Recharger la page. Si cela arrive toujours : vérifier que l'horloge du PC est juste et que `SESSION_DOMAIN` est vide en local.
Page **403** « Vous ne disposez pas des droits nécessaires… »	L'utilisateur n'a pas la permission (middleware) ou la policy refuse.	Vérifier ses rôles (*Administration › Utilisateurs*) et les permissions du rôle.
Page **404** « Not Found »	URL inconnue, ou enregistrement inexistant / supprimé.	Vérifier l'URL ; `php artisan route:list`.
« Trop de tentatives » à la connexion	Limitation à 5 essais par minute.	Attendre ; en développement `php artisan cache:clear`.
Les modifications de CSS ne s'affichent pas	Ressources non recompilées ou cache du navigateur.	`npm run build`, puis `Ctrl + F5`.
Une modification de vue Blade ne s'affiche pas	Cache des vues.	`php artisan view:clear`.
Une modification de `.env` n'est pas prise en compte	Configuration en cache.	`php artisan optimize:clear`.
`Failed to listen on 127.0.0.1:8000`	Le port est déjà utilisé (une autre fenêtre `serve` est ouverte).	Fermer l'autre fenêtre ou `php artisan serve --port=8001`.
Import ou dépôt de fichier refusé pour la taille	Limites de PHP.	Dans `php.ini` : augmenter `upload_max_filesize` et `post_max_size`, puis relancer `php artisan serve`.
`Class "…" not found` après ajout d'un fichier	Autoload Composer pas à jour, ou espace de noms erroné.	Vérifier `namespace` et le nom du fichier ; `composer dump-autoload`.

Partie 6 — Maintenance et évolution

Les exercices de cette partie ont été **réellement réalisés** sur une copie du projet, et les tests ont été lancés après chaque exercice : les extraits marqués « fichier de l'exercice » proviennent de cette copie. Après les trois exercices, la suite complète passait : **107 tests réussis** (les 102 d'origine + 5 nouveaux). Ces modifications ne font **pas** partie de l'application livrée : ce sont des exercices.

Avant toute modification

1. Travaille sur une **copie** du projet et une **base de développement**, jamais sur la base réelle.
2. Fais une sauvegarde (section 6.8) ou, mieux, place le projet sous **Git** (section 6.10) pour pouvoir revenir en arrière.
3. Après la modification : `php artisan test`, puis vérification à l'écran.

6.1 Modifier une page existante

Objectif : afficher le domaine de compétence dans la liste des besoins.

Étape 1 — Trouver les fichiers. L'URL est `/besoins`. `php artisan route:list --path=besoins` indique `BesoinController@index` ; la méthode renvoie `view('besoins.index', ...)`, donc `resources/views/besoins/index.blade.php`.

Étape 2 — Charger la donnée. Le domaine est une relation (`BesoinFormation::domaine`). On l'ajoute au chargement anticipé dans le contrôleur, pour éviter une requête par ligne :

Étape 3 — Afficher la colonne dans l'en-tête et dans chaque ligne du tableau.

Les deux modifications, telles qu'appliquées (format *diff* : `-` ligne retirée, `+` ligne ajoutée) :

```
diff --git a/app/Http/Controllers/BesoinController.php b/app/Http/Controllers/BesoinController.php
index 97d5a8d..d189cd6 100644
--- a/app/Http/Controllers/BesoinController.php
+++ b/app/Http/Controllers/BesoinController.php
@@ -27,7 +27,7 @@ public function index(Request $r): View
         $this->authorize('viewAny', BesoinFormation::class);
         $f = $r->validate(['annee' => ['nullable', 'integer'], 'statut' => ['nullable',
Rule::enum(StatutBesoin::class)], 'service_id' => ['nullable', 'integer'],
             'priorite_id' => ['nullable', 'integer'], 'q' => ['nullable', 'string', 'max:100']]);
-        $besoins = BesoinFormation::visiblesPar($r->user())->with(['service.direction', 'priorite',
'auteur'])
+        $besoins = BesoinFormation::visiblesPar($r->user())->with(['service.direction', 'priorite',
'auteur', 'domaine'])
             ->when($f['annee'] ?? null, fn ($q, $v) => $q->where('annee', $v))
             ->when($f['statut'] ?? null, fn ($q, $v) => $q->where('statut', $v))
             ->when($f['service_id'] ?? null, fn ($q, $v) => $q->where('service_id', $v))
diff --git a/resources/views/besoins/index.blade.php b/resources/views/besoins/index.blade.php
index 3ed07a8..92667fa 100644
--- a/resources/views/besoins/index.blade.php
+++ b/resources/views/besoins/index.blade.php
@@ -18,11 +18,11 @@
             </x-vide>
         @else
             <div class="overflow-x-auto"><table>
-                <thead><tr><th>Référence</th><th>Intitulé</th><th>Année</th><th>Service</th>
<th>Priorité</th><th class="text-right">Participants</th><th>Statut</th><th>Soumise le</th></tr></thead>
+                <thead><tr><th>Référence</th><th>Intitulé</th><th>Année</th><th>Service</th>
<th>Domaine</th><th>Priorité</th><th class="text-right">Participants</th><th>Statut</th><th>Soumise
le</th></tr></thead>
```

```
                <tbody>@foreach ($besoins as $b)
                    <tr>
                        <td class="font-mono whitespace-nowrap"><a href="{{ route('besoins.show', $b)
}}">{{ $b->reference }}</a></td>
-                       <td>{{ $b->intitule }}</td><td>{{ $b->annee }}</td><td>{{ $b->service->nom }}
</td><td>{{ $b->priorite?->libelle ?? '—' }}</td>
+                       <td>{{ $b->intitule }}</td><td>{{ $b->annee }}</td><td>{{ $b->service->nom }}
</td><td>{{ $b->domaine?->nom ?? '—' }}</td><td>{{ $b->priorite?->libelle ?? '—' }}</td>
                        <td class="text-right">{{ $b->nb_participants ?? '—' }}</td><td><x-badge
:statut="$b->statut" /></td><td>{{ date_fr($b->soumis_le) }}</td>
                    </tr>
                @endforeach</tbody>
```

`$b->domaine?->nom ?? '—'` : l'opérateur `?->` évite une erreur si le besoin n'a pas de domaine, et `??` affiche un tiret à la place.

Étape 4 — Vérifier. Recharger la page (`Ctrl + F5`). Aucune compilation n'est nécessaire, car aucune nouvelle classe CSS n'a été utilisée.

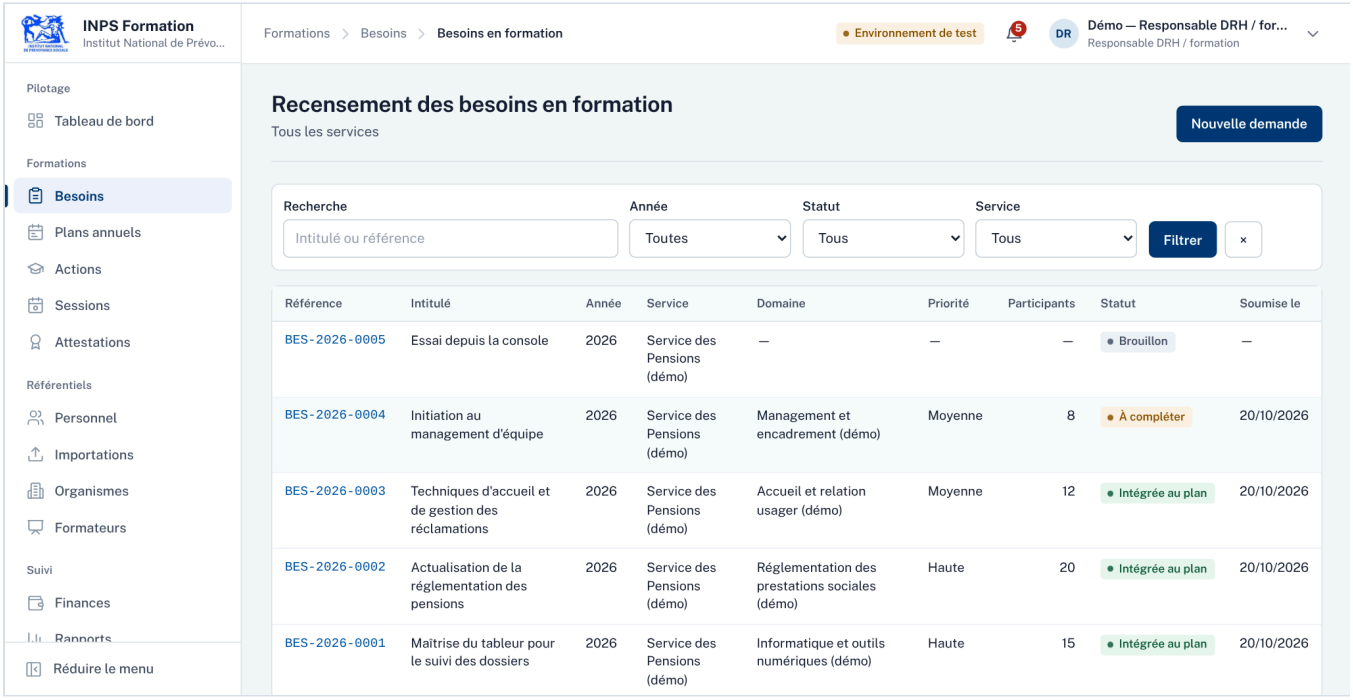


Figure 5 — Résultat de l'exercice 6.1 dans la copie de travail.

6.2 Ajouter un champ à un formulaire et à la base de données

Objectif : enregistrer la **date d'embauche** (facultative) des agents.

Un nouveau champ touche toujours les mêmes couches : **migration → modèle → validation → formulaire → affichage → test**.

Étape 1 — Créer la migration (jamais modifier une migration existante) :

```
php artisan make:migration ajouter_date_embauche_aux_agents --table=agents
```

Laravel crée `database/migrations/AAAA_MM_JJ_HHMMSS_ajouter_date_embauche_aux_agents.php` (la date et l'heure du jour forment le début du nom). On la complète :

`database/migrations/2026_10_21_080428_ajouter_date_embauche_aux_agents.php` — fichier de l'exercice (lignes 1 à 25)

```
<?php
```

```php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

/**
 * Ajoute la date d'embauche (facultative) à la fiche agent.
 */
return new class extends Migration
{
    public function up(): void
    {
        Schema::table('agents', function (Blueprint $table) {
            $table->date('date_embauche')->nullable()->after('fonction');
        });
    }

    public function down(): void
    {
        Schema::table('agents', function (Blueprint $table) {
            $table->dropColumn('date_embauche');
        });
    }
};
```

`->nullable()` est indispensable : les agents existants n'ont pas de date d'embauche. `->after('fonction')` place la colonne dans la table (simple confort de lecture). `down()` permet d'annuler.

Étape 2 — Appliquer : `php artisan migrate` (sortie obtenue : `2026_10_21_080428_ajouter_date_embauche_aux_agents ... DONE`).

Étape 3 — Modèle, validation, vues :

```
diff --git a/app/Http/Requests/AgentRequest.php b/app/Http/Requests/AgentRequest.php
index b0b43f0..478013a 100644
--- a/app/Http/Requests/AgentRequest.php
+++ b/app/Http/Requests/AgentRequest.php
@@ -28,6 +28,7 @@ public function rules(): array
             'direction_id' => ['nullable', 'integer', 'exists:directions,id'],
             'service_id' => ['nullable', 'integer', Rule::exists('services', 'id')->when($this-
>input('direction_id'), fn ($r) => $r->where('direction_id', $this->input('direction_id')))],
             'fonction' => ['nullable', 'string', 'max:190'],
+            'date_embauche' => ['nullable', 'date', 'before_or_equal:today'],
             'statut_professionnel_id' => ['nullable', 'integer', Rule::exists('nomenclatures', 'id')-
>where('type', 'statut_professionnel')],
             'email_pro' => ['nullable', 'email', 'max:190'],
             'telephone_pro' => ['nullable', 'string', 'max:40', 'regex:/^[0-9+ ().-]{6,40}$/'],
@@ -36,6 +37,7 @@ public function rules(): array

     public function messages(): array
     {
-        return ['service_id.exists' => 'Le service choisi n\'appartient pas à la direction
sélectionnée.', 'matricule.regex' => 'Le matricule ne peut contenir que des lettres, chiffres, tirets,
barres obliques et points.'];
+        return ['service_id.exists' => 'Le service choisi n\'appartient pas à la direction
sélectionnée.', 'matricule.regex' => 'Le matricule ne peut contenir que des lettres, chiffres, tirets,
barres obliques et points.',
+            'date_embauche.before_or_equal' => 'La date d\'embauche ne peut pas être dans le futur.'];
     }
 }
diff --git a/app/Models/Agent.php b/app/Models/Agent.php
index e46055c..d4e53a3 100644
--- a/app/Models/Agent.php
+++ b/app/Models/Agent.php
@@ -14,13 +14,13 @@
 class Agent extends Model
 {
     protected $fillable = [
-        'matricule', 'nom', 'prenom', 'direction_id', 'service_id', 'fonction',
```

```
+        'matricule', 'nom', 'prenom', 'direction_id', 'service_id', 'fonction', 'date_embauche',
         'statut_professionnel_id', 'email_pro', 'telephone_pro', 'source', 'reference_externe',
     ];

     protected function casts(): array
     {
-        return ['actif' => 'boolean', 'date_desactivation' => 'date'];
+        return ['actif' => 'boolean', 'date_desactivation' => 'date', 'date_embauche' => 'date'];
     }

     public function direction(): BelongsTo
diff --git a/resources/views/agents/form.blade.php b/resources/views/agents/form.blade.php
index a4ac7f9..156f770 100644
--- a/resources/views/agents/form.blade.php
+++ b/resources/views/agents/form.blade.php
@@ -11,6 +11,7 @@
                 <x-champ nom="direction_id" libelle="Direction / structure" type="select"
:valeur="$agent->direction_id" :options="$directions->pluck('nom', 'id')" />
                 <x-champ nom="service_id" libelle="Service" type="select" :valeur="$agent->service_id"
:options="$services->mapWithKeys(fn ($s) => [$s->id => $s->nom.' ('.$s->direction->code.')'])" />
                 <x-champ nom="statut_professionnel_id" libelle="Statut professionnel" type="select"
:valeur="$agent->statut_professionnel_id" :options="$statuts->pluck('libelle', 'id')" />
+                <x-champ nom="date_embauche" libelle="Date d'embauche (facultatif)" type="date"
:valeur="$agent->date_embauche" />
                 <x-champ nom="email_pro" libelle="E-mail professionnel (facultatif)" type="email"
:valeur="$agent->email_pro" />
                 <x-champ nom="telephone_pro" libelle="Téléphone professionnel (facultatif)"
:valeur="$agent->telephone_pro" />
             </div>
diff --git a/resources/views/agents/show.blade.php b/resources/views/agents/show.blade.php
index 4c9a0d8..3258235 100644
--- a/resources/views/agents/show.blade.php
+++ b/resources/views/agents/show.blade.php
@@ -20,6 +20,7 @@
                 <dt>Direction</dt><dd>{{ $agent->direction?->nom ?? '—' }}</dd>
                 <dt>Service</dt><dd>{{ $agent->service?->nom ?? '—' }}</dd>
                 <dt>Fonction</dt><dd>{{ $agent->fonction ?? '—' }}</dd>
+                <dt>Date d'embauche</dt><dd>{{ date_fr($agent->date_embauche) }}</dd>
                 <dt>Statut prof.</dt><dd>{{ $agent->statutProfessionnel?->libelle ?? '—' }}</dd>
                 <dt>E-mail pro.</dt><dd>{{ $agent->email_pro ?? '—' }}</dd>
                 <dt>Téléphone</dt><dd>{{ $agent->telephone_pro ?? '—' }}</dd>
```

- **Modèle** : ajouter le champ à `$fillable` (sinon il est ignoré à l'enregistrement — et signalé en développement), et le *cast* `date` pour manipuler une vraie date.
- **Validation** : `nullable` (facultatif), `date`, `before_or_equal:today` (pas dans le futur), avec un message en français.
- **Formulaire** : une ligne `<x-champ ... type="date">` ; le composant gère la valeur, l'ancienne saisie et l'erreur.
- **Fiche** : `date_fr()` affiche la date au format français.
- Le contrôleur **n'a pas besoin d'être modifié** : `AgentController::store` enregistre déjà `$r->validated()`.

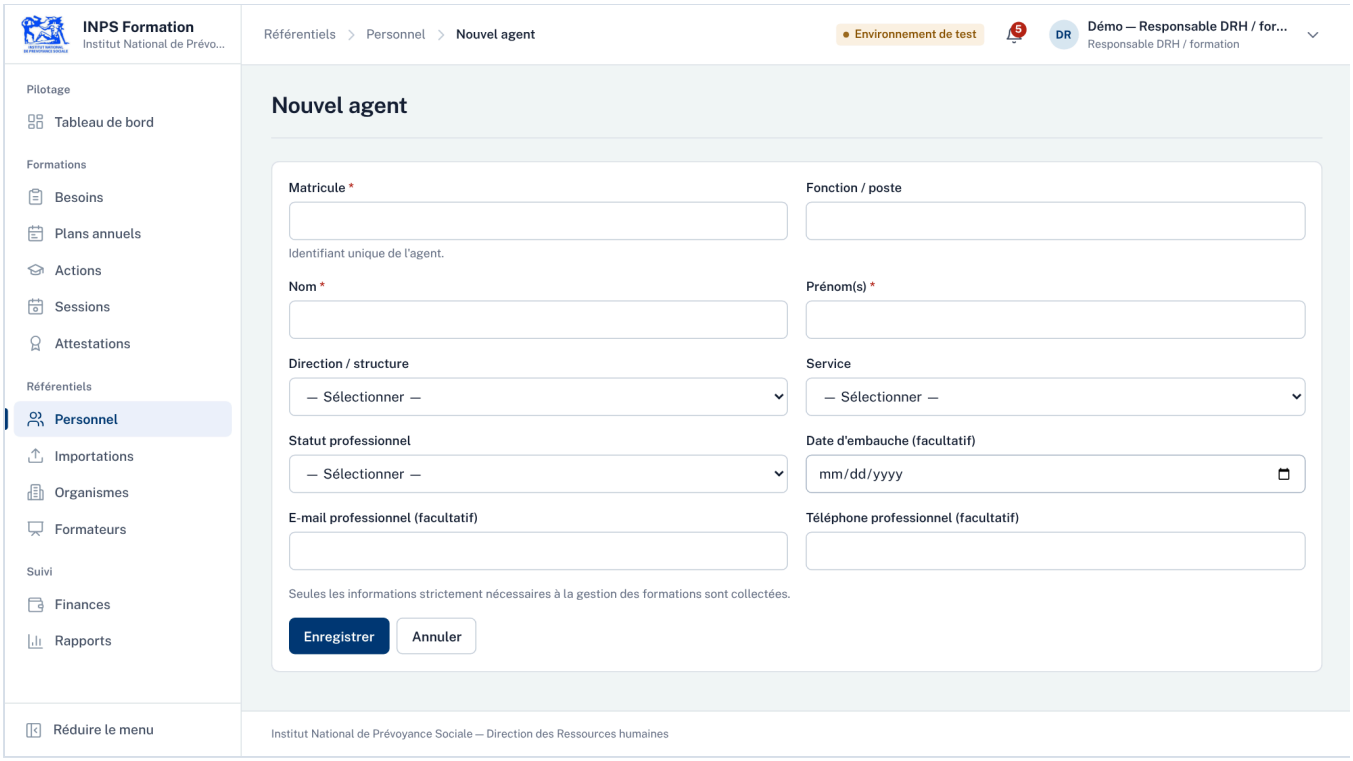


Figure 6 — Le nouveau champ dans le formulaire (copie de travail).

Étape 4 — Écrire le test (`php artisan make:test AgentDateEmbaucheTest`, puis remplacer le contenu) :

`tests/Feature/AgentDateEmbaucheTest.php` — fichier de l'exercice (lignes 1 à 30)

```php
<?php

namespace Tests\Feature;

use App\Models\Agent;
use Tests\TestCase;

class AgentDateEmbaucheTest extends TestCase
{
    public function test_la_date_d_embauche_est_enregistree_et_affichee(): void
    {
        $drh = $this->utilisateur('responsable_drh');
        $this->actingAs($drh)->post(route('agents.store'), [
            'matricule' => 'T0001', 'nom' => 'Test', 'prenom' => 'Awa', 'date_embauche' => '2015-03-02',
        ])->assertRedirect();

        $agent = Agent::where('matricule', 'T0001')->firstOrFail();
        $this->assertSame('2015-03-02', $agent->date_embauche->format('Y-m-d'));
        $this->actingAs($drh)->get(route('agents.show', $agent))->assertSee('02/03/2015');
    }

    public function test_une_date_d_embauche_future_est_refusee(): void
    {
        $this->actingAs($this->utilisateur('responsable_drh'))->post(route('agents.store'), [
            'matricule' => 'T0002', 'nom' => 'Test', 'prenom' => 'Moussa', 'date_embauche' => now()->addYear()->toDateString(),
        ])->assertSessionHasErrors('date_embauche');

        $this->assertDatabaseMissing('agents', ['matricule' => 'T0002']);
    }
}
```

`php artisan test --filter=AgentDateEmbaucheTest` → 2 tests réussis (6 assertions).

Pour aller plus loin : si la date doit aussi être importée, ajouter `date_embauche` aux colonnes facultatives de `ImportService::COLONNES['agents']`, la valider dans `validerAgent` et l'écrire dans `creerAgent` ; pour l'export, ajouter la colonne dans `AgentController::export`.

6.3 Créer une nouvelle fonctionnalité complète

Objectif : un **catalogue des salles de formation** géré par l'administrateur fonctionnel, avec sa propre permission. C'est l'exercice le plus complet : migration, modèle, permission, route, contrôleur, vue, menu et tests.

Étape 1 — Générer les fichiers de base avec Artisan

```
php artisan make:model Salle -m
php artisan make:controller Admin/SalleController
php artisan make:test SalleTest
```

`-m` crée la migration en même temps que le modèle. Sortie réelle :

```
INFO  Model [app/Models/Salle.php] created successfully.
INFO  Migration [database/migrations/2026_10_21_080512_create_salles_table.php] created successfully.
INFO  Controller [app/Http/Controllers/Admin/SalleController.php] created successfully.
INFO  Test [tests/Feature/SalleTest.php] created successfully.
```

Étape 2 — La table

`database/migrations/2026_10_21_080512_create_salles_table.php` — fichier de l'exercice (lignes 1 à 29)

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

/**
 * Catalogue des salles de formation (exercice de la formation technique).
 */
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('salles', function (Blueprint $table) {
            $table->id();
            $table->string('nom', 150)->unique();
            $table->string('lieu', 255)->nullable();
            $table->unsignedSmallInteger('capacite')->nullable();
            $table->text('equipements')->nullable();
            $table->boolean('actif')->default(true);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('salles');
    }
};
```

Étape 3 — Le modèle

`app/Models/Salle.php` — fichier de l'exercice (lignes 1 à 18)

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

/** Salle de formation (catalogue géré par l'administrateur fonctionnel). */
class Salle extends Model
{
    protected $table = 'salles';

    protected $fillable = ['nom', 'lieu', 'capacite', 'equipements', 'actif'];

    protected function casts(): array
    {
        return ['actif' => 'boolean', 'capacite' => 'integer'];
    }
}
```

Étape 4 — La permission

Toute nouvelle fonctionnalité protégée a sa permission dans le catalogue, `app/Support/CataloguePermissions.php` (constante `PERMISSIONS`), et on l'ajoute à la proposition du rôle concerné (constante `ROLES`, pour les **nouvelles** installations) :

```
diff --git a/app/Support/CataloguePermissions.php b/app/Support/CataloguePermissions.php
index a0a8ee9..58c27aa 100644
--- a/app/Support/CataloguePermissions.php
+++ b/app/Support/CataloguePermissions.php
@@ -57,6 +57,7 @@ final class CataloguePermissions
         'espace.personnel' => ['Espace agent', 'Consulter ses propres formations et attestations',
false],

         'parametres.gerer' => ['Paramétrage', 'Gérer les référentiels et paramètres métier', true],
+        'salles.gerer' => ['Paramétrage', 'Gérer le catalogue des salles de formation', false],
         'utilisateurs.gerer' => ['Administration', 'Gérer les comptes utilisateurs', true],
         'roles.gerer' => ['Administration', 'Gérer les rôles et permissions', true],
         'audit.voir' => ['Administration', 'Consulter le journal d\'audit', true],
@@ -70,7 +71,7 @@ final class CataloguePermissions
             'utilisateurs.gerer', 'roles.gerer', 'audit.voir', 'audit.exporter', 'systeme.superviser',
'tableau_bord.voir',
         ]],
         'admin_fonctionnel' => ['Administrateur fonctionnel', 'Référentiels, paramètres métier et
organisation.', [
-            'parametres.gerer', 'tableau_bord.voir', 'agents.voir_tous', 'agents.gerer',
'agents.importer', 'organismes.voir', 'organismes.gerer',
+            'parametres.gerer', 'tableau_bord.voir', 'agents.voir_tous', 'agents.gerer',
'agents.importer', 'organismes.voir', 'organismes.gerer', 'salles.gerer',
         ]],
         'responsable_drh' => ['Responsable DRH / formation', 'Gestion des besoins, plans, actions,
participants, évaluations et rapports.', [
             'tableau_bord.voir', 'agents.voir_tous', 'agents.gerer', 'agents.importer',
'agents.exporter',
diff --git a/resources/views/components/layout.blade.php b/resources/views/components/layout.blade.php
index 28d691a..4109318 100644
--- a/resources/views/components/layout.blade.php
+++ b/resources/views/components/layout.blade.php
@@ -35,6 +35,7 @@
             ['Paramètres', 'admin.parametres.index', 'admin.parametres.*', $p('parametres.gerer'),
'parametres'],
             ['Référentiels', ['admin.referentiels.index', 'directions'], 'admin.referentiels.*',
$p('parametres.gerer'), 'referentiels'],
             ['Circuits de validation', 'admin.circuits.index', 'admin.circuits.*',
$p('parametres.gerer'), 'circuits'],
```

```
+            ['Salles de formation', 'admin.salles.index', 'admin.salles.*', $p('salles.gerer'),
'organismes'],
             ['Utilisateurs', 'admin.utilisateurs.index', 'admin.utilisateurs.*',
$p('utilisateurs.gerer'), 'utilisateurs'],
             ['Rôles et permissions', 'admin.roles.index', 'admin.roles.*', $p('roles.gerer'), 'roles'],
             ['Journal d\'audit', 'admin.audit.index', 'admin.audit.*', $p('audit.voir'), 'audit'],
diff --git a/routes/web.php b/routes/web.php
index a680ea8..122a333 100644
--- a/routes/web.php
+++ b/routes/web.php
@@ -6,6 +6,7 @@
 use App\Http\Controllers\Admin\ParametreController;
 use App\Http\Controllers\Admin\ReferentielController;
 use App\Http\Controllers\Admin\RoleController;
+use App\Http\Controllers\Admin\SalleController;
 use App\Http\Controllers\Admin\SystemeController;
 use App\Http\Controllers\Admin\UtilisateurController;
 use App\Http\Controllers\AgentController;
@@ -247,6 +248,11 @@
             Route::get('/systeme', [SystemeController::class, 'index'])->name('systeme.index');
             Route::post('/systeme/sauvegarde', [SystemeController::class, 'sauvegarder'])-
>name('systeme.sauvegarde');
         });
+        Route::middleware('permission:salles.gerer')->group(function () {
+            Route::get('/salles', [SalleController::class, 'index'])->name('salles.index');
+            Route::post('/salles', [SalleController::class, 'store'])->name('salles.store');
+            Route::put('/salles/{salle}', [SalleController::class, 'update'])->name('salles.update');
+        });
         Route::middleware('permission:parametres.gerer')->group(function () {
             Route::get('/parametres', [ParametreController::class, 'index'])->name('parametres.index');
             Route::put('/parametres', [ParametreController::class, 'update'])->name('parametres.update');
```

Le même extrait montre aussi les deux autres modifications de fichiers existants : les routes (étape 6) et le menu (étape 8).

Puis on enregistre la permission en base en relançant le référentiel (sans risque, idempotent) : `php artisan db:seed`.

Attention

Sur une installation **existante**, le seeder crée la permission mais **ne modifie pas** les rôles déjà en place (pour respecter les choix de l'administrateur). Il faut donc attribuer la permission au rôle voulu dans *Administration › Rôles et permissions* (compte disposant de `roles.gerer`, et ne détenant pas lui-même le rôle modifié).



Figure 7 — La nouvelle permission « salles.gerer » cochée pour le rôle Administrateur fonctionnel.

Étape 5 — Le contrôleur

`app/Http/Controllers/Admin/SalleController.php` — fichier de l'exercice (lignes 1 à 49)

```php
<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Models\Salle;
use App\Services\Audit;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;
use Illuminate\View\View;

/** Catalogue des salles de formation (permission « salles.gerer »). */
class SalleController extends Controller
{
    public function index(): View
    {
        return view('admin.salles.index', ['salles' => Salle::orderBy('nom')->get()]);
    }

    public function store(Request $r): RedirectResponse
    {
        $salle = Salle::create($this->valider($r));
        Audit::enregistrer('salle.creation', 'salle', "Création de la salle « {$salle->nom} »");

        return back()->with('succes', 'Salle enregistrée.');
    }

    public function update(Request $r, Salle $salle): RedirectResponse
    {
        $salle->update($this->valider($r, $salle));
        Audit::enregistrer('salle.modification', 'salle', "Modification de la salle « {$salle->nom} »");

        return back()->with('succes', 'Salle mise à jour.');
    }

    private function valider(Request $r, ?Salle $salle = null): array
    {
        $r->merge(['actif' => $r->boolean('actif')]);

        return $r->validate([
            'nom' => ['required', 'string', 'max:150', Rule::unique('salles', 'nom')->ignore($salle?-
>id)],
            'lieu' => ['nullable', 'string', 'max:255'],
            'capacite' => ['nullable', 'integer', 'min:1', 'max:2000'],
            'equipements' => ['nullable', 'string', 'max:2000'],
            'actif' => ['boolean'],
        ]);
    }
}
```

Schéma habituel : valider → enregistrer → journaliser (`Audit::enregistrer`) → rediriger avec un message. La règle `Rule::unique(...)->ignore($salle?->id)` interdit deux salles de même nom, sauf la salle modifiée elle-même. Ici, la logique est assez simple pour rester dans le contrôleur ; dès qu'apparaissent des règles métier (par exemple « une salle ne peut pas accueillir deux sessions en même temps »), il faut créer un `app/Services/SalleService.php`.

Étape 6 — Les routes

Dans `routes/web.php`, à l'intérieur du groupe `administration` (donc préfixe `/administration`, noms `admin.…`), un groupe protégé par la nouvelle permission (voir l'extrait de l'étape 4). Vérification : `php artisan route:list --path=salles`.

Étape 7 — La vue

`resources/views/admin/salles/index.blade.php`, construite uniquement avec les composants existants (`x-layout`, `x-entete`, `x-carte`, `x-champ`, `x-badge`, `x-vide`) : elle hérite donc automatiquement du style et de l'accessibilité de l'application.

`resources/views/admin/salles/index.blade.php` — fichier de l'exercice (lignes 1 à 40)

```
<x-layout titre="Salles de formation">
    <x-entete titre="Salles de formation" sous-titre="Catalogue des salles disponibles pour les sessions."
/>

    <x-carte titre="Ajouter une salle" class="mb-5">
        <form method="POST" action="{{ route('admin.salles.store') }}" class="space-y-4">
            @csrf
            <div class="grid gap-4 md:grid-cols-3">
                <x-champ nom="nom" libelle="Nom de la salle" :requis="true" />
                <x-champ nom="lieu" libelle="Bâtiment / étage" />
                <x-champ nom="capacite" libelle="Capacité (places)" type="number" :attrs="['min' => 1]" />
                <x-champ class="md:col-span-3" nom="equipements" libelle="Équipements" type="textarea"
aide="Vidéoprojecteur, ordinateurs, connexion…" />
            </div>
            <input type="hidden" name="actif" value="1">
            <button class="btn btn-primaire">Enregistrer la salle</button>
        </form>
    </x-carte>

    <x-carte titre="Salles enregistrées ({{ $salles->count() }})" sans-marge>
        @if ($salles->isEmpty())
            <x-vide message="Aucune salle enregistrée : ajoutez la première avec le formulaire ci-dessus."
/>
        @else
            <div class="overflow-x-auto">
                <table>
                    <thead><tr><th>Nom</th><th>Lieu</th><th class="text-right">Capacité</th>
<th>Équipements</th><th>État</th></tr></thead>
                    <tbody>
                    @foreach ($salles as $s)
                        <tr>
                            <td class="font-semibold">{{ $s->nom }}</td>
                            <td>{{ $s->lieu ?? '—' }}</td>
                            <td class="text-right">{{ $s->capacite ?? '—' }}</td>
                            <td>{{ $s->equipements ?? '—' }}</td>
                            <td><x-badge :texte="$s->actif ? 'Active' : 'Inactive'" :couleur="$s->actif ?
'vert' : 'gris'" /></td>
                        </tr>
                    @endforeach
                    </tbody>
                </table>
            </div>
        @endif
    </x-carte>
</x-layout>
```

Étape 8 — Le menu

Une ligne dans le tableau `$menu` de `resources/views/components/layout.blade.php` (voir l'extrait de l'étape 4). L'entrée n'apparaît qu'aux détenteurs de `salles.gerer`.

Étape 9 — Les tests

`tests/Feature/SalleTest.php` — fichier de l'exercice (lignes 1 à 31)

```
<?php

namespace Tests\Feature;
```

```php
use App\Models\Salle;
use Tests\TestCase;

class SalleTest extends TestCase
{
    public function test_l_administrateur_fonctionnel_gere_les_salles(): void
    {
        $admin = $this->utilisateur('admin_fonctionnel');
        $this->actingAs($admin)->get(route('admin.salles.index'))->assertOk()->assertSee('Salles de
formation');
        $this->actingAs($admin)->post(route('admin.salles.store'), ['nom' => 'Salle Niger', 'capacite' =>
25, 'actif' => 1])
             ->assertSessionHas('succes');
        $this->assertDatabaseHas('salles', ['nom' => 'Salle Niger', 'capacite' => 25]);
        $this->assertDatabaseHas('audit_logs', ['action' => 'salle.creation']);
    }

    public function test_le_nom_d_une_salle_est_unique(): void
    {
        Salle::create(['nom' => 'Salle Niger']);
        $this->actingAs($this->utilisateur('admin_fonctionnel'))->post(route('admin.salles.store'), ['nom'
=> 'Salle Niger'])
             ->assertSessionHasErrors('nom');
    }

    public function test_un_chef_de_service_n_a_pas_acces(): void
    {
        $this->actingAs($this->utilisateur('chef_service'))->get(route('admin.salles.index'))-
>assertForbidden();
    }
}
```

Les trois tests vérifient le cas normal, la règle d'unicité et le **refus d'accès** (toujours tester qu'un profil non autorisé est refusé). Résultat réel :

```
   PASS  Tests\Feature\SalleTest
  ✓ l administrateur fonctionnel gere les salles                         4.15s
  ✓ le nom d une salle est unique                                        0.03s
  ✓ un chef de service n a pas acces                                     0.03s
  Tests:    3 passed (8 assertions)
  Duration: 4.33s
```

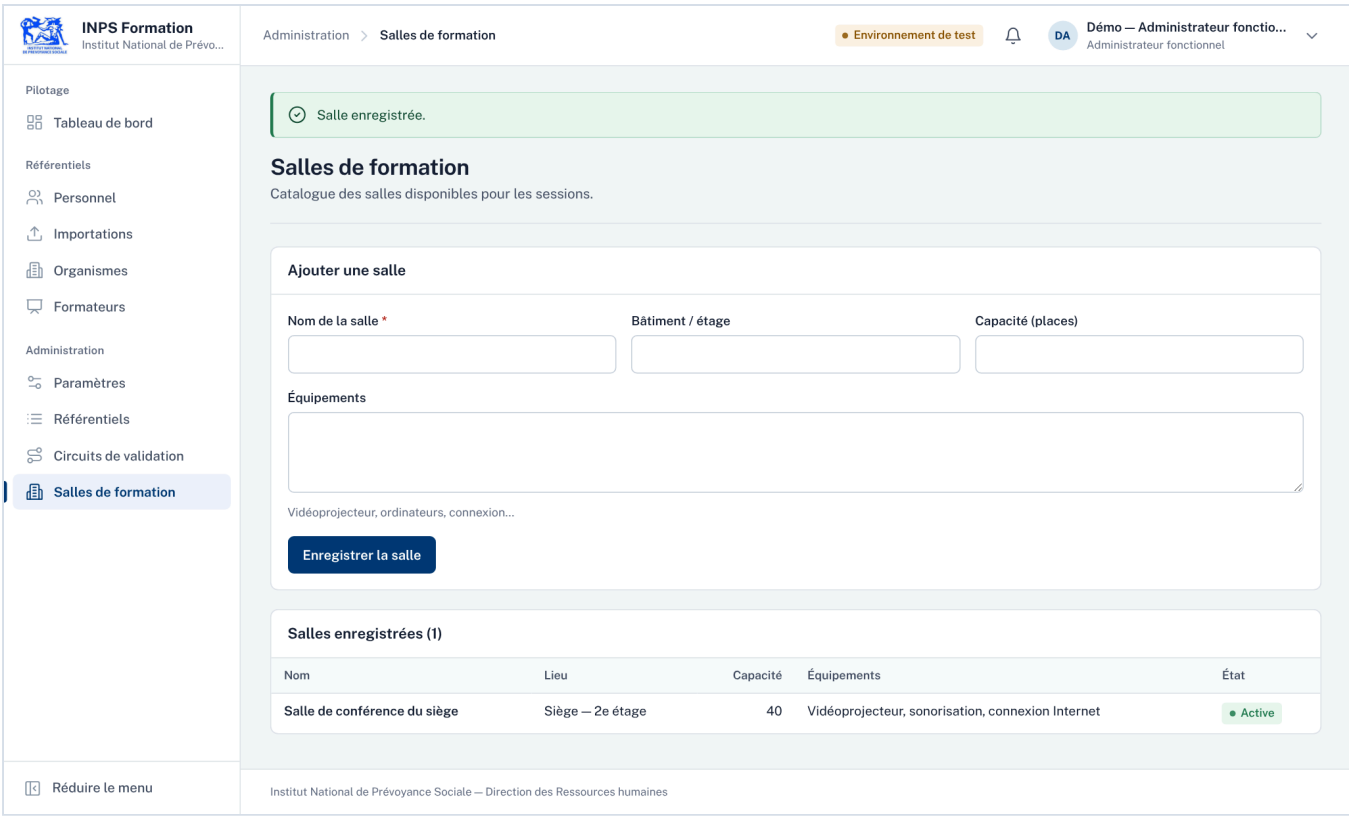


Figure 8 — La nouvelle fonctionnalité, utilisée par l'administrateur fonctionnel (copie de travail).

Si le CSS de la nouvelle vue utilise des classes Tailwind jamais employées ailleurs, il faut relancer `npm run build` (les classes sont générées d'après le contenu des vues). Ici, toutes les classes existaient déjà.

6.4 Mémo : ajouter une route, un contrôleur, un modèle, une vue

Je veux…	Commande / fichier	À ne pas oublier
une table	`php artisan make:migration create_xxx_table`	`down()` ; clés étrangères avec la bonne règle de suppression
une colonne	`php artisan make:migration ajouter_xxx_aux_yyy --table=yyy`	`nullable()` si des lignes existent déjà
un modèle	`php artisan make:model Xxx` (ou `-m` avec migration)	`$table` si le nom est français, `$fillable`, `casts()`
un contrôleur	`php artisan make:controller XxxController` (ou `Admin/XxxController`)	`authorize()`, validation, appel au service
une validation de formulaire	`php artisan make:request XxxRequest`	`authorize()` doit renvoyer `true` pour les profils autorisés (par défaut `false` !)
une policy	`php artisan make:policy XxxPolicy --model=Xxx`	nom = modèle + `Policy` pour la découverte automatique
un service	créer `app/Services/XxxService.php` à la main	transactions, `RegleMetierException`, `Audit::enregistrer`
une route	`routes/web.php`, dans le bon groupe	`->name(...)`, middleware `permission:`
une vue	`resources/views/module/page.blade.php`	`<x-layout>`, `@csrf` dans les formulaires
une entrée de menu	tableau `$menu` de `components/layout.blade.php`	la permission en 4e colonne

Je veux…	Commande / fichier	À ne pas oublier
une permission	`app/Support/CataloguePermissions.php` + `php artisan db:seed`	attribuer la permission au rôle dans l'écran *Rôles*
une commande planifiée	`php artisan make:command` + `routes/console.php`	le planificateur doit tourner
un test	`php artisan make:test XxxTest`	cas autorisé **et** cas refusé

6.5 Modifier les droits d'accès

Trois niveaux, du plus simple au plus technique :

1. **Changer qui a quoi** (aucun code) : *Administration › Rôles et permissions* → cocher/décocher. Effet immédiat, tracé dans l'audit. Exemple : donner `rapports.voir` au rôle Chef de service.
2. **Créer un nouveau rôle** (aucun code) : même écran, « Nouveau rôle », puis cocher ses permissions et l'attribuer aux comptes dans *Utilisateurs*.
3. **Protéger différemment une fonctionnalité** (code) : - changer la permission exigée par une route : le middleware `permission:` dans `routes/web.php` ; - changer une règle par enregistrement : la policy (ex. `BesoinFormationPolicy::view`) **et** le scope correspondant (`BesoinFormation::visiblesPar`) pour que liste et fiche restent cohérentes ; - masquer le bouton correspondant dans la vue (`@permission`, `@can`) ; - adapter les tests de `tests/Feature/AutorisationTest.php`.

> **Attention**
>
> Ne jamais ajouter de condition du type « si l'utilisateur s'appelle… » ou « si c'est l'administrateur alors tout est permis » : le projet repose sur le **refus par défaut** et sur des permissions explicites.

6.6 Corriger un bug : un cas réel de ce projet

Le symptôme. En version 1.0.0, une fois un logo déposé dans les paramètres, il s'affichait pour les administrateurs mais apparaissait comme une **image cassée** pour tous les autres utilisateurs.

Reproduire. Se connecter avec un compte sans la permission `parametres.gerer` (par exemple un chef de service) après avoir déposé un logo : l'image ne s'affiche pas.

Chercher la cause. Ouvrir l'adresse de l'image dans un nouvel onglet (clic droit › « Ouvrir l'image dans un nouvel onglet ») : la réponse est une page **403**. On cherche alors comment l'adresse est produite (code de la version 1.0.0) :

```
// routes/web.php (version 1.0.0), dans le groupe Route::middleware('permission:parametres.gerer')
Route::get('/parametres/logo', [ParametreController::class, 'logo'])->name('parametres.logo');

{{-- resources/views/components/layout.blade.php (version 1.0.0), menu latéral affiché à TOUS les
utilisateurs --}}
$logo = \App\Models\Parametre::valeur('institution.logo');
@if ($logo)
    <img src="{{ route('admin.parametres.logo') }}" alt="Logo" class="h-9 w-9 object-contain bg-white
rounded">
@else
```

La vue `layout.blade.php`, affichée à **tous** les utilisateurs, appelait la route `admin.parametres.logo`, déclarée **dans** le groupe protégé par `permission:parametres.gerer`. Le middleware `ExigerPermission` refusait donc l'image à tous ceux qui n'avaient pas cette permission. C'est une erreur de conception (une

ressource publique placée derrière une permission d'administration), pas une erreur de syntaxe : aucun message n'apparaît dans les journaux.

Écrire un test qui reproduit le problème, avant de corriger : un utilisateur quelconque (ou même un visiteur non connecté, puisque le logo s'affiche aussi sur la page de connexion) doit obtenir le logo. C'est ce que vérifie désormais `InterfaceIdentiteTest::test_ressources_institutionnelles_absentes_puis_deposees` (après `auth()->logout()`, `GET /identite/logo` doit répondre 200 avec le bon type de contenu).

Corriger. En version 1.1.0, une route **publique en lecture seule**, limitée aux deux types autorisés, sert le logo et la photographie :

`routes/web.php` — lignes 40 à 41

```
// Logo et photographie institutionnels (publics : affichés sur la page de connexion)
Route::get('/identite/{type}', [IdentiteController::class, 'fichier'])->whereIn('type', ['logo',
'photo'])->name('identite.fichier');
```

`app/Http/Controllers/IdentiteController.php` — lignes 1 à 29

```php
<?php

namespace App\Http\Controllers;

use App\Support\Identite;
use Symfony\Component\HttpFoundation\BinaryFileResponse;

/**
 * Logo et photographie institutionnels : publics par nature (affichés sur la page de connexion),
 * servis sans authentification. Aucun autre fichier n'est accessible par cette route.
 */
class IdentiteController extends Controller
{
    public function fichier(string $type): BinaryFileResponse
    {
        $chemin = Identite::chemin($type);
        abort_unless($chemin, 404);
        $mime = mime_content_type($chemin) ?: 'application/octet-stream';
        abort_unless(in_array($mime, ['image/png', 'image/jpeg', 'image/webp', 'image/svg+xml'], true),
404);

        return response()->file($chemin, [
            'Content-Type' => $mime,
            'Cache-Control' => 'public, max-age=86400',
            'X-Content-Type-Options' => 'nosniff',
            // Un SVG ne doit jamais exécuter de script
            'Content-Security-Policy' => "default-src 'none'; style-src 'unsafe-inline'; sandbox",
        ]);
    }
}
```

La correction est volontairement restreinte : seuls `logo` et `photo` sont acceptés (`whereIn`), seuls des types d'image sont servis, avec une politique de sécurité qui interdit tout script ; l'ancienne route a été supprimée.

Vérifier : `php artisan test` (toute la suite), puis contrôle à l'écran avec un compte non administrateur.

Méthode générale de correction

1. **Reproduire** le problème de façon fiable (quel compte, quelle page, quelles données).
2. **Lire le message** : écran (en développement, `APP_DEBUG=true` affiche le détail) ou `storage\logs\laravel-AAAA-MM-JJ.log`.
3. **Localiser** : URL → `route:list` → contrôleur → service ; la pile d'appels de l'erreur indique le fichier et la ligne.

4. **Comprendre** avant de modifier : le comportement est-il un bug, ou une règle métier voulue (souvent signalée par une `RegleMetierException` avec un message clair) ?
5. **Écrire un test** qui échoue tant que le bug existe.
6. **Corriger** au bon endroit (règle métier → service ; droits → route/policy ; affichage → vue).
7. **Lancer toute la suite de tests**, vérifier à l'écran, noter la correction dans `CHANGELOG.md`.

Outils de diagnostic : `dd($variable)` (affiche et arrête) ou `dump($variable)` dans le code pendant le développement — à retirer ensuite ; `Log::info('message', ['cle' => $valeur])` écrit dans le journal ; `DB::listen(fn ($q) => logger($q->sql))` affiche les requêtes SQL (c'est ainsi qu'ont été obtenues celles de la section 2.17).

6.7 Écrire et exécuter des tests

Configuration. Les tests utilisent une base **séparée**, `inps_formation_test` (définie dans `phpunit.xml`, surchargeable par un fichier `.env.testing`). `TestCase` refuse de s'exécuter si le nom de la base ne se termine pas par `_test` : impossible d'effacer la base réelle par erreur. Chaque test repart d'une base vide (`RefreshDatabase`) avec le référentiel initial chargé, plus deux services et l'année 2026 créés par `TestCase::setUp`.

`tests/TestCase.php` — lignes 14 à 51

```
/**
 * Base des tests : base MySQL de test dédiée, référentiel initial chargé, fabrique d'utilisateurs par
rôle.
 */
abstract class TestCase extends BaseTestCase
{
    use RefreshDatabase;

    protected bool $seed = true;

    protected string $seeder = ReferentielInitialSeeder::class;

    protected Service $service1;

    protected Service $service2;

    protected function setUp(): void
    {
        parent::setUp();
        if (config('database.connections.mysql.database') !== 'inps_formation_test' && !
str_ends_with((string) config('database.connections.mysql.database'), '_test')) {
            $this->fail('Les tests doivent utiliser une base dont le nom se termine par « _test ».');
        }
        $d = Direction::create(['code' => 'T-DIR', 'nom' => 'Direction de test', 'actif' => true]);
        $this->service1 = Service::create(['direction_id' => $d->id, 'code' => 'T-S1', 'nom' => 'Service
test 1', 'actif' => true]);
        $this->service2 = Service::create(['direction_id' => $d->id, 'code' => 'T-S2', 'nom' => 'Service
test 2', 'actif' => true]);
        AnneeReference::create(['annee' => 2026, 'statut' => 'ouverte']);
    }

    protected function utilisateur(string $role, ?Service $service = null, array $attributs = []): User
    {
        $u = new User(array_merge(['name' => 'Test '.$role, 'email' =>
$role.'.'.uniqid().'@exemple.invalid', 'password' => 'Mot-de-passe-Test-2026!',
            'service_id' => ($service ?? $this->service1)->id], $attributs));
        $u->actif = true;
        $u->save();
        $u->roles()->attach(Role::where('code', $role)->value('id'));

        return $u;
```

```
    }
}
```

Exécuter :

Commande	Effet
`php artisan test`	Toute la suite (102 tests, environ 30 secondes).
`php artisan test --filter=BesoinPlanTest`	Un fichier de tests.
`php artisan test --filter=test_cycle_de_vie`	Les tests dont le nom contient ce texte.
`php artisan test --stop-on-failure`	S'arrête au premier échec.

Écrire un test : une méthode publique dont le nom commence par `test_`, dans `tests/Feature/` (comportement de l'application par requêtes HTTP ou appels de services) ou `tests/Unit/` (petites fonctions isolées). Outils principaux :

Outil	Rôle
`$this->utilisateur('chef_service')`	Crée un compte actif avec ce rôle (méthode de `TestCase`).
`$this->actingAs($u)->get(...)`, `->post(...)`	Simule une requête de cet utilisateur (la protection CSRF est désactivée automatiquement pendant les tests).
`->assertOk()`, `->assertForbidden()`, `->assertRedirect()`	Vérifie le code de réponse (200, 403, 302).
`->assertSee('texte')`	Le texte est dans la page.
`->assertSessionHasErrors('champ')`	La validation a refusé ce champ.
`$this->assertDatabaseHas('table', [...])`	Une ligne correspondante existe.
`$this->expectException(RegleMetierException::class)`	L'appel suivant doit être refusé par une règle métier.

Les 12 fichiers de tests existants sont autant d'exemples ; `tests/Feature/BesoinPlanTest.php` est le plus complet.

6.8 Sauvegarder et restaurer

Le projet (le code)

Le code est déjà dans l'archive ZIP livrée. Pour **tes** modifications, le plus sûr est Git (section 6.10) ; à défaut, une copie datée du dossier du projet **sans** `vendor` et `node_modules` (qui se réinstallent), mais **avec** `.env` conservé à part, en lieu sûr.

Les données : la commande intégrée

```
php artisan inps:sauvegarde
```

Sortie réelle (copie de travail, sans `BACKUP_PASSWORD`) :

```
Sauvegarde créée : …/storage/app/sauvegardes/inps-formation-20261021-080733.zip (28 Ko)
BACKUP_PASSWORD n'est pas défini : l'archive n'est PAS chiffrée.
```

L'archive contient la base (`base.sql`), les documents déposés (`fichiers/`) et le manifeste d'intégrité. Sous Windows, renseigner d'abord `MYSQLDUMP_PATH` et `MYSQL_CLIENT_PATH` dans `.env` (section 4.17), et définir `BACKUP_PASSWORD` pour chiffrer.

Restauration (⚠ remplace la base cible) — de préférence d'abord dans une base **de vérification** :

```
php artisan inps:restaurer storage\app\sauvegardes\inps-formation-20261021-080733.zip --
base=inps_formation_verif
```

La commande demande de retaper le nom de la base pour confirmer. Le test automatique `SauvegardeRestaurationTest` vérifie qu'une sauvegarde chiffrée se restaure à l'identique dans une base distincte.

Les données : sauvegarde manuelle avec mysqldump

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" -u root -p --single-transaction --routines --
default-character-set=utf8mb4 inps_formation > C:\sauvegardes\inps_formation_2026-10-21.sql
```

et copier le dossier `storage\app\private` (documents). Restauration :

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p --default-character-set=utf8mb4
inps_formation < C:\sauvegardes\inps_formation_2026-10-21.sql
```

À retenir

Une sauvegarde n'a de valeur que si sa **restauration a été testée**. Conserve aussi une copie **hors du serveur**, et garde la valeur de `APP_KEY` et de `BACKUP_PASSWORD` en lieu sûr : sans `BACKUP_PASSWORD`, une archive chiffrée est irrécupérable.

6.9 Préparer une mise en production sécurisée

La procédure serveur complète (Linux, Nginx/Apache, droits des fichiers, HTTPS, cron) est dans `docs/INSTALLATION_PRODUCTION.md` ; la checklist finale (section 10 de ce document) est reprise ici :

- `APP_ENV=production`, `APP_DEBUG=false`, `FORCE_HTTPS=true`, `SESSION_SECURE_COOKIE=true` ;
- `APP_KEY` générée et sauvegardée hors serveur ;
- `.env` protégé, absent de tout dépôt ; aucun secret dans le code ;
- MySQL non accessible depuis le réseau ; compte applicatif aux privilèges limités ;
- déclencheurs du journal d'audit installés (`php artisan inps:sante` → OK) ;
- certificat HTTPS valide ; HTTP redirigé vers HTTPS ;
- racine web = dossier `public/` ; `storage/` et `.env` inaccessibles par URL ;
- planificateur actif (`schedule:run` chaque minute) ;
- `BACKUP_PASSWORD` défini ; première sauvegarde faite ; **restauration testée** ; copie hors serveur ;
- administrateur technique créé ; **aucun compte ni donnée de démonstration** ;
- rôles et circuits de validation configurés selon les décisions de l'INPS ; paramètres « à valider » confirmés (`docs/DECISIONS_A_VALIDER.md`) ;
- SMTP configuré et testé, ou e-mails laissés désactivés ;
- recette fonctionnelle signée (`docs/RECETTE_INPS.md`).

Commandes d'optimisation à lancer sur le serveur après chaque déploiement :

```
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan optimize
```

(`optimize` met en cache la configuration, les routes et les vues ; après toute modification de `.env`, relancer `php artisan optimize`.)

Le contrôle de santé, tel qu'il s'affiche sur la copie de travail (environnement de développement) :

```
  OK      Connexion à la base (mysql)
  OK      Migrations appliquées (11)
  OK      Protection du journal d'audit (déclencheurs)
  OK      Au moins un administrateur technique actif
  OK      Clé d'application définie
  OK      Stockage privé accessible en écriture
  ATTENTION SESSION_ENCRYPT=false
  ATTENTION Aucun serveur SMTP configuré (MAIL_MAILER=log) : seules les notifications internes sont
actives
  OK      Dernière sauvegarde : 2026-10-21T08:07:33+00:00
  ATTENTION BACKUP_PASSWORD non défini : sauvegardes non chiffrées
  ATTENTION Planificateur (schedule:run) non détecté depuis 25 h
```

En production, toutes les lignes doivent être « OK » ; les « ATTENTION » ci-dessus sont normales en développement (pas de SMTP, pas de chiffrement des sauvegardes, pas de planificateur).

6.10 Conseil : utiliser Git

Git enregistre l'historique de chaque modification et permet de revenir en arrière. Sur ton poste (après installation de « Git for Windows ») :

```
cd C:\dev\inps-formation-1.1.0\inps-formation
git init
git add -A
git commit -m "Version 1.1.0 livrée"
```

Le fichier `.gitignore` du projet exclut déjà `.env`, `vendor/`, `node_modules/` et les fichiers générés. Ensuite, après chaque modification réussie : `git add -A` puis `git commit -m "Description de la modification"`. Pour voir ce qui a changé : `git diff`. Les exercices de cette partie ont été suivis exactement de cette façon.

Partie 7 — La formation vidéo

7.1 Ce qui a été produit

Les vidéos ont été **réellement produites** sous forme de fichiers MP4 (H.264, 1920 × 1080, son AAC), un fichier par épisode. Chaque épisode combine :

- des **diapositives** dont les extraits de code sont lus directement dans les fichiers du projet, avec leurs vrais numéros de lignes (même mécanisme que ce document) ;
- des **démonstrations enregistrées** dans l'application en fonctionnement, avec les comptes de démonstration (curseur visible, zones encadrées) ;
- une **narration en français** produite par une **voix de synthèse** (Piper, voix « fr-siwis-medium », licence CC-BY 4.0), et non par une voix humaine ;
- des **sous-titres français** intégrés au fichier (piste activable dans le lecteur) et fournis à part en `.srt`.

Limites à connaître

- La voix est synthétique : certains mots techniques ou sigles peuvent être mal prononcés. Les sous-titres reprennent le texte exact de la narration.
- La qualité de la voix n'a pas pu être vérifiée à l'écoute pendant la production (aucun moyen d'écoute disponible dans l'environnement) ; seule la présence d'un signal de parole a été contrôlée.
- Les démonstrations ont été enregistrées dans l'environnement de préparation (Linux, Chromium) et non sur ton poste Windows ; les écrans de l'application sont identiques, mais les manipulations propres à Windows (invite de commandes, services) sont montrées sur des diapositives.
- Les épisodes 10 et 11 montrent une **copie de travail** dans laquelle les exercices ont été réalisés ; ces modifications ne font pas partie de l'application livrée.

7.2 Les épisodes

N°	Épisode	Niveau	Durée	Fichier
01	Présentation et architecture générale	Débutant	4 min 54 s	`E01-presentation-architecture.mp4`
02	Installer, démarrer et mettre à jour sous Windows	Débutant	4 min 28 s	`E02-installation-windows.mp4`
03	Le trajet d'une requête : point d'entrée, routes et middlewares	Débutant → intermédiaire	4 min 36 s	`E03-requete-routes-middlewares.mp4`
04	Contrôleur, validation, policy et service : déclarer un besoin	Intermédiaire	4 min 41 s	`E04-controleur-validation-service.mp4`
05	La base de données : migrations, tables, relations et modèles	Intermédiaire	5 min 05 s	`E05-base-de-donnees.mp4`
06	Sécurité : authentification, rôles, permissions et journal d'audit	Intermédiaire	4 min 21 s	`E06-securite-roles-permissions.mp4`
07	L'interface : Blade, composants, Tailwind CSS, JavaScript et Vite	Intermédiaire	4 min 06 s	`E07-interface-blade-css-js.mp4`
08	Les modules métier (1) : du plan annuel aux présences	Intermédiaire	3 min 55 s	`E08-modules-plan-a-presence.mp4`

N°	Épisode	Niveau	Durée	Fichier
09	Les modules métier (2) : évaluations, attestations, finances, documents, rapports, notifications	Intermédiaire	3 min 30 s	`E09-modules-evaluations-finances-rapports.mp4`
10	Maintenance (1) : modifier une page et ajouter un champ	Intermédiaire → avancé	3 min 58 s	`E10-maintenance-page-et-champ.mp4`
11	Maintenance (2) : créer une fonctionnalité complète	Avancé	3 min 39 s	`E11-maintenance-nouvelle-fonctionnalite.mp4`
12	Tests, débogage, sauvegardes et mise en production	Avancé	4 min 32 s	`E12-tests-debogage-sauvegarde-production.mp4`
	Total		**51 min 51 s**	

Conseil de lecture : regarde un épisode, puis lis la partie correspondante de ce document avec le code ouvert dans VS Code. Dans VLC, les sous-titres s'activent par *Sous-titres › Piste de sous-titres*.

7.3 Réenregistrer les vidéos avec ta propre voix

Si tu préfères une narration humaine (pour une présentation interne, par exemple), le kit fourni permet de refaire les épisodes sans repartir de zéro :

Élément du kit	Usage
Les scripts complets (annexe D de ce document)	Le texte exact de chaque scène, dans l'ordre, avec ce qui est affiché.
Les fichiers `.srt`	Le même texte, minuté ; modifiables dans un éditeur de texte.
Les images des diapositives (`diapositives/E…/NN.png`)	Chaque diapositive en 1920 × 1080, numérotée comme la scène.

Méthode simple avec OBS Studio (gratuit, Windows)

1. Installe OBS Studio, crée une scène « Capture d'écran » et vérifie le micro (Paramètres › Audio).
2. Pour les diapositives : ouvre les images `NN.png` d'un épisode dans la visionneuse de photos Windows, en plein écran ; pour les démonstrations, ouvre l'application dans le navigateur avec le compte indiqué.
3. Lance l'enregistrement, lis le texte de la scène en affichant l'image ou en refaisant la manipulation, passe à la scène suivante.
4. Coupe les hésitations avec un éditeur simple (Clipchamp, intégré à Windows 11, ou Shotcut).

Méthode « voix seule » : enregistre uniquement ta voix scène par scène (Audacity, gratuit), puis assemble dans Clipchamp ou Shotcut les images `NN.png` avec tes fichiers audio ; la durée de chaque image s'aligne sur celle de ta piste.

Méthode par captures d'écran animées : pour une démonstration, l'enregistreur intégré de Windows 11 (outil Capture d'écran, mode vidéo, ou Xbox Game Bar avec `Windows + Alt + R`) suffit.

Glossaire

Terme	Définition dans le contexte du projet
Artisan	La console de Laravel : `php artisan <commande>`.
Autoload / PSR-4	Mécanisme de Composer qui trouve le fichier d'une classe à partir de son nom (`App\Services\BesoinService` → `app/Services/BesoinService.php`).
Blade	Moteur de gabarits de Laravel (`*.blade.php`).
Cast	Conversion automatique d'une colonne par Eloquent (date, booléen, énumération, décimal).
Composant Blade	Morceau de vue réutilisable, appelé par `<x-nom>` (dossier `resources/views/components`).
Contrôleur	Classe qui reçoit une requête et renvoie une réponse (`app/Http/Controllers`).
CSP	*Content Security Policy* : en-tête qui limite ce que le navigateur peut exécuter ; ici, uniquement les scripts de l'application.
CSRF	Attaque qui fait envoyer un formulaire à l'insu de l'utilisateur ; bloquée par le jeton `@csrf`.
Eager loading	Chargement anticipé des relations (`with([...])`) pour éviter une requête par ligne.
Eloquent	L'ORM de Laravel : les tables sont manipulées comme des objets PHP (modèles).
Énumération (enum)	Liste fermée de valeurs (`StatutBesoin`…), avec libellé et couleur.
FormRequest	Classe de validation d'un formulaire (`app/Http/Requests`).
`$fillable`	Liste des champs qu'un modèle accepte en remplissage de masse ; protège contre l'affectation de masse.
Gate	Système d'autorisation de Laravel, branché sur les permissions par `Gate::after`.
Idempotent	Qui peut être relancé sans effet indésirable (ex. `ReferentielInitialSeeder`).
lockForUpdate	Verrou SQL (`SELECT … FOR UPDATE`) qui empêche deux opérations simultanées de s'écraser.
Middleware	Filtre exécuté avant/après le contrôleur (`auth`, `permission:`…).
Migration	Fichier PHP qui crée ou modifie la structure de la base.
Modèle	Classe Eloquent représentant une table (`app/Models`).
Morph map / polymorphe	Relation où une table (`documents`, `decisions`) se rattache à plusieurs types d'objets via un type + un identifiant.
Notification interne	Message stocké dans la table `notifications`, visible via la cloche.
Permission	Droit élémentaire (`besoins.creer`) ; attribuée aux rôles.
Planificateur	`php artisan schedule:run`, exécuté chaque minute, qui lance les tâches de `routes/console.php`.
Policy	Classe qui décide de l'accès à un enregistrement précis (`app/Policies`).
RegleMetierException	Exception lancée par un service quand une règle de gestion interdit l'action ; transformée en message à l'écran.
Rôle	Ensemble de permissions attribué à des comptes.
Route	Association entre une URL, une méthode HTTP et une méthode de contrôleur (`routes/web.php`).
Scope	Méthode de modèle qui ajoute une condition réutilisable à une requête (`visiblesPar`).
Seeder	Classe qui insère des données initiales (`database/seeders`).
Service	Classe qui contient les règles métier d'un module (`app/Services`).

Terme	Définition dans le contexte du projet
Soft delete	Suppression logique : la ligne reste, `deleted_at` est rempli.
Tailwind CSS	Système de styles par classes utilitaires (`flex`, `gap-2`…), configuré dans `resources/css/app.css`.
Transaction	Groupe d'écritures SQL validées ensemble ou annulées ensemble (`DB::transaction`).
Vite	Outil qui compile le CSS et le JavaScript vers `public/build`.
XSS	Injection de script dans une page ; empêchée par l'échappement `{{ }}` et la CSP.

Annexes

Les annexes A, B et C sont **générées automatiquement** à partir du projet (commande `php artisan route:list --json`, tables `permissions` et `roles`, schéma `information_schema` de MySQL). Elles reflètent donc exactement la version 1.1.0. L'annexe D contient les scripts complets des épisodes vidéo.

Annexe A — Liste complète des routes

Liste produite par `php artisan route:list --json` le jour de la rédaction : **148 routes**. La colonne « Droits » reprend les middlewares `permission:` appliqués (les groupes cumulent leurs conditions : toutes doivent être satisfaites).

`/`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/`	`accueil`	`TableauBordController@accueil`	auth

`actions`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/actions`	`actions.index`	`ActionController@index`	`actions.voir`
POST	`/actions`	`actions.store`	`ActionController@store`	`actions.voir` + `actions.gerer`
GET	`/actions/creer`	`actions.create`	`ActionController@create`	`actions.voir` + `actions.gerer`
GET	`/actions/{action}`	`actions.show`	`ActionController@show`	`actions.voir`
PUT	`/actions/{action}`	`actions.update`	`ActionController@update`	`actions.voir` + `actions.gerer`
POST	`/actions/{action}/depenses`	`depenses.store`	`DepenseController@store`	`finances.voir` + `finances.gerer`
GET	`/actions/{action}/depenses/creer`	`depenses.create`	`DepenseController@create`	`finances.voir` + `finances.gerer`
GET	`/actions/{action}/modifier`	`actions.edit`	`ActionController@edit`	`actions.voir` + `actions.gerer`
POST	`/actions/{action}/sessions`	`sessions.store`	`SessionController@store`	`actions.voir` + `actions.gerer`
GET	`/actions/{action}/sessions/creer`	`sessions.create`	`SessionController@create`	`actions.voir` + `actions.gerer`
POST	`/actions/{action}/statut`	`actions.statut`	`ActionController@statut`	`actions.voir` + `actions.gerer`

`administration`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/administration/audit`	`admin.audit.index`	`Admin\AuditController@index`	`audit.voir`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/administration/audit/export`	`admin.audit.export`	`Admin\AuditController@export`	`audit.voir` + `audit.exporter`
GET	`/administration/circuits`	`admin.circuits.index`	`Admin\CircuitController@index`	`parametres.gerer`
POST	`/administration/circuits`	`admin.circuits.store`	`Admin\CircuitController@store`	`parametres.gerer`
POST	`/administration/circuits/{circuit}/activation`	`admin.circuits.activer`	`Admin\CircuitController@activer`	`parametres.gerer`
POST	`/administration/circuits/{circuit}/etapes`	`admin.circuits.etapes.store`	`Admin\CircuitController@ajouterEtape`	`parametres.gerer`
DELETE	`/administration/etapes/{etape}`	`admin.circuits.etapes.destroy`	`Admin\CircuitController@retirerEtape`	`parametres.gerer`
GET	`/administration/parametres`	`admin.parametres.index`	`Admin\ParametreController@index`	`parametres.gerer`
PUT	`/administration/parametres`	`admin.parametres.update`	`Admin\ParametreController@update`	`parametres.gerer`
GET	`/administration/referentiels/{type}`	`admin.referentiels.index`	`Admin\ReferentielController@index`	`parametres.gerer`
POST	`/administration/referentiels/{type}`	`admin.referentiels.store`	`Admin\ReferentielController@store`	`parametres.gerer`
PUT	`/administration/referentiels/{type}/{id}`	`admin.referentiels.update`	`Admin\ReferentielController@update`	`parametres.gerer`
GET	`/administration/roles`	`admin.roles.index`	`Admin\RoleController@index`	`roles.gerer`
POST	`/administration/roles`	`admin.roles.store`	`Admin\RoleController@store`	`roles.gerer`
DELETE	`/administration/roles/{role}`	`admin.roles.destroy`	`Admin\RoleController@destroy`	`roles.gerer`
PUT	`/administration/roles/{role}`	`admin.roles.update`	`Admin\RoleController@update`	`roles.gerer`
GET	`/administration/roles/{role}/modifier`	`admin.roles.edit`	`Admin\RoleController@edit`	`roles.gerer`
GET	`/administration/systeme`	`admin.systeme.index`	`Admin\SystemeController@index`	`systeme.superviser`
POST	`/administration/systeme/sauvegarde`	`admin.systeme.sauvegarde`	`Admin\SystemeController@sauvegarder`	`systeme.superviser`
GET	`/administration/utilisateurs`	`admin.utilisateurs.index`	`Admin\UtilisateurController@index`	`utilisateurs.gerer`
POST	`/administration/utilisateurs`	`admin.utilisateurs.store`	`Admin\UtilisateurController@store`	`utilisateurs.gerer`
GET	`/administration/utilisateurs/creer`	`admin.utilisateurs.create`	`Admin\UtilisateurController@create`	`utilisateurs.gerer`

Méthode	URI	Nom	Contrôleur@méthode	Droits
PUT	`/administration/utilisateurs/{utilisateur}`	`admin.utilisateurs.update`	`Admin\UtilisateurController@update`	`utilisateurs.gerer`
POST	`/administration/utilisateurs/{utilisateur}/activation`	`admin.utilisateurs.activation`	`Admin\UtilisateurController@activation`	`utilisateurs.gerer`
GET	`/administration/utilisateurs/{utilisateur}/modifier`	`admin.utilisateurs.edit`	`Admin\UtilisateurController@edit`	`utilisateurs.gerer`
POST	`/administration/utilisateurs/{utilisateur}/reinitialiser`	`admin.utilisateurs.reinitialiser`	`Admin\UtilisateurController@reinitialiserMotDePasse`	`utilisateurs.gerer`

`agents`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/agents`	`agents.index`	`AgentController@index`	`agents.voir_tous|agents.voir_service`
POST	`/agents`	`agents.store`	`AgentController@store`	`agents.voir_tous|agents.voir_service` + `agents.gerer`
GET	`/agents/creer`	`agents.create`	`AgentController@create`	`agents.voir_tous|agents.voir_service` + `agents.gerer`
GET	`/agents/export`	`agents.export`	`AgentController@export`	`agents.voir_tous|agents.voir_service` + `agents.exporter`
GET	`/agents/{agent}`	`agents.show`	`AgentController@show`	`agents.voir_tous|agents.voir_service`
PUT	`/agents/{agent}`	`agents.update`	`AgentController@update`	`agents.voir_tous|agents.voir_service` + `agents.gerer`
POST	`/agents/{agent}/activation`	`agents.activation`	`AgentController@activation`	`agents.voir_tous|agents.voir_service` + `agents.gerer`
GET	`/agents/{agent}/historique.pdf`	`agents.historique-pdf`	`AgentController@historiquePdf`	`agents.voir_tous|agents.voir_service`
GET	`/agents/{agent}/modifier`	`agents.edit`	`AgentController@edit`	`agents.voir_tous|agents.voir_service` + `agents.gerer`

`attestations`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/attestations`	`attestations.index`	`AttestationController@index`	`attestations.emettre`
POST	`/attestations/{attestation}/annulation`	`attestations.annuler`	`AttestationController@annuler`	`attestations.emettre`
GET	`/attestations/{attestation}/pdf`	`attestations.pdf`	`AttestationController@pdf`	`attestations.emettre`

besoins

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/besoins`	`besoins.index`	`BesoinController@index`	`besoins.voir_tous|besoins.voir_service`
POST	`/besoins`	`besoins.store`	`BesoinController@store`	`besoins.voir_tous|besoins.voir_service`
GET	`/besoins/creer`	`besoins.create`	`BesoinController@create`	`besoins.voir_tous|besoins.voir_service`
DELETE	`/besoins/{besoin}`	`besoins.destroy`	`BesoinController@destroy`	`besoins.voir_tous|besoins.voir_service`
GET	`/besoins/{besoin}`	`besoins.show`	`BesoinController@show`	`besoins.voir_tous|besoins.voir_service`
PUT	`/besoins/{besoin}`	`besoins.update`	`BesoinController@update`	`besoins.voir_tous|besoins.voir_service`
GET	`/besoins/{besoin}/modifier`	`besoins.edit`	`BesoinController@edit`	`besoins.voir_tous|besoins.voir_service`
POST	`/besoins/{besoin}/transition`	`besoins.transition`	`BesoinController@transition`	`besoins.voir_tous|besoins.voir_service`

connexion

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/connexion`	`login`	`Auth\ConnexionController@create`	guest
POST	`/connexion`	`login.store`	`Auth\ConnexionController@store`	guest, throttle:connexion

contacts

Méthode	URI	Nom	Contrôleur@méthode	Droits
DELETE	`/contacts/{contact}`	`organismes.contacts.destroy`	`OrganismeController@retirerContact`	`organismes.voir` + `organismes.gerer`

deconnexion

Méthode	URI	Nom	Contrôleur@méthode	Droits
POST	`/deconnexion`	`logout`	`Auth\ConnexionController@destroy`	auth

depenses

Méthode	URI	Nom	Contrôleur@méthode	Droits
PUT	`/depenses/{depense}`	`depenses.update`	`DepenseController@update`	`finances.voir` + `finances.gerer`
GET	`/depenses/{depense}/modifier`	`depenses.edit`	`DepenseController@edit`	`finances.voir` + `finances.gerer`

documents

Méthode	URI	Nom	Contrôleur@méthode	Droits
DELETE	`/documents/{document}`	`documents.destroy`	`DocumentController@destroy`	auth

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/documents/{document}/telecharger`	`documents.telecharger`	`DocumentController@telecharger`	auth
POST	`/documents/{type}/{id}`	`documents.store`	`DocumentController@store`	auth

`finances`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/finances`	`depenses.index`	`DepenseController@index`	`finances.voir`
GET	`/finances/export`	`depenses.export`	`DepenseController@export`	`finances.voir` + `finances.exporter`

`formateurs`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/formateurs`	`formateurs.index`	`FormateurController@index`	`organismes.voir`
POST	`/formateurs`	`formateurs.store`	`FormateurController@store`	`organismes.voir` + `organismes.gerer`
GET	`/formateurs/creer`	`formateurs.create`	`FormateurController@create`	`organismes.voir` + `organismes.gerer`
PUT	`/formateurs/{formateur}`	`formateurs.update`	`FormateurController@update`	`organismes.voir` + `organismes.gerer`
GET	`/formateurs/{formateur}/modifier`	`formateurs.edit`	`FormateurController@edit`	`organismes.voir` + `organismes.gerer`

`identite`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/identite/{type}`	`identite.fichier`	`IdentiteController@fichier`	—

`imports`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/imports`	`imports.index`	`ImportController@index`	`agents.importer`
POST	`/imports`	`imports.store`	`ImportController@store`	`agents.importer`
GET	`/imports/modele/{type}`	`imports.modele`	`ImportController@modele`	`agents.importer`
GET	`/imports/{import}`	`imports.show`	`ImportController@show`	`agents.importer`
POST	`/imports/{import}/abandonner`	`imports.abandonner`	`ImportController@abandonner`	`agents.importer`
POST	`/imports/{import}/confirmer`	`imports.confirmer`	`ImportController@confirmer`	`agents.importer`

inscriptions

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/inscriptions/{inscription}	inscriptions.show	InscriptionController@show	inscriptions.gerer
PUT	/inscriptions/{inscription}/acquis	evaluations.acquis	EvaluationController@acquis	evaluations.saisir
POST	/inscriptions/{inscription}/attestation	attestations.store	AttestationController@store	attestations.emettre
POST	/inscriptions/{inscription}/confirmer	inscriptions.confirmer	InscriptionController@confirmer	inscriptions.gerer
POST	/inscriptions/{inscription}/desister	inscriptions.desister	InscriptionController@desister	inscriptions.gerer
GET	/inscriptions/{inscription}/evaluations	evaluations.edit	EvaluationController@edit	evaluations.saisir
POST	/inscriptions/{inscription}/impact	evaluations.impact	EvaluationController@impact	evaluations.saisir
POST	/inscriptions/{inscription}/remplacer	inscriptions.remplacer	InscriptionController@remplacer	inscriptions.gerer
PUT	/inscriptions/{inscription}/satisfaction	evaluations.satisfaction	EvaluationController@satisfaction	evaluations.saisir

lignes

Méthode	URI	Nom	Contrôleur@méthode	Droits
DELETE	/lignes/{ligne}	plans.lignes.destroy	PlanController@retirerLigne	plans.gerer
PUT	/lignes/{ligne}	plans.lignes.update	PlanController@modifierLigne	plans.gerer
GET	/lignes/{ligne}/modifier	plans.lignes.edit	PlanController@editLigne	plans.gerer

mon-espace

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/mon-espace	espace.index	EspaceAgentController@index	espace.personnel
GET	/mon-espace/attestations/{attestation}	espace.attestation	EspaceAgentController@attestation	espace.personnel

mon-mot-de-passe

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/mon-mot-de-passe	mot-de-passe.edit	Auth\MotDePasseController@edit	auth
PUT	/mon-mot-de-passe	mot-de-passe.update	Auth\MotDePasseController@update	auth

mot-de-passe-oublie

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/mot-de-passe-oublie	password.request	Auth\MotDePasseController@demande	guest
POST	/mot-de-passe-oublie	password.email	Auth\MotDePasseController@envoyerLien	guest, throttle:reinitialisation

notifications

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/notifications	notifications.index	NotificationController@index	auth
POST	/notifications/tout-lire	notifications.tout-lire	NotificationController@toutLire	auth
POST	/notifications/{id}/lue	notifications.lue	NotificationController@lue	auth

organismes

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/organismes	organismes.index	OrganismeController@index	organismes.voir
POST	/organismes	organismes.store	OrganismeController@store	organismes.voir + organismes.gerer
GET	/organismes/creer	organismes.create	OrganismeController@create	organismes.voir + organismes.gerer
GET	/organismes/{organisme}	organismes.show	OrganismeController@show	organismes.voir
PUT	/organismes/{organisme}	organismes.update	OrganismeController@update	organismes.voir + organismes.gerer
POST	/organismes/{organisme}/appreciations	organismes.appreciations.store	OrganismeController@apprecier	organismes.voir + organismes.gerer|evaluations.saisir
POST	/organismes/{organisme}/contacts	organismes.contacts.store	OrganismeController@ajouterContact	organismes.voir + organismes.gerer
POST	/organismes/{organisme}/conventions	organismes.conventions.store	OrganismeController@ajouterConvention	organismes.voir + organismes.contrats
GET	/organismes/{organisme}/modifier	organismes.edit	OrganismeController@edit	organismes.voir + organismes.gerer

plans

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	/plans	plans.index	PlanController@index	plans.voir
POST	/plans	plans.store	PlanController@store	plans.gerer
GET	/plans/{plan}	plans.show	PlanController@show	plans.voir
GET	/plans/{plan}/export.pdf	plans.pdf	PlanController@exportPdf	plans.voir
GET	/plans/{plan}/export.xlsx	plans.excel	PlanController@exportExcel	plans.voir

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/plans/{plan}/imprimer`	`plans.imprimer`	`PlanController@imprimer`	`plans.voir`
POST	`/plans/{plan}/revision`	`plans.reviser`	`PlanController@reviser`	`plans.reviser`

`presences`

Méthode	URI	Nom	Contrôleur@méthode	Droits
POST	`/presences/{presence}/correction`	`presences.corriger`	`PresenceController@corriger`	`presences.valider`

`rapports`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/rapports`	`rapports.index`	`RapportController@index`	`rapports.voir`
GET	`/rapports/{code}`	`rapports.show`	`RapportController@show`	`rapports.voir`
GET	`/rapports/{code}/export/{format}`	`rapports.export`	`RapportController@export`	`rapports.voir`

`reinitialisation`

Méthode	URI	Nom	Contrôleur@méthode	Droits
POST	`/reinitialisation`	`password.update`	`Auth\MotDePasseController@reinitialiser`	guest, throttle:reinitialisation
GET	`/reinitialisation/{token}`	`password.reset`	`Auth\MotDePasseController@formulaireReinitialisation`	guest

`seances`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/seances/{seance}/presences`	`presences.edit`	`PresenceController@edit`	`presences.saisir|presences.valider`
PUT	`/seances/{seance}/presences`	`presences.update`	`PresenceController@update`	`presences.saisir|presences.valider`

`sessions`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/sessions`	`sessions.index`	`SessionController@index`	`actions.voir`
GET	`/sessions/{session}`	`sessions.show`	`SessionController@show`	`actions.voir`
PUT	`/sessions/{session}`	`sessions.update`	`SessionController@update`	`actions.voir` + `actions.gerer`
POST	`/sessions/{session}/annulation`	`sessions.annuler`	`SessionController@annuler`	`actions.voir` + `actions.gerer`
POST	`/sessions/{session}/attestations`	`attestations.session`	`AttestationController@emettreSession`	`attestations.emettre`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/sessions/{session}/bilan`	`sessions.bilan`	`EvaluationController@bilan`	`actions.voir`
PUT	`/sessions/{session}/bilan`	`sessions.bilan.update`	`EvaluationController@enregistrerBilan`	`actions.voir` + `actions.gerer`
POST	`/sessions/{session}/convocations`	`inscriptions.convoquer`	`InscriptionController@convoquer`	`inscriptions.gerer`
GET	`/sessions/{session}/convocations.pdf`	`sessions.convocations`	`SessionController@convocations`	`actions.voir` + `inscriptions.gerer`
GET	`/sessions/{session}/emargement.pdf`	`sessions.emargement`	`SessionController@emargement`	`actions.voir`
POST	`/sessions/{session}/inscriptions`	`inscriptions.store`	`InscriptionController@store`	`inscriptions.gerer`
GET	`/sessions/{session}/modifier`	`sessions.edit`	`SessionController@edit`	`actions.voir` + `actions.gerer`
POST	`/sessions/{session}/presences/validation`	`presences.valider`	`PresenceController@valider`	`presences.valider`
POST	`/sessions/{session}/report`	`sessions.reporter`	`SessionController@reporter`	`actions.voir` + `actions.gerer`
POST	`/sessions/{session}/statut`	`sessions.statut`	`SessionController@statut`	`actions.voir` + `actions.gerer`

`tableau-de-bord`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/tableau-de-bord`	`tableau-bord`	`TableauBordController@index`	`tableau_bord.voir`

`Système (Laravel)`

Méthode	URI	Nom	Contrôleur@méthode	Droits
GET	`/up`	—	`Closure`	—

`versions`

Méthode	URI	Nom	Contrôleur@méthode	Droits
PUT	`/versions/{version}`	`plans.versions.update`	`PlanController@modifierVersion`	`plans.gerer`
POST	`/versions/{version}/besoins`	`plans.integrer`	`PlanController@integrerBesoins`	`plans.gerer`
POST	`/versions/{version}/decision`	`plans.decider`	`PlanController@decider`	`plans.voir` + `plans.valider`
POST	`/versions/{version}/lignes`	`plans.lignes.store`	`PlanController@ajouterLigne`	`plans.gerer`
POST	`/versions/{version}/soumettre`	`plans.soumettre`	`PlanController@soumettre`	`plans.gerer`

Annexe B — Matrice des permissions par rôle

Source : tables `permissions`, `roles` et `permission_role` après `ReferentielInitialSeeder` (proposition initiale, modifiable dans *Administration › Rôles et permissions*). **39 permissions**, **8 rôles**.

Abréviations : **AT** = Administrateur technique ; **AF** = Administrateur fonctionnel ; **DRH** = Responsable DRH / formation ; **CS** = Chef de service ; **AV** = Autorité de validation ; **AG** = Agent ; **GF** = Gestionnaire financier ; **AU** = Auditeur / contrôleur. La colonne « S » signale une permission sensible.

Module	Code	Libellé	S	AT	AF	DRH	CS	AV	AG	GF	AU
Tableau de bord	`tableau_bord.voir`	Consulter le tableau de bord institutionnel		✔	✔	✔		✔			✔
Personnel	`agents.voir_tous`	Consulter tous les agents			✔	✔					✔
Personnel	`agents.voir_service`	Consulter les agents de son service					✔				
Personnel	`agents.gerer`	Créer, modifier et désactiver des agents	●		✔	✔					
Personnel	`agents.importer`	Importer des agents et des historiques	●		✔	✔					
Personnel	`agents.exporter`	Exporter le référentiel du personnel	●			✔					
Besoins	`besoins.voir_tous`	Consulter les besoins de tous les services				✔		✔			✔
Besoins	`besoins.voir_service`	Consulter les besoins de son service					✔				
Besoins	`besoins.creer`	Créer et soumettre des besoins pour son service				✔	✔				
Besoins	`besoins.creer_tous_services`	Créer des besoins pour n'importe quel service				✔					
Besoins	`besoins.instruire`	Instruire les besoins (examen, compléments, acceptation, rejet)	●			✔					
Plans	`plans.voir`	Consulter les plans annuels				✔		✔		✔	✔
Plans	`plans.gerer`	Élaborer les plans annuels et leurs versions	●			✔					
Plans	`plans.valider`	Prendre des décisions de validation sur les plans (selon les étapes)	●					✔			
Plans	`plans.reviser`	Ouvrir une révision d'un plan validé	●			✔					
Actions	`actions.voir`	Consulter les actions et sessions de formation				✔	✔	✔		✔	✔
Actions	`actions.gerer`	Gérer les actions, sessions, reports et annulations	●			✔					
Organismes	`organismes.voir`	Consulter les organismes et formateurs			✔	✔				✔	✔

Module	Code	Libellé	S	AT	AF	DRH	CS	AV	AG	GF	AU
Organismes	`organismes.gerer`	Gérer les organismes, contacts et formateurs			✔	✔					
Organismes	`organismes.contrats`	Consulter et gérer les conventions et données contractuelles	●			✔				✔	
Participants	`inscriptions.gerer`	Gérer les inscriptions, convocations et remplacements	●			✔					
Présences	`presences.saisir`	Saisir les présences				✔	✔				
Présences	`presences.valider`	Valider et corriger les relevés de présence	●			✔					
Présences	`presences.motifs`	Consulter les motifs d'absence (accès restreint)	●			✔					
Évaluations	`evaluations.saisir`	Saisir les évaluations (satisfaction, acquis, impact)				✔					
Évaluations	`evaluations.resultats`	Consulter les résultats individuels (accès restreint)	●			✔					
Évaluations	`attestations.emettre`	Émettre et annuler des attestations	●			✔					
Finances	`finances.voir`	Consulter les données financières	●			✔		✔		✔	✔
Finances	`finances.gerer`	Enregistrer et corriger les dépenses	●							✔	
Finances	`finances.exporter`	Exporter les données financières	●							✔	
Documents	`documents.confidentiels`	Accéder aux documents confidentiels	●			✔				✔	
Rapports	`rapports.voir`	Consulter et exporter les rapports statistiques				✔		✔			✔
Espace agent	`espace.personnel`	Consulter ses propres formations et attestations							✔		
Paramétrage	`parametres.gerer`	Gérer les référentiels et paramètres métier	●		✔						
Administration	`utilisateurs.gerer`	Gérer les comptes utilisateurs	●	✔							
Administration	`roles.gerer`	Gérer les rôles et permissions	●	✔							
Administration	`audit.voir`	Consulter le journal d'audit	●	✔							✔
Administration	`audit.exporter`	Exporter le journal d'audit	●	✔							✔
Administration	`systeme.superviser`	Superviser l'état technique (santé, sauvegardes)	●	✔							

Annexe C — Catalogue des tables MySQL

Produit à partir de `information_schema` de la base de développement : **53 tables**. Légende : **PK** clé primaire, **UNI** valeur unique, **MUL** colonne indexée (souvent une clé étrangère), ∅ = pas de valeur par défaut.

`actions_formation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`reference`	varchar(30)	non	UNI	∅
`annee`	smallint unsigned	non	MUL	∅
`intitule`	varchar(255)	non		∅
`description`	text	oui		∅
`domaine_id`	bigint unsigned	oui	MUL	∅
`objectifs_pedagogiques`	text	oui		∅
`type`	varchar(10)	non	MUL	∅
`modalite`	varchar(15)	non		presentiel
`organisme_id`	bigint unsigned	oui	MUL	∅
`public_cible`	varchar(255)	oui		∅
`cout_previsionnel`	decimal(15,2)	non		0.00
`statut`	varchar(20)	non	MUL	planifiee
`motif_statut`	varchar(500)	oui		∅
`plan_id`	bigint unsigned	oui	MUL	∅
`plan_action_code`	varchar(30)	oui		∅
`historique_importe`	tinyint(1)	non		0
`cree_par`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `domaine_id` → `domaines.id` (mise à NULL) ; `cree_par` → `users.id` (mise à NULL) ; `plan_id` → `plans_formation.id` (mise à NULL) ; `organisme_id` → `organismes.id` (mise à NULL).

Contraintes d'unicité : `reference`.

`agents`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`matricule`	varchar(30)	non	UNI	∅
`nom`	varchar(100)	non	MUL	∅
`prenom`	varchar(150)	non		∅
`direction_id`	bigint unsigned	oui	MUL	∅
`service_id`	bigint unsigned	oui	MUL	∅

Colonne	Type	NULL	Clé	Défaut
`fonction`	varchar(190)	oui		∅
`statut_professionnel_id`	bigint unsigned	oui	MUL	∅
`email_pro`	varchar(190)	oui		∅
`telephone_pro`	varchar(40)	oui		∅
`actif`	tinyint(1)	non		1
`date_desactivation`	date	oui		∅
`motif_desactivation`	varchar(255)	oui		∅
`source`	varchar(30)	non		saisie
`reference_externe`	varchar(100)	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `direction_id` → `directions.id` (suppression interdite si référencé) ; `statut_professionnel_id` → `nomenclatures.id` (mise à NULL) ; `service_id` → `services.id` (suppression interdite si référencé).

Contraintes d'unicité : `matricule`.

`annees_reference`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`annee`	smallint unsigned	non	UNI	∅
`statut`	varchar(20)	non		ouverte
`date_limite_besoins`	date	oui		∅
`date_cloture`	date	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `annee`.

`appreciations_prestataires`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`organisme_id`	bigint unsigned	non	MUL	∅
`action_id`	bigint unsigned	oui	MUL	∅
`note`	tinyint unsigned	non		∅
`commentaire`	text	oui		∅
`user_id`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `action_id` → `actions_formation.id` (mise à NULL) ; `organisme_id` → `organismes.id` (suppression en cascade) ; `user_id` → `users.id` (suppression interdite si référencé).

`attestations`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`reference`	varchar(40)	non	UNI	∅
`inscription_id`	bigint unsigned	non	UNI	∅
`code_verification`	varchar(64)	non	UNI	∅
`emise_le`	timestamp	non		∅
`emise_par`	bigint unsigned	non	MUL	∅
`annulee_le`	timestamp	oui		∅
`motif_annulation`	varchar(500)	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `emise_par` → `users.id` (suppression interdite si référencé) ; `inscription_id` → `inscriptions.id` (suppression interdite si référencé).

Contraintes d'unicité : `code_verification`, `inscription_id`, `reference`.

`audit_logs`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`user_id`	bigint unsigned	oui	MUL	∅
`action`	varchar(80)	non	MUL	∅
`objet_type`	varchar(80)	oui	MUL	∅
`objet_id`	bigint unsigned	oui		∅
`description`	varchar(500)	oui		∅
`contexte`	json	oui		∅
`ip`	varchar(45)	oui		∅
`user_agent`	varchar(255)	oui		∅
`created_at`	timestamp	non	MUL	CURRENT_TIMESTAMP

Clés étrangères : `user_id` → `users.id` (mise à NULL).

`besoin_agent`

Colonne	Type	NULL	Clé	Défaut
`besoin_id`	bigint unsigned	non	PRI	∅
`agent_id`	bigint unsigned	non	PRI	∅

Clés étrangères : `agent_id` → `agents.id` (suppression en cascade) ; `besoin_id` → `besoins_formation.id` (suppression en cascade).

`besoins_formation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`reference`	varchar(30)	non	UNI	∅
`annee`	smallint unsigned	non	MUL	∅
`service_id`	bigint unsigned	non	MUL	∅
`intitule`	varchar(255)	non		∅
`contexte`	text	oui		∅
`competences_visees`	text	oui		∅
`objectifs`	text	oui		∅
`publics_concernes`	text	oui		∅
`nb_participants`	int unsigned	oui		∅
`priorite_id`	bigint unsigned	oui	MUL	∅
`type_formation_id`	bigint unsigned	oui	MUL	∅
`domaine_id`	bigint unsigned	oui	MUL	∅
`duree_estimee_jours`	decimal(6,1)	oui		∅
`periode_souhaitee`	varchar(100)	oui		∅
`cout_previsionnel`	decimal(15,2)	oui		∅
`statut`	varchar(30)	non	MUL	brouillon
`auteur_id`	bigint unsigned	non	MUL	∅
`soumis_le`	timestamp	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅
`deleted_at`	timestamp	oui		∅

Clés étrangères : `auteur_id` → `users.id` (suppression interdite si référencé) ; `domaine_id` → `domaines.id` (mise à NULL) ; `priorite_id` → `nomenclatures.id` (mise à NULL) ; `service_id` → `services.id` (suppression interdite si référencé) ; `type_formation_id` → `nomenclatures.id` (mise à NULL).

Contraintes d'unicité : `reference`.

`cache`

Colonne	Type	NULL	Clé	Défaut
`key`	varchar(255)	non	PRI	∅
`value`	mediumtext	non		∅
`expiration`	bigint	non	MUL	∅

`cache_locks`

Colonne	Type	NULL	Clé	Défaut
`key`	varchar(255)	non	PRI	∅
`owner`	varchar(255)	non		∅
`expiration`	bigint	non	MUL	∅

`circuits_validation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`objet`	varchar(30)	non	MUL	∅
`nom`	varchar(190)	non		∅
`actif`	tinyint(1)	non		0
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

`contacts_organisme`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`organisme_id`	bigint unsigned	non	MUL	∅
`nom`	varchar(190)	non		∅
`fonction`	varchar(190)	oui		∅
`telephone`	varchar(40)	oui		∅
`email`	varchar(190)	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `organisme_id` → `organismes.id` (suppression en cascade).

`conventions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`organisme_id`	bigint unsigned	non	MUL	∅
`reference`	varchar(100)	non		∅
`objet`	varchar(255)	non		∅
`date_debut`	date	oui		∅
`date_fin`	date	oui		∅
`montant`	decimal(15,2)	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `organisme_id` → `organismes.id` (suppression en cascade).

Contraintes d'unicité : `organisme_id,reference`.

`decisions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`decidable_type`	varchar(255)	non	MUL	∅
`decidable_id`	bigint unsigned	non		∅
`action`	varchar(40)	non		∅
`statut_avant`	varchar(30)	oui		∅
`statut_apres`	varchar(30)	oui		∅
`etape_ordre`	smallint unsigned	oui		∅
`commentaire`	text	oui		∅
`user_id`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Clés étrangères : `user_id` → `users.id` (suppression interdite si référencé).

`depenses`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`action_id`	bigint unsigned	non	MUL	∅
`session_id`	bigint unsigned	oui	MUL	∅
`categorie_id`	bigint unsigned	non	MUL	∅
`libelle`	varchar(255)	non		∅
`organisme_id`	bigint unsigned	oui	MUL	∅
`beneficiaire`	varchar(190)	oui		∅
`montant`	decimal(15,2)	non		∅
`statut`	varchar(20)	non	MUL	engagee
`date_engagement`	date	non		∅
`numero_facture`	varchar(100)	oui		∅
`date_facture`	date	oui		∅
`date_paiement`	date	oui		∅
`reference_paiement`	varchar(100)	oui		∅
`rapprochee`	tinyint(1)	non		0
`observations`	text	oui		∅
`saisi_par`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `action_id` → `actions_formation.id` (suppression interdite si référencé) ; `categorie_id` → `nomenclatures.id` (suppression interdite si référencé) ; `organisme_id` → `organismes.id` (mise à NULL) ; `saisi_par` → `users.id` (suppression interdite si référencé) ; `session_id` → `sessions_formation.id` (mise à NULL).

`directions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`code`	varchar(30)	non	UNI	∅
`nom`	varchar(190)	non		∅
`actif`	tinyint(1)	non	MUL	1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `code`.

`documents`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`documentable_type`	varchar(255)	non	MUL	∅
`documentable_id`	bigint unsigned	non		∅
`categorie_id`	bigint unsigned	oui	MUL	∅
`titre`	varchar(255)	non		∅
`nom_original`	varchar(255)	non		∅
`chemin`	varchar(255)	non	UNI	∅
`mime`	varchar(120)	non		∅
`taille`	bigint unsigned	non		∅
`sha256`	char(64)	non		∅
`confidentiel`	tinyint(1)	non		0
`depose_par`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅
`deleted_at`	timestamp	oui		∅

Clés étrangères : `categorie_id` → `nomenclatures.id` (mise à NULL) ; `depose_par` → `users.id` (suppression interdite si référencé).

Contraintes d'unicité : `chemin`.

`domaines`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅

Colonne	Type	NULL	Clé	Défaut
`code`	varchar(30)	non	UNI	∅
`nom`	varchar(190)	non		∅
`description`	text	oui		∅
`actif`	tinyint(1)	non		1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `code`.

`envois_email`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`user_id`	bigint unsigned	oui	MUL	∅
`sujet`	varchar(255)	non		∅
`statut`	varchar(20)	non		∅
`erreur`	varchar(500)	oui		∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Clés étrangères : `user_id` → `users.id` (mise à NULL).

`etapes_validation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`circuit_id`	bigint unsigned	non	MUL	∅
`ordre`	smallint unsigned	non		∅
`libelle`	varchar(190)	non		∅
`role_id`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `circuit_id` → `circuits_validation.id` (suppression en cascade) ; `role_id` → `roles.id` (suppression interdite si référencé).

Contraintes d'unicité : `circuit_id,ordre`.

`evaluations_acquis`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`inscription_id`	bigint unsigned	non	UNI	∅
`methode`	varchar(50)	non		test

Colonne	Type	NULL	Clé	Défaut
`score`	decimal(6,2)	oui		∅
`score_max`	decimal(6,2)	oui		∅
`reussi`	tinyint(1)	non		∅
`appreciation`	text	oui		∅
`evaluateur_id`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `evaluateur_id` → `users.id` (mise à NULL) ; `inscription_id` → `inscriptions.id` (suppression en cascade).

Contraintes d'unicité : `inscription_id`.

`evaluations_impact`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`inscription_id`	bigint unsigned	non	MUL	∅
`date_evaluation`	date	non		∅
`niveau_mise_en_pratique`	tinyint unsigned	non		∅
`competences_observees`	text	oui		∅
`commentaire`	text	oui		∅
`evaluateur_id`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `evaluateur_id` → `users.id` (mise à NULL) ; `inscription_id` → `inscriptions.id` (suppression en cascade).

`evaluations_satisfaction`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`inscription_id`	bigint unsigned	non	UNI	∅
`note_globale`	tinyint unsigned	non		∅
`note_contenu`	tinyint unsigned	oui		∅
`note_formateur`	tinyint unsigned	oui		∅
`note_organisation`	tinyint unsigned	oui		∅
`note_utilite`	tinyint unsigned	oui		∅
`commentaire`	text	oui		∅
`suggestions`	text	oui		∅
`saisi_par`	bigint unsigned	oui	MUL	∅

Colonne	Type	NULL	Clé	Défaut
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `inscription_id` → `inscriptions.id` (suppression en cascade) ; `saisi_par` → `users.id` (mise à NULL).

Contraintes d'unicité : `inscription_id`.

`failed_jobs`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`uuid`	varchar(255)	non	UNI	∅
`connection`	varchar(255)	non	MUL	∅
`queue`	varchar(255)	non		∅
`payload`	longtext	non		∅
`exception`	longtext	non		∅
`failed_at`	timestamp	non		CURRENT_TIMESTAMP

Contraintes d'unicité : `uuid`.

`formateurs`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`type`	varchar(10)	non		∅
`agent_id`	bigint unsigned	oui	MUL	∅
`organisme_id`	bigint unsigned	oui	MUL	∅
`nom`	varchar(100)	non		∅
`prenom`	varchar(150)	oui		∅
`email`	varchar(190)	oui		∅
`telephone`	varchar(40)	oui		∅
`domaines_expertise`	text	oui		∅
`actif`	tinyint(1)	non		1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `organisme_id` → `organismes.id` (mise à NULL) ; `agent_id` → `agents.id` (mise à NULL).

`import_lignes`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅

Colonne	Type	NULL	Clé	Défaut
`import_id`	bigint unsigned	non	MUL	∅
`numero`	int unsigned	non		∅
`donnees`	json	non		∅
`statut`	varchar(20)	non		∅
`messages`	json	oui		∅

Clés étrangères : `import_id` → `imports.id` (suppression en cascade).

`imports`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`type`	varchar(30)	non		∅
`nom_fichier`	varchar(255)	non		∅
`statut`	varchar(20)	non		analyse
`total_lignes`	int unsigned	non		0
`lignes_valides`	int unsigned	non		0
`lignes_erreur`	int unsigned	non		0
`lignes_doublon`	int unsigned	non		0
`lignes_importees`	int unsigned	non		0
`message`	text	oui		∅
`user_id`	bigint unsigned	non	MUL	∅
`termine_le`	timestamp	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `user_id` → `users.id` (suppression interdite si référencé).

`inscription_evenements`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`inscription_id`	bigint unsigned	non	MUL	∅
`evenement`	varchar(40)	non		∅
`details`	varchar(500)	oui		∅
`user_id`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Clés étrangères : `inscription_id` → `inscriptions.id` (suppression en cascade) ; `user_id` → `users.id` (mise à NULL).

`inscriptions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`session_id`	bigint unsigned	non	MUL	∅
`agent_id`	bigint unsigned	non	MUL	∅
`statut`	varchar(20)	non	MUL	inscrit
`rang_attente`	int unsigned	oui		∅
`convoque_le`	timestamp	oui		∅
`confirme_le`	timestamp	oui		∅
`desiste_le`	timestamp	oui		∅
`motif_desistement`	varchar(500)	oui		∅
`remplace_par_id`	bigint unsigned	oui	MUL	∅
`taux_presence`	decimal(5,2)	oui		∅
`resultat`	varchar(20)	oui		∅
`resultat_calcule_le`	timestamp	oui		∅
`inscrit_par`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `agent_id` → `agents.id` (suppression interdite si référencé) ; `inscrit_par` → `users.id` (mise à NULL) ; `remplace_par_id` → `inscriptions.id` (mise à NULL) ; `session_id` → `sessions_formation.id` (suppression interdite si référencé).

Contraintes d'unicité : `session_id,agent_id`.

`job_batches`

Colonne	Type	NULL	Clé	Défaut
`id`	varchar(255)	non	PRI	∅
`name`	varchar(255)	non		∅
`total_jobs`	int	non		∅
`pending_jobs`	int	non		∅
`failed_jobs`	int	non		∅
`failed_job_ids`	longtext	non		∅
`options`	mediumtext	oui		∅
`cancelled_at`	int	oui		∅
`created_at`	int	non		∅
`finished_at`	int	oui		∅

`jobs`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`queue`	varchar(255)	non	MUL	∅
`payload`	longtext	non		∅
`attempts`	smallint unsigned	non		∅
`reserved_at`	int unsigned	oui		∅
`available_at`	int unsigned	non		∅
`created_at`	int unsigned	non		∅

`migrations`

Colonne	Type	NULL	Clé	Défaut
`id`	int unsigned auto	non	PRI	∅
`migration`	varchar(255)	non		∅
`batch`	int	non		∅

`nomenclatures`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`type`	varchar(50)	non	MUL	∅
`code`	varchar(50)	non		∅
`libelle`	varchar(190)	non		∅
`ordre`	smallint unsigned	non		0
`actif`	tinyint(1)	non		1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `type,code`.

`notifications`

Colonne	Type	NULL	Clé	Défaut
`id`	char(36)	non	PRI	∅
`type`	varchar(255)	non		∅
`notifiable_type`	varchar(255)	non	MUL	∅
`notifiable_id`	bigint unsigned	non		∅
`data`	text	non		∅
`read_at`	timestamp	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

`organismes`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`raison_sociale`	varchar(190)	non	MUL	∅
`sigle`	varchar(50)	oui		∅
`adresse`	varchar(255)	oui		∅
`telephone`	varchar(40)	oui		∅
`email`	varchar(190)	oui		∅
`site_web`	varchar(190)	oui		∅
`domaines_intervention`	text	oui		∅
`identifiant_fiscal`	varchar(60)	oui		∅
`actif`	tinyint(1)	non		1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

`parametres`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`cle`	varchar(100)	non	UNI	∅
`valeur`	text	oui		∅
`type`	varchar(20)	non		string
`groupe`	varchar(50)	non	MUL	∅
`libelle`	varchar(190)	non		∅
`aide`	varchar(500)	oui		∅
`a_valider_inps`	tinyint(1)	non		0
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `cle`.

`password_reset_tokens`

Colonne	Type	NULL	Clé	Défaut
`email`	varchar(255)	non	PRI	∅
`token`	varchar(255)	non		∅
`created_at`	timestamp	oui		∅

`permission_role`

Colonne	Type	NULL	Clé	Défaut
`role_id`	bigint unsigned	non	PRI	∅

Colonne	Type	NULL	Clé	Défaut
`permission_id`	bigint unsigned	non	PRI	∅

Clés étrangères : `role_id` → `roles.id` (suppression en cascade) ; `permission_id` → `permissions.id` (suppression en cascade).

`permissions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`code`	varchar(80)	non	UNI	∅
`module`	varchar(50)	non	MUL	∅
`libelle`	varchar(190)	non		∅
`sensible`	tinyint(1)	non		0
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `code`.

`plan_actions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`plan_version_id`	bigint unsigned	non	MUL	∅
`code`	varchar(30)	non		∅
`besoin_id`	bigint unsigned	oui	MUL	∅
`intitule`	varchar(255)	non		∅
`axe_id`	bigint unsigned	oui	MUL	∅
`domaine_id`	bigint unsigned	oui	MUL	∅
`direction_id`	bigint unsigned	oui	MUL	∅
`service_id`	bigint unsigned	oui	MUL	∅
`priorite_id`	bigint unsigned	oui	MUL	∅
`type`	varchar(10)	non		externe
`objectifs`	text	oui		∅
`public_cible`	varchar(255)	oui		∅
`nb_participants`	int unsigned	oui		∅
`cout_estime`	decimal(15,2)	non		0.00
`periode_debut`	date	oui		∅
`periode_fin`	date	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `axe_id` → `nomenclatures.id` (mise à NULL) ; `besoin_id` → `besoins_formation.id` (mise à NULL) ; `direction_id` → `directions.id` (mise à NULL) ; `domaine_id` → `domaines.id` (mise à NULL) ; `plan_version_id` → `plan_versions.id` (suppression en cascade) ; `priorite_id` → `nomenclatures.id` (mise à NULL) ; `service_id` → `services.id` (mise à NULL).

Contraintes d'unicité : `plan_version_id,besoin_id`, `plan_version_id,code`.

`plan_versions`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`plan_id`	bigint unsigned	non	MUL	∅
`numero`	smallint unsigned	non		∅
`statut`	varchar(20)	non	MUL	brouillon
`circuit_id`	bigint unsigned	oui	MUL	∅
`etape_courante`	smallint unsigned	oui		∅
`motif_revision`	text	oui		∅
`budget_global`	decimal(15,2)	oui		∅
`cree_par`	bigint unsigned	non	MUL	∅
`soumise_le`	timestamp	oui		∅
`validee_le`	timestamp	oui		∅
`verrouillee_le`	timestamp	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `plan_id` → `plans_formation.id` (suppression en cascade) ; `circuit_id` → `circuits_validation.id` (mise à NULL) ; `cree_par` → `users.id` (suppression interdite si référencé).

Contraintes d'unicité : `plan_id,numero`.

`plans_formation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`annee`	smallint unsigned	non	UNI	∅
`titre`	varchar(255)	non		∅
`objectifs`	text	oui		∅
`cree_par`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `cree_par` → `users.id` (suppression interdite si référencé).

Contraintes d'unicité : `annee`.

`presence_corrections`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`presence_id`	bigint unsigned	non	MUL	∅
`ancien_statut`	varchar(20)	non		∅
`nouveau_statut`	varchar(20)	non		∅
`motif`	varchar(500)	non		∅
`user_id`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Clés étrangères : `presence_id` → `presences.id` (suppression en cascade) ; `user_id` → `users.id` (suppression interdite si référencé).

`presences`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`seance_id`	bigint unsigned	non	MUL	∅
`inscription_id`	bigint unsigned	non	MUL	∅
`statut`	varchar(20)	non		∅
`motif_absence`	varchar(500)	oui		∅
`saisi_par`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `inscription_id` → `inscriptions.id` (suppression en cascade) ; `saisi_par` → `users.id` (mise à NULL) ; `seance_id` → `seances.id` (suppression en cascade).

Contraintes d'unicité : `seance_id,inscription_id`.

`rappels_envoyes`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`cle`	varchar(190)	non	UNI	∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Contraintes d'unicité : `cle`.

`role_user`

Colonne	Type	NULL	Clé	Défaut
`role_id`	bigint unsigned	non	PRI	∅
`user_id`	bigint unsigned	non	PRI	∅
`attribue_le`	timestamp	oui		∅

Colonne	Type	NULL	Clé	Défaut
`attribue_par`	bigint unsigned	oui	MUL	∅

Clés étrangères : `user_id` → `users.id` (suppression en cascade) ; `role_id` → `roles.id` (suppression interdite si référencé) ; `attribue_par` → `users.id` (mise à NULL).

`roles`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`code`	varchar(50)	non	UNI	∅
`nom`	varchar(150)	non		∅
`description`	text	oui		∅
`systeme`	tinyint(1)	non		0
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Contraintes d'unicité : `code`.

`seances`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`session_id`	bigint unsigned	non	MUL	∅
`date`	date	non		∅
`libelle`	varchar(100)	oui		∅
`heure_debut`	time	oui		∅
`heure_fin`	time	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `session_id` → `sessions_formation.id` (suppression en cascade).

`services`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`direction_id`	bigint unsigned	non	MUL	∅
`code`	varchar(30)	non	UNI	∅
`nom`	varchar(190)	non		∅
`actif`	tinyint(1)	non	MUL	1
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `direction_id` → `directions.id` (suppression interdite si référencé).

Contraintes d'unicité : `code`.

`session_changements`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`session_id`	bigint unsigned	non	MUL	∅
`type`	varchar(30)	non		∅
`ancienne_date_debut`	date	oui		∅
`ancienne_date_fin`	date	oui		∅
`nouvelle_date_debut`	date	oui		∅
`nouvelle_date_fin`	date	oui		∅
`ancien_statut`	varchar(20)	oui		∅
`nouveau_statut`	varchar(20)	oui		∅
`motif`	varchar(500)	oui		∅
`user_id`	bigint unsigned	non	MUL	∅
`created_at`	timestamp	non		CURRENT_TIMESTAMP

Clés étrangères : `session_id` → `sessions_formation.id` (suppression en cascade) ; `user_id` → `users.id` (suppression interdite si référencé).

`sessions`

Colonne	Type	NULL	Clé	Défaut
`id`	varchar(255)	non	PRI	∅
`user_id`	bigint unsigned	oui	MUL	∅
`ip_address`	varchar(45)	oui		∅
`user_agent`	text	oui		∅
`payload`	longtext	non		∅
`last_activity`	int	non	MUL	∅

`sessions_formation`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`action_id`	bigint unsigned	non	MUL	∅
`code`	varchar(40)	non	UNI	∅
`date_debut`	date	non	MUL	∅
`date_fin`	date	non		∅
`heure_debut`	time	oui		∅
`heure_fin`	time	oui		∅

Colonne	Type	NULL	Clé	Défaut
`duree_heures`	decimal(6,1)	oui		∅
`lieu`	varchar(255)	oui		∅
`places`	int unsigned	non		20
`liste_attente_active`	tinyint(1)	non		0
`formateur_id`	bigint unsigned	oui	MUL	∅
`statut`	varchar(20)	non	MUL	planifiee
`motif_statut`	varchar(500)	oui		∅
`bilan_pedagogique`	text	oui		∅
`presences_validees_le`	timestamp	oui		∅
`presences_validees_par`	bigint unsigned	oui	MUL	∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `action_id` → `actions_formation.id` (suppression interdite si référencé) ; `formateur_id` → `formateurs.id` (mise à NULL) ; `presences_validees_par` → `users.id` (mise à NULL).

Contraintes d'unicité : `code`.

`users`

Colonne	Type	NULL	Clé	Défaut
`id`	bigint unsigned auto	non	PRI	∅
`name`	varchar(190)	non		∅
`email`	varchar(190)	non	UNI	∅
`email_verified_at`	timestamp	oui		∅
`password`	varchar(255)	non		∅
`agent_id`	bigint unsigned	oui	UNI	∅
`service_id`	bigint unsigned	oui	MUL	∅
`direction_id`	bigint unsigned	oui	MUL	∅
`actif`	tinyint(1)	non	MUL	1
`doit_changer_mot_de_passe`	tinyint(1)	non		0
`mot_de_passe_modifie_le`	timestamp	oui		∅
`derniere_connexion_le`	timestamp	oui		∅
`derniere_connexion_ip`	varchar(45)	oui		∅
`remember_token`	varchar(100)	oui		∅
`created_at`	timestamp	oui		∅
`updated_at`	timestamp	oui		∅

Clés étrangères : `agent_id` → `agents.id` (mise à NULL) ; `direction_id` → `directions.id` (mise à NULL) ; `service_id` → `services.id` (mise à NULL).

Contraintes d'unicité : `agent_id`, `email`.

Annexe D — Scripts des épisodes vidéo

Pour chaque scène : ce qui est affiché, puis le texte exact de la narration. Ces scripts servent aussi de base pour réenregistrer les vidéos (section 7.3).

Épisode 1 — Présentation et architecture générale

Scène 01 · Titre · *Présentation et architecture générale*

> Bienvenue dans la formation technique de l'application INPS Formation. Cette série de vidéos s'appuie uniquement sur le code réel du projet, dans sa version 1.1.0. Dans ce premier épisode, nous voyons ce que fait l'application, quelles technologies elle utilise, comment ces technologies communiquent, et comment les dossiers du projet sont organisés.

Scène 02 · Diapositive · *Le programme de la formation*

> La formation comporte douze épisodes, du niveau débutant au niveau avancé. Les deux premiers posent les bases : l'architecture et l'installation sous Windows. Les épisodes trois à sept expliquent le code de A à Z. Les épisodes huit et neuf passent en revue chaque module métier. Enfin, les épisodes dix à douze vous apprennent à maintenir et à faire évoluer l'application. Un document PDF accompagne les vidéos : il reprend toutes les explications et tous les extraits de code, avec les numéros de lignes.

Scène 03 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/tableau-de-bord`

> Voici l'application, vue par un compte de démonstration de la Direction des ressources humaines. Elle gère tout le cycle de la formation du personnel : le recensement des besoins par les services, le plan annuel et sa validation, les actions et les sessions de formation, les inscriptions et les présences, les évaluations et les attestations, le suivi des dépenses, les documents, et les rapports statistiques. Chaque utilisateur ne voit que les menus correspondant à ses permissions.

Scène 04 · Diapositive · *Une application web monolithique Laravel*

> Techniquement, c'est une application web monolithique construite avec le framework Laravel. Monolithique veut dire que tout est dans un seul projet : les pages, la logique métier, l'accès à la base de données et les exports. Le serveur fabrique directement les pages HTML ; il n'y a pas d'application JavaScript séparée. Le projet suit le modèle M V C, modèle, vue, contrôleur, complété par une couche de services métier. Retenez cette règle d'or : les contrôleurs restent courts, et les règles de gestion vivent dans le dossier app, Services.

Scène 05 · Diapositive · *Les technologies et leur rôle*

> Voici les technologies et leur rôle. PHP est le langage dans lequel tout le côté serveur est écrit. Laravel, version treize, fournit les routes, la sécurité, l'accès à la base, la validation et les commandes Artisan. MySQL, version huit, stocke les données dans cinquante-trois tables. Blade est le moteur de gabarits qui produit les pages HTML. Tailwind CSS gère l'apparence, et un petit fichier JavaScript sans

bibliothèque ajoute les comportements interactifs. Vite compile le CSS et le JavaScript. Composer installe les bibliothèques PHP, et npm les outils JavaScript. Dompdf produit les PDF, et PhpSpreadsheet les fichiers Excel.

Scène 06 · Schéma / capture — architecture-couches.png · *Les couches de l'application*

Ce schéma montre comment les parties communiquent. Le navigateur envoie une requête au fichier public, index point php, le seul point d'entrée. La requête traverse des filtres appelés middlewares : session, protection contre la falsification, connexion, permissions. La route désigne le contrôleur. Le contrôleur valide les données, vérifie les autorisations, puis appelle un service métier. Le service utilise les modèles Eloquent, qui lisent et écrivent dans MySQL. Enfin, une vue Blade fabrique la page HTML renvoyée au navigateur, avec le CSS et le JavaScript compilés par Vite.

Scène 07 · Schéma / capture — trajet-requete.png · *Exemple : soumettre un besoin de formation*

Prenons un exemple concret : un chef de service clique sur Enregistrer et soumettre. Le formulaire envoie une requête POST vers l'adresse slash besoins. Les middlewares contrôlent la session et les droits. La route besoins point store appelle la méthode store du contrôleur BesoinController. La classe BesoinRequest valide les champs, puis le service BesoinService crée la demande et la soumet, dans une transaction. Enfin, le navigateur est redirigé vers la fiche de la demande. Nous suivrons ce trajet ligne par ligne dans l'épisode quatre.

Scène 08 · Commandes · *L'arborescence du projet*

Voici les dossiers à connaître. Le dossier app contient tout le code PHP : les contrôleurs, les middlewares, les classes de validation, les modèles, les services et les policies. Les commandes Artisan propres à l'INPS sont dans app, Console, Commands. Le dossier database contient les migrations, qui décrivent la structure de la base. Le dossier resources contient les vues Blade, le CSS et le JavaScript. Le fichier routes, web point php liste les cent quarante-huit adresses de l'application. Enfin, le fichier point env contient la configuration de votre installation : il est secret et ne doit jamais être partagé. Ne modifiez jamais les dossiers vendor et node modules : ils sont recréés automatiquement.

Scène 09 · Diapositive · *À retenir*

À retenir : l'application est un projet Laravel unique, avec MySQL et des vues Blade. Une requête traverse toujours les mêmes étapes : middlewares, route, contrôleur, service, modèle et base de données. Pour comprendre n'importe quel écran, commencez par le fichier des routes. Dans le prochain épisode, nous installons, démarrons et mettons à jour l'application sous Windows.

Épisode 2 — Installer, démarrer et mettre à jour sous Windows

Scène 01 · Titre · *Installer, démarrer et mettre à jour sous Windows*

Dans cet épisode, nous voyons comment installer, démarrer, arrêter et mettre à jour l'application sur un poste Windows, et à quoi sert chaque outil : Composer, npm, Vite, Artisan, le fichier point env et MySQL. Les commandes sont présentées à l'écran ; elles se tapent dans l'invite de commandes de

Windows.

Scène 02 · Diapositive · *Le rôle de chaque outil*

PHP exécute l'application. Composer lit les fichiers composer point json et composer point lock et télécharge les bibliothèques PHP dans le dossier vendor. npm fait de même pour les outils JavaScript, dans le dossier node modules ; il ne sert que si vous modifiez le CSS ou le JavaScript. Vite compile ces fichiers dans le dossier public, build. L'archive livrée contient déjà ce dossier compilé. Artisan est la console de Laravel : toutes les commandes commencent par php artisan. Enfin, MySQL est installé comme un service Windows nommé MySQL80, qui doit être démarré avant l'application.

Scène 03 · Commandes · *Le fichier .env (développement)*

Le fichier point env contient la configuration de votre installation. APP ENV vaut local en développement et production sur le serveur. APP KEY est la clé de chiffrement, générée une seule fois par la commande key generate ; ne la changez plus ensuite. APP DEBUG affiche le détail des erreurs : il doit valoir false en production. Les variables D B donnent l'accès à MySQL. MAIL MAILER égal log signifie qu'aucun e-mail n'est réellement envoyé. Après toute modification de ce fichier, lancez php artisan optimize clear. Et ne partagez jamais ce fichier : il contient des mots de passe.

Scène 04 · Commandes · *Première installation*

Pour une première installation, on se place dans le dossier du projet, on copie le fichier d'exemple point env point example en point env, et on y renseigne la base de données. composer install installe les bibliothèques. key generate crée la clé de l'application. migrate tiret tiret seed crée les cinquante-trois tables et charge le référentiel initial : les permissions, les rôles et les paramètres. La commande inps creer admin crée le premier administrateur, avec une saisie masquée du mot de passe : il n'existe aucun compte par défaut. Enfin, php artisan serve démarre le serveur de développement à l'adresse cent vingt-sept point zéro point zéro point un, port huit mille.

Scène 05 · Commandes · *Démarrer et arrêter au quotidien*

Au quotidien, si l'ordinateur a redémarré, il faut d'abord démarrer MySQL : ouvrez une invite de commandes en administrateur et tapez net start MySQL80. Puis, dans le dossier du projet, lancez php artisan serve, et ouvrez le navigateur. Pour arrêter l'application, appuyez sur contrôle C dans cette fenêtre. Pour que MySQL démarre tout seul avec Windows, ouvrez la console des services, ou tapez en administrateur la commande s c config MySQL80 start égal auto.

Scène 06 · Commandes · *Mettre à jour vers une nouvelle version*

Pour mettre à jour, décompressez la nouvelle version dans un nouveau dossier. Copiez-y l'ancien fichier point env et le dossier storage, app, private, qui contient les documents déposés. Puis lancez composer install, php artisan migrate, qui applique uniquement les nouvelles migrations sans toucher à vos données, et php artisan optimize clear, qui vide les caches. Enfin, rechargez la page avec contrôle F5.

Scène 07 · Commandes · *Les erreurs les plus fréquentes*

> Voici les erreurs les plus fréquentes, dont certaines que vous avez déjà rencontrées. Le message SQLSTATE deux mille deux, l'ordinateur cible l'a expressément refusée, signifie que MySQL est arrêté. L'erreur mille quarante-cinq signifie que l'identifiant ou le mot de passe MySQL du fichier point env est faux. Could not open input file artisan : vous n'êtes pas dans le dossier du projet. Vite manifest not found : les ressources n'ont pas été compilées. Si la commande mysql n'est pas reconnue, utilisez son chemin complet. Et pour toute erreur cinq cents, ouvrez le dernier fichier du dossier storage, logs : l'erreur y est décrite en détail.

Scène 08 · Commandes · *Le contrôle de santé : php artisan inps:sante*

> Enfin, une commande propre au projet vérifie l'état général : php artisan inps sante. Elle contrôle la connexion à la base, les migrations, la protection du journal d'audit, la présence d'un administrateur, la clé de l'application et le stockage. Les lignes attention affichées ici sont normales sur un poste de développement : sessions non chiffrées, pas de serveur de messagerie, pas de sauvegarde récente ni chiffrée, pas de planificateur. En production, toutes les lignes doivent être OK. Dans le prochain épisode, nous suivons le trajet d'une requête dans le code.

Épisode 3 — Le trajet d'une requête : point d'entrée, routes et middlewares

Scène 01 · Titre · *Le trajet d'une requête : routes et middlewares*

> Dans cet épisode, nous suivons une requête depuis son arrivée sur le serveur jusqu'au contrôleur : le point d'entrée, l'assemblage de l'application, les routes, et les middlewares qui filtrent chaque requête.

Scène 02 · Code — `public/index.php` · *Le point d'entrée unique : public/index.php*

> Toutes les requêtes arrivent dans un seul fichier : public, index point php. Il vérifie d'abord le mode maintenance. Puis il charge l'autoloader de Composer, qui permet à PHP de trouver automatiquement chaque classe à partir de son nom. Enfin, il démarre Laravel à partir du fichier bootstrap, app point php, et lui confie la requête. Seul le dossier public doit être accessible depuis le web : le code et le fichier point env restent hors de portée.

Scène 03 · Code — `bootstrap/app.php#L12-L36` · *L'assemblage : bootstrap/app.php*

> Le fichier bootstrap, app point php configure trois choses. Les fichiers de routes, d'abord. Les middlewares, ensuite : le projet ajoute à toutes les pages ses deux filtres maison, EntetesSecurite et VerifierCompteActif, et crée le raccourci permission, que nous retrouverons dans les routes. La gestion des erreurs, enfin : quand un service lance une RegleMetierException, Laravel renvoie l'utilisateur sur la page précédente, conserve sa saisie et affiche le message d'erreur. C'est pour cela que les contrôleurs n'ont presque jamais de bloc try catch.

Scène 04 · Code — `routes/web.php#L52-L67` · *Les routes : routes/web.php*

> Une route associe une méthode HTTP et une adresse à une méthode de contrôleur. Ici, le groupe middleware auth exige d'être connecté pour tout ce qu'il contient. La route du tableau de bord exige en plus la permission tableau bord point voir. Et chaque route porte un nom : par exemple tableau tiret

> bord. Dans le code, on écrit route, tableau tiret bord, plutôt que l'adresse : si l'adresse change un jour, aucun lien n'est à corriger.

Scène 05 · Code — `routes/web.php#L90-L100` · *Le module des besoins*

> Voici les routes des besoins. Le groupe exige la permission besoins point voir tous, ou besoins point voir service : la barre verticale signifie au moins une des deux. Entre accolades, besoin est un paramètre. Dans le contrôleur, Laravel le transforme automatiquement en objet BesoinFormation, ou renvoie une erreur quatre cent quatre s'il n'existe pas. Attention : la route ne vérifie pas si ce besoin appartient au service de l'utilisateur. Ce contrôle fin est fait dans le contrôleur, par une policy.

Scène 06 · Commandes · *Retrouver un écran : php artisan route:list*

> Pour retrouver le code d'un écran, regardez son adresse dans le navigateur, puis tapez php artisan route list, tiret tiret path égal besoins. La commande affiche chaque route, son nom, et le contrôleur avec sa méthode. C'est le premier réflexe à avoir quand on découvre un écran.

Scène 07 · Code — `app/Http/Middleware/ExigerPermission.php@handle` · *Middleware 1 : ExigerPermission*

> Un middleware est un filtre placé avant le contrôleur. Voici ExigerPermission, appelé par le raccourci permission deux points. Il découpe la liste des permissions demandées, et si l'utilisateur n'en possède aucune, la fonction abort arrête tout avec une erreur quatre cent trois, accès refusé. Sinon, il passe la requête au filtre suivant avec next. C'est le principe du refus par défaut : sans permission explicite, pas d'accès.

Scène 08 · Démonstration — compte `demo.chef_service@exemple.invalid` ; pages : `/besoins`, `/plans`

> Démonstration : ce chef de service voit la liste des besoins de son service. S'il tape directement l'adresse des plans annuels, pour laquelle il n'a pas la permission, le middleware répond par une erreur quatre cent trois, avec le message : vous ne disposez pas des droits nécessaires pour accéder à cette page. C'est la page d'erreur standard de Laravel. Le menu ne lui proposait d'ailleurs pas cette entrée : mais c'est bien le serveur qui garantit la sécurité, pas le menu.

Scène 09 · Code — `app/Http/Middleware/VerifierCompteActif.php@handle` · *Middleware 2 : VerifierCompteActif*

> Le deuxième middleware maison s'applique à toutes les pages. Si un administrateur désactive un compte pendant que la personne travaille, sa requête suivante la déconnecte, avec une trace dans le journal d'audit. Et si le compte doit changer de mot de passe, par exemple après un mot de passe temporaire, toutes les pages renvoient vers l'écran de changement du mot de passe.

Scène 10 · Code — `app/Http/Middleware/EntetesSecurite.php@handle` · *Middleware 3 : EntetesSecurite*

> Le troisième agit après le contrôleur : il ajoute des en-têtes de sécurité à la réponse. Le plus important est la Content Security Policy : le navigateur n'exécute que les scripts servis par l'application elle-même. Un script injecté par un pirate serait bloqué. C'est pourquoi aucune vue ne contient de script :

> tout le JavaScript est dans un seul fichier compilé. Les autres en-têtes interdisent l'affichage dans un cadre, et empêchent la mise en cache des pages contenant des données personnelles.

Scène 11 · Diapositive · *À retenir*

> À retenir : toute requête passe par index point php, puis bootstrap, puis les middlewares, avant d'atteindre la route et le contrôleur. Le middleware permission contrôle l'accès à chaque module, avec un refus par défaut. Les erreurs métier sont traitées en un seul endroit. Dans le prochain épisode, nous entrons dans le contrôleur des besoins, sa validation, sa policy et son service.

Épisode 4 — Contrôleur, validation, policy et service : déclarer un besoin

Scène 01 · Titre · *Contrôleur, validation, policy et service*

> Dans cet épisode, nous suivons un cas réel de bout en bout : un chef de service déclare un besoin de formation et le soumet à la Direction des ressources humaines, qui l'accepte. Nous verrons le contrôleur, la validation, la policy, le service métier, et les requêtes SQL réellement exécutées.

Scène 02 · Démonstration — compte `demo.chef_service@exemple.invalid` ; pages : `/besoins`

> Le chef de service ouvre la liste des besoins et clique sur nouvelle demande. Il saisit l'intitulé, choisit une priorité, indique six participants, puis clique sur enregistrer et soumettre, et confirme. La demande est créée avec une référence automatique, au statut soumise. Voyons ce qui s'est passé dans le code.

Scène 03 · Code — `app/Http/Controllers/BesoinController.php#L42-L57` · *Le contrôleur : BesoinController*

> Le formulaire a été affiché par la méthode create, qui vérifie d'abord avec authorize que l'utilisateur peut créer un besoin. L'envoi arrive dans la méthode store. Remarquez son paramètre : un BesoinRequest, et non une simple requête. La validation est donc déjà faite quand la méthode commence. Le contrôleur ne contient aucune règle : il appelle le service pour créer la demande, puis, comme le bouton soumettre valait un, il demande la transition soumission. Enfin, il redirige vers la fiche avec un message de succès.

Scène 04 · Code — `app/Http/Requests/BesoinRequest.php#L9-L35` · *La validation : BesoinRequest*

> Voici la classe de validation. La méthode authorize exige la permission besoins point créer. La méthode rules liste les règles de chaque champ : l'année doit exister dans la table des années de référence, le service doit exister et être actif, l'intitulé est obligatoire et limité à deux cent cinquante-cinq caractères, la priorité doit être une nomenclature de type priorité. Si une règle échoue, Laravel renvoie au formulaire avec les messages en français, sans exécuter le contrôleur.

Scène 05 · Code — `app/Policies/BesoinFormationPolicy.php#L17-L33` · *La policy : BesoinFormationPolicy*

> La policy décide, enregistrement par enregistrement, qui peut faire quoi. Pour voir une demande : la DRH, qui a besoins voir tous, voit tout ; un chef de service ne voit que les demandes de son propre service. Pour la modifier, la demande doit être encore modifiable, c'est-à-dire en brouillon ou à

compléter, et appartenir au service de l'utilisateur. Laravel associe automatiquement cette policy au modèle BesoinFormation, grâce à son nom.

Scène 06 · Code — `app/Services/BesoinService.php#L22-L31` · *Le service : la machine à états*

Les règles vivent dans le service BesoinService. Cette constante, TRANSITIONS, est une machine à états : pour chaque action, elle indique depuis quels statuts elle est permise, vers quel statut elle mène, et si un commentaire est obligatoire. Par exemple, l'acceptation est possible depuis soumise ou en examen, et mène à acceptée ; le rejet exige un commentaire.

Scène 07 · Code — `app/Services/BesoinService.php#L71-L113` · *Le service : la transition*

La méthode transition applique ces règles. Elle fait d'abord les contrôles simples : action connue, étape activée dans les paramètres, année encore ouverte, commentaire présent si nécessaire. Puis elle ouvre une transaction, et relit la demande avec lockForUpdate, c'est-à-dire en la verrouillant : si deux personnes décident en même temps, la seconde attendra et verra le nouveau statut. Si le statut de départ n'est pas autorisé, elle lance une RegleMetierException avec un message clair. Sinon, elle change le statut, enregistre la décision dans l'historique, écrit le journal d'audit, et notifie les bonnes personnes.

Scène 08 · Commandes · *Les requêtes SQL réellement exécutées*

Voici les requêtes SQL qui ont réellement été exécutées pendant ce scénario, enregistrées avec l'outil DB listen de Laravel. La première charge les permissions de l'utilisateur. Les deux suivantes viennent des règles de validation. Puis le générateur de référence lit la dernière référence de l'année sous verrou, et le service insère la demande au statut brouillon, sa décision de création et la ligne d'audit. La transition relit la demande verrouillée, la passe au statut soumise, et enregistre la décision et l'audit. Enfin, le service cherche les utilisateurs qui ont la permission d'instruire, pour les notifier. Le détail des vingt-six requêtes est dans le document PDF.

Scène 09 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/besoins`

Côté Direction des ressources humaines, la nouvelle demande apparaît en tête de liste, et une notification a été reçue. La fiche affiche le bloc d'instruction, visible uniquement grâce à la policy instruire. La responsable clique sur accepter, saisit un commentaire facultatif et valide. Le même service, avec la même méthode transition, fait passer la demande au statut acceptée, et l'historique des décisions, en bas de la fiche, garde la trace de qui a décidé, quand, et pourquoi.

Scène 10 · Diapositive · *À retenir*

À retenir : un contrôleur suit toujours le même schéma : autoriser, valider, appeler le service, rediriger. La validation est faite dans une classe FormRequest, les droits sur un enregistrement précis dans une policy, et toutes les règles de gestion dans le service, à l'intérieur d'une transaction. Dans le prochain épisode, nous découvrons la base de données : migrations, tables, relations et modèles.

Épisode 5 — La base de données : migrations, tables, relations et modèles

Scène 01 · Titre · *La base de données*

> Dans cet épisode, nous découvrons la base de données MySQL : comment sa structure est décrite par des migrations, comment les tables sont reliées entre elles, comment les modèles Eloquent les représentent en PHP, et comment manipuler les données sans danger.

Scène 02 · Commandes · *Les migrations : la structure sous forme de code*

> La base n'est jamais créée à la main. Sa structure est décrite par neuf migrations, des fichiers PHP du dossier database, migrations, exécutés dans l'ordre de leur nom par la commande php artisan migrate. Laravel note chaque migration appliquée dans la table migrations, si bien qu'une migration ne s'exécute qu'une seule fois. La commande migrate status montre leur état. La dernière, numéro sept cents, a été ajoutée par la version un point un.

Scène 03 · Code — `database/migrations/2026_01_01_000100_create_organisation_tables.php#L14-L30` · *Une migration : les directions et les services*

> Voici la création des tables directions et services. La méthode id crée la clé primaire auto-incrémentée. string code, unique, crée une colonne texte dont la valeur ne peut pas être en double. foreignId direction id, constrained, crée une clé étrangère vers la table directions : impossible d'enregistrer un service rattaché à une direction inexistante. Et restrict on delete interdit de supprimer une direction qui a encore des services. La base protège elle-même la cohérence des données.

Scène 04 · Code — `database/migrations/2026_01_01_000300_create_besoins_plans_tables.php#L14-L38` · *La table des besoins*

> La table des besoins de formation suit les mêmes principes. La référence est unique. Le service est une clé étrangère. Le statut est une simple chaîne de caractères, dont les valeurs possibles sont définies en PHP par une énumération. L'auteur est une clé étrangère vers les utilisateurs, et soft deletes ajoute une colonne deleted at : supprimer un brouillon le masque sans effacer la ligne.

Scène 05 · Schéma / capture — schema-er.png · *53 tables regroupées par domaine*

> Ce schéma, généré directement à partir de la base, montre les cinquante-trois tables regroupées par domaine, et leurs clés étrangères. On y trouve l'organisation et les référentiels, le personnel et les droits, les besoins et les plans, les actions et les sessions, la participation, et enfin les finances, les documents et la traçabilité. Huit tables techniques appartiennent à Laravel lui-même.

Scène 06 · Schéma / capture — chaine-metier.png · *La chaîne métier : du besoin à l'attestation*

> Voici la chaîne principale. Un besoin accepté devient une ligne de plan, identifiée par un code comme P A deux mille vingt-six tiret zéro zéro un. Une action de formation est rattachée à cette ligne par le même code : c'est ce qui permet de comparer le prévu et le réalisé. Une action a des sessions ; une

session a des séances, une par jour ouvré, et des inscriptions. Chaque inscription a une présence par séance, et au plus une attestation. Une contrainte d'unicité empêche d'inscrire deux fois le même agent à la même session.

Scène 07 · Code — `app/Models/BesoinFormation.php#L14-L35` · *Le modèle Eloquent : BesoinFormation*

En PHP, chaque table est représentée par un modèle Eloquent, dans le dossier app, Models. La propriété table donne le nom de la table. La liste fillable énumère les seuls champs qu'on peut remplir en masse depuis un formulaire : le statut, l'auteur et la référence n'y figurent pas. C'est la protection contre l'affectation de masse. La méthode casts convertit les colonnes : le statut devient automatiquement un objet de l'énumération StatutBesoin, et les montants gardent deux décimales.

Scène 08 · Code — `app/Models/BesoinFormation.php#L37-L70` · *Les relations entre modèles*

Les relations traduisent les clés étrangères. belongs to signifie appartient à : le besoin appartient à un service, et à un auteur. belongs to many relie plusieurs agents à un besoin, à travers la table de liaison besoin agent. morph many est une relation polymorphe : la même table decisions sert à la fois aux besoins et aux versions de plan. Dans une vue, on écrit simplement besoin, flèche, service, flèche, nom, et Laravel exécute la requête nécessaire.

Scène 09 · Code — `app/Models/BesoinFormation.php#L82-L93` · *Le cloisonnement : un scope*

Ce scope, visibles par, ajoute automatiquement la bonne condition à une requête selon l'utilisateur. La DRH voit tout ; un chef de service ne voit que son service ; et quelqu'un sans permission ne voit rien, grâce à la condition un égal zéro. La liste des besoins l'utilise : la policy protège la fiche, le scope protège la liste.

Scène 10 · Diapositive · *Les seeders : les données initiales*

Les seeders insèrent les données initiales. ReferentielInitialSeeder crée les trente-neuf permissions, les huit rôles proposés, les listes de valeurs et les paramètres. Il ne crée aucun compte : il est utilisable en production. Il peut être relancé sans risque après une mise à jour : il ajoute les nouvelles permissions sans modifier les rôles déjà ajustés par l'administrateur. DemoSeeder crée des données entièrement fictives, et refuse de s'exécuter en production.

Scène 11 · Diapositive · *Manipuler les données sans danger*

Pour manipuler les données, une règle simple : toute modification passe par l'interface ou par les services, jamais par une requête SQL directe. Le service applique les règles, écrit l'historique et le journal d'audit ; une requête SQL directe contournerait tout cela. Pour consulter, vous pouvez utiliser MySQL Workbench ou le client mysql en lecture seule. Le document PDF donne des exemples de requêtes. Notez que l'application ne supprime presque rien : les agents sont désactivés, les sessions et les attestations sont annulées avec un motif. Et attention à la commande migrate fresh : elle efface toutes les tables. Elle est réservée aux bases de développement.

Épisode 6 — Sécurité : authentification, rôles, permissions et journal d'audit

Scène 01 · Titre · *Sécurité : authentification, rôles et permissions*

> Dans cet épisode, nous étudions la sécurité de l'application : comment un utilisateur se connecte, comment ses droits sont calculés à partir de ses rôles, comment l'application empêche quelqu'un de s'attribuer des droits, et comment toutes les opérations sensibles sont tracées.

Scène 02 · Démonstration — compte `sans connexion` ; pages : `/connexion`

> Voici la page de connexion. Avec un mauvais mot de passe, le message reste volontairement vague : identifiants incorrects ou compte désactivé. Il ne révèle pas si le compte existe. Avec le bon mot de passe, l'utilisateur arrive sur son écran d'accueil, adapté à ses droits.

Scène 03 · Code — `app/Http/Controllers/Auth/ConnexionController.php@store` · *La connexion : ConnexionController*

> Le traitement est dans la méthode store du contrôleur de connexion. Auth attempt cherche le compte par son e-mail, uniquement s'il est actif, et compare le mot de passe avec son empreinte : la base ne contient jamais de mot de passe en clair. Un échec est journalisé avec une empreinte de l'adresse, pas l'adresse elle-même. En cas de succès, la session est régénérée pour empêcher la fixation de session, et la connexion est tracée dans le journal d'audit. La route est aussi limitée à cinq tentatives par minute pour un même e-mail et une même adresse I P.

Scène 04 · Code — `app/Models/User.php#L58-L98` · *Les permissions viennent des rôles*

> Un utilisateur a des rôles, et chaque rôle a des permissions. Aucun droit n'est donné directement à une personne, et aucun rôle n'a tous les droits. La méthode permission codes charge toutes les permissions de l'utilisateur en une seule requête, et les garde en mémoire pendant la requête. La méthode a permission répond vrai ou faux. Notez qu'un compte désactivé n'a aucune permission, même si ses rôles sont toujours en base.

Scène 05 · Code — `app/Support/CataloguePermissions.php#L68-L92` · *Le catalogue : 39 permissions, 8 rôles proposés*

> La liste des permissions et la proposition de rôles sont écrites dans le fichier app, Support, CataloguePermissions. Par exemple, le chef de service peut voir les agents et les besoins de son service, créer des besoins, voir les actions et saisir les présences. Ces rôles sont une proposition : l'administrateur peut les ajuster dans l'écran rôles et permissions, sans toucher au code.

Scène 06 · Code — `app/Providers/AppServiceProvider.php#L57-L80` · *Brancher les permissions sur Laravel*

> Dans le fournisseur de services AppServiceProvider, plusieurs réglages globaux de sécurité. La politique de mot de passe : douze caractères minimum, majuscules, minuscules, chiffres et symboles. La limitation des tentatives de connexion. Gate after branche les permissions de la base sur le système

> d'autorisation de Laravel : c'est ce qui fait fonctionner la directive arobase can dans les vues. Et la directive arobase permission, créée ici, affiche un bouton seulement si l'utilisateur détient la permission.

Scène 07 · Démonstration — compte `demo.admin_technique@exemple.invalid` ; pages : `/administration/roles`, `/administration/audit`

> Connectons-nous en administrateur technique. L'écran rôles et permissions liste les rôles ; ouvrons celui de chef de service. Les permissions sont regroupées par module, et les permissions sensibles sont signalées. Puis le journal d'audit : chaque connexion, chaque décision, chaque modification sensible y est enregistrée, avec l'utilisateur, la date, l'adresse I P et l'objet concerné.

Scène 08 · Code — `app/Http/Controllers/Admin/RoleController.php@update` · *Empêcher l'auto-attribution de droits*

> Plusieurs protections empêchent l'élévation de privilèges. Ici, dans RoleController, un administrateur ne peut pas modifier un rôle qu'il détient lui-même : il ne peut donc pas s'ajouter des droits. Chaque modification de permissions est journalisée, avec les permissions ajoutées et retirées. De même, seul un détenteur de la permission gérer les rôles peut attribuer un rôle d'administration, et le dernier administrateur technique actif ne peut pas perdre son rôle.

Scène 09 · Code — `app/Models/AuditLog.php@booted` · *Un journal d'audit infalsifiable*

> Le journal d'audit est en ajout seul. Le modèle AuditLog refuse toute modification ou suppression au niveau de PHP. Et la migration crée deux déclencheurs MySQL qui refusent tout UPDATE ou DELETE sur la table, même en SQL direct. La commande inps sante vérifie que ces déclencheurs sont bien présents.

Scène 10 · Diapositive · *Synthèse des protections*

> En synthèse : la connexion est protégée par le hachage des mots de passe, la limitation des tentatives et la régénération de session. L'accès est contrôlé à trois niveaux : le middleware permission, les policies et les scopes, toujours avec un refus par défaut. Les formulaires sont protégés contre la falsification par le jeton C S R F, l'affichage contre l'injection de script par l'échappement et la politique de sécurité du contenu. Les documents sont vérifiés et stockés hors de la racine web, et le journal d'audit ne peut pas être effacé. Les tests automatiques vérifient ces protections à chaque exécution. Dans le prochain épisode : l'interface, avec Blade, Tailwind et JavaScript.

Épisode 7 — L'interface : Blade, composants, Tailwind CSS, JavaScript et Vite

Scène 01 · Titre · *L'interface*

> Dans cet épisode, nous voyons comment les pages sont fabriquées : les vues Blade, les composants réutilisables, le système de styles Tailwind, le petit fichier JavaScript, et la compilation avec Vite.

Scène 02 · Diapositive · *La syntaxe Blade essentielle*

> Les vues sont des fichiers point blade point php, rangés par module dans resources, views. Les doubles accolades affichent une valeur en neutralisant les caractères spéciaux : c'est la protection contre l'injection de script. Arobase if et arobase foreach écrivent les conditions et les boucles. Arobase csrf est obligatoire dans tout formulaire : il insère le jeton anti-falsification. Arobase can et arobase permission n'affichent un élément que si l'utilisateur en a le droit. Et une balise qui commence par x tiret utilise un composant réutilisable.

Scène 03 · Code — `resources/views/components/layout.blade.php#L11-L43` · *La coquille de toutes les pages : le menu*

> Toutes les pages connectées sont enveloppées par le composant layout. Son menu latéral est un simple tableau PHP : pour chaque entrée, le libellé, la route, le motif qui détermine l'entrée active, la permission requise, et l'icône. Une entrée n'apparaît que si l'utilisateur détient la permission indiquée en quatrième colonne. Ajouter une entrée de menu revient donc à ajouter une ligne ici.

Scène 04 · Code — `resources/views/components/champ.blade.php#L1-L16` · *Un composant : le champ de formulaire*

> Le composant champ produit un champ de formulaire complet. Arobase props déclare ses paramètres et leurs valeurs par défaut. La fonction old réaffiche la saisie de l'utilisateur après une erreur de validation. Et les attributs aria relient le champ à son message d'erreur, pour les lecteurs d'écran. Grâce à ce composant, tous les formulaires de l'application ont le même comportement.

Scène 05 · Code — `resources/views/besoins/form.blade.php#L1-L18` · *Un formulaire complet en quelques lignes*

> Voici le début du formulaire de déclaration d'un besoin. Le formulaire pointe vers la route besoins point store, ou besoins point update en modification. Il contient arobase csrf, et arobase method PUT en modification. Chaque champ tient en une ligne grâce au composant champ, regroupés dans des cartes.

Scène 06 · Code — `resources/css/app.css#L9-L46` · *Les styles : les jetons de la charte*

> Tout le style est dans resources, css, app point css. Il importe Tailwind et la police Public Sans, embarquée localement. Le bloc theme définit les couleurs de la charte : chaque couleur déclarée ici crée automatiquement des classes, comme text encre trois ou bg inps. La couleur principale lit un paramètre modifiable dans l'écran des paramètres, sans recompiler. Les composants maison, comme les boutons et les cartes, sont définis plus bas dans le même fichier.

Scène 07 · Code — `resources/js/app.js#L11-L34` · *Le JavaScript : des comportements par attributs*

> Le JavaScript tient dans un seul fichier, sans bibliothèque. Il écoute les événements au niveau du document et réagit aux attributs data présents dans le HTML. Par exemple, data confirm demande une confirmation avant d'envoyer un formulaire, et data dialog open ouvre une boîte de dialogue. Aucun script n'est écrit dans les vues : c'est indispensable pour respecter la politique de sécurité du contenu.

Scène 08 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/sessions/1`, `/besoins/creer`

> Démonstration des comportements JavaScript : le bouton réduire le menu bascule le menu en icônes, et le choix est mémorisé. Dans le formulaire de besoin, le champ de filtre de la liste des agents masque, pendant la frappe, les agents qui ne correspondent pas. Tout cela est déclenché par de simples attributs data dans les vues.

Scène 09 · Commandes · *La compilation : Vite*

> Vite compile le CSS et le JavaScript. npm ci installe les outils, puis npm run build produit les fichiers optimisés dans public, build, avec un manifeste que la directive arobase vite lit pour insérer les bonnes balises. Pendant le développement, npm run dev recompile instantanément à chaque enregistrement. Retenez : après toute modification du CSS, du JavaScript, ou après avoir utilisé une nouvelle classe Tailwind dans une vue, il faut relancer npm run build.

Scène 10 · Diapositive · *À retenir*

> À retenir : les vues sont dans resources, views, enveloppées par le composant layout et construites avec les composants maison. Masquer un bouton dans une vue est un confort, pas une sécurité : la sécurité est toujours assurée par le serveur. Les styles sont dans app point css, les comportements dans app point js, et toute modification se compile avec npm run build. Dans le prochain épisode, nous parcourons les modules métier, du plan annuel aux présences.

Épisode 8 — Les modules métier (1) : du plan annuel aux présences

Scène 01 · Titre · *Les modules métier (1)*

> Dans cet épisode, nous parcourons quatre modules qui s'enchaînent : le plan annuel et son circuit de validation, les actions et sessions de formation, les inscriptions, et les présences. Pour chacun : les fichiers, le fonctionnement technique et les règles métier.

Scène 02 · Diapositive · *Plans annuels : les fichiers*

> Le module des plans annuels repose sur le contrôleur PlanController et le service PlanService. Il y a un seul plan par année, et chaque plan a des versions numérotées. Une version n'est modifiable qu'en brouillon, ou après un rejet. Une fois validée, elle est verrouillée ; pour la changer, on ouvre une révision, qui crée une nouvelle version en copiant les lignes avec les mêmes codes.

Scène 03 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/plans/1`

> Voici le plan deux mille vingt-six de la démonstration. On voit sa version, son statut, les lignes regroupées par axe stratégique, et l'historique des décisions du circuit de validation. Plus bas, la comparaison entre le prévu et le réalisé : pour chaque ligne, les dépenses payées et les agents formés par les actions rattachées.

Scène 04 · Code — `app/Services/PlanService.php@peutDecider` · *Qui peut valider une étape ?*

> La soumission exige un circuit de validation actif, configuré dans l'administration : des étapes ordonnées, chacune confiée à un rôle. Pour décider à une étape, cette méthode exige trois conditions : la permission plans valider, le rôle de l'étape en cours, et, si le paramètre de séparation des tâches est actif, ne pas être l'auteur de la version. On ne valide pas son propre travail.

Scène 05 · Code — `app/Services/ActionService.php@genererSeances` · *Actions et sessions : les séances automatiques*

> Une action de formation reçoit une référence automatique, et peut être rattachée à une ligne du plan. Chaque session reçoit un code, et le service génère automatiquement une séance par jour ouvré entre la date de début et la date de fin. Ces séances servent ensuite à la saisie des présences. Rien n'est jamais effacé : chaque report, modification de dates ou annulation est enregistré dans la table session changements, avec l'auteur et le motif.

Scène 06 · Code — `app/Services/ActionService.php#L177-L201` · *Les changements de statut d'une session*

> Les changements de statut sont encadrés par ce tableau des passages autorisés : planifiée, confirmée, en cours, terminée. Et une règle importante : une session ne peut pas être terminée tant que le relevé de présence n'a pas été validé. Pour les autres cas, on utilise les fonctions de report ou d'annulation, qui exigent un motif.

Scène 07 · Code — `app/Services/InscriptionService.php#L26-L57` · *Inscriptions : capacité et liste d'attente*

> Les inscriptions sont gérées par InscriptionService. La session est relue sous verrou, pour éviter de dépasser la capacité si deux personnes inscrivent en même temps. Pour chaque agent, le service refuse les agents inactifs, les doubles inscriptions, et les chevauchements de dates avec une autre session. Si la session est pleine, l'agent va en liste d'attente, si l'option est activée. Un désistement libère la place et fait monter automatiquement le premier de la liste d'attente.

Scène 08 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/sessions/1`, `/seances/1/presences`

> Voici une session terminée de la démonstration : ses séances, ses participants avec leur statut, leur taux de présence et leur résultat. La page de saisie des présences propose, pour chaque participant, présent, retard, partiel ou absent. Une fois le relevé validé, une présence ne se modifie plus que par une correction motivée et tracée.

Scène 09 · Code — `app/Services/PresenceService.php@calculerResultat` · *Le calcul de l'achèvement*

> À la validation du relevé, le service calcule le résultat de chaque participant. Le taux de présence est la somme des poids de chaque séance, divisée par le nombre de séances : présent vaut un, partiel vaut zéro virgule cinq, absent vaut zéro. Par exemple, présent, partiel, présent sur trois séances donne quatre-vingt-trois pour cent. La formation est achevée si le taux atteint le seuil, quatre-vingts pour cent par défaut, et, si le paramètre l'exige, si l'évaluation des acquis est réussie. Le seuil et les poids sont des paramètres modifiables par l'INPS.

Scène 10 · Diapositive · *À retenir*

> À retenir : le plan est versionné et validé par un circuit configurable ; la session génère ses séances et garde l'historique de ses changements ; les inscriptions sont contrôlées sous verrou ; et l'achèvement est calculé individuellement, selon des paramètres. Dans le prochain épisode : les évaluations, les attestations, les finances, les documents, les rapports et les notifications.

Épisode 9 — Les modules métier (2) : évaluations, attestations, finances, documents, rapports, notifications

Scène 01 · Titre · *Les modules métier (2)*

> Suite du parcours des modules : les évaluations et les attestations, le suivi financier, les documents, les rapports, et les notifications avec leurs rappels automatiques.

Scène 02 · Code — `app/Services/AttestationService.php@conditionsNonRemplies` · *Attestations : des conditions vérifiées*

> Trois évaluations existent : la satisfaction, notée de un à cinq, les acquis, dont les résultats individuels sont en accès restreint, et l'impact différé, quelques mois après la formation. Une attestation n'est délivrée que si toutes les conditions sont remplies : inscription active, relevé de présence validé, formation achevée, et, si les paramètres l'exigent, évaluations renseignées. La méthode renvoie la liste exacte des conditions manquantes, affichée à l'utilisateur. Chaque attestation reçoit une référence et un code de vérification aléatoire imprimé sur le PDF.

Scène 03 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/attestations`, `/finances`

> Voici la liste des attestations émises, avec leur référence. Puis l'écran des finances : le total prévu, engagé et payé, et le détail des dépenses par action, avec leur statut.

Scène 04 · Code — `app/Services/DepenseService.php@corriger` · *Finances : chaque correction est tracée*

> Les dépenses sont rattachées à une action, avec trois statuts : engagée, payée ou annulée. Toute correction exige un motif, et le journal d'audit conserve les valeurs avant et après de chaque champ financier modifié. Les contrôles sont dans le service : montant strictement positif, date de paiement obligatoire pour une dépense payée, session appartenant bien à l'action. Ce module suit les coûts de formation ; il ne remplace pas la comptabilité de l'INPS.

Scène 05 · Code — `app/Services/DocumentService.php#L39-L50` · *Documents : un dépôt sécurisé*

> Les pièces jointes passent par DocumentService. Le service vérifie l'extension, mais aussi le contenu réel du fichier : un fichier renommé en point pdf est refusé. Le fichier est rangé sur un disque privé, hors de la racine web, sous un nom aléatoire, avec son empreinte SHA deux cent cinquante-six. Le téléchargement passe toujours par un contrôleur qui vérifie les droits, et chaque téléchargement d'un document confidentiel est journalisé.

Scène 06 · Code — `app/Services/RapportService.php@RAPPORTS` · *Rapports : 13 rapports déclarés*

> Les treize rapports statistiques sont déclarés dans cette constante. Chaque rapport renvoie la même structure : un titre, des colonnes, des lignes, et une note expliquant la règle de calcul. L'écran, l'impression, le PDF et l'Excel utilisent ce même résultat : les chiffres sont identiques sur tous les supports. Les rapports contenant des montants sont réservés aux détenteurs de la permission finances voir.

Scène 07 · Démonstration — compte `demo.responsable_drh@exemple.invalid` ; pages : `/rapports`, `/rapports/agents_formes?annee=2026`

> Voici la liste des rapports, puis le rapport des agents formés par direction et par service, pour deux mille vingt-six. Les boutons permettent l'export en Excel ou en PDF, et l'impression. Chaque export est enregistré dans le journal d'audit.

Scène 08 · Code — `app/Console/Commands/Echeances.php@une` · *Notifications : chaque rappel une seule fois*

> Les services envoient des notifications internes lors des événements : soumission, décision, convocation, validation en attente. Chaque jour, la commande inps echeances génère les rappels : sessions à venir, convocations à préparer, évaluations à faire, échéances annuelles. Pour qu'un rappel ne parte qu'une fois, sa clé est insérée dans la table rappels envoyés, qui a une contrainte d'unicité : si l'insertion échoue, c'est que le rappel a déjà été envoyé. Attention : ces rappels ne fonctionnent que si le planificateur de Laravel tourne chaque minute.

Scène 09 · Diapositive · *À retenir*

> À retenir : les attestations, les dépenses et les documents sont tous protégés par des règles écrites dans leur service, et tracés dans le journal d'audit. Les rapports utilisent un seul calcul pour tous les supports, et les rappels dépendent du planificateur. Dans le prochain épisode, nous passons à la pratique : modifier une page existante et ajouter un champ dans un formulaire et dans la base.

Épisode 10 — Maintenance (1) : modifier une page et ajouter un champ

Scène 01 · Titre · *Maintenance (1)*

> Place à la pratique. Dans cet épisode, nous réalisons deux modifications réelles sur une copie du projet : ajouter une colonne dans une liste, puis ajouter un nouveau champ dans un formulaire et dans la base de données. Ces exercices ont été réalisés et testés pour de vrai ; ils ne font pas partie de l'application livrée.

Scène 02 · Diapositive · *Avant toute modification*

> Trois règles avant de commencer. Travaillez toujours sur une copie du projet, avec une base de développement, jamais sur les données réelles. Gardez un moyen de revenir en arrière : une sauvegarde, ou mieux, Git, qui enregistre l'historique de chaque modification. Et après chaque modification, lancez les tests, puis vérifiez à l'écran.

Scène 03 · Diapositive · *Exercice 1 : afficher le domaine dans la liste des besoins*

> Premier exercice : afficher le domaine de compétence dans la liste des besoins. On part de l'adresse, slash besoins. La commande route list indique la méthode index du contrôleur des besoins, qui renvoie la vue besoins point index, c'est-à-dire le fichier index point blade point php du dossier besoins.

Scène 04 · Code — `tuto1.diff` · *Les deux modifications*

> Voici les deux modifications, au format diff : les lignes moins sont retirées, les lignes plus sont ajoutées. Dans le contrôleur, on ajoute la relation domaine au chargement anticipé : ainsi, Laravel charge tous les domaines en une seule requête, au lieu d'une requête par ligne. Dans la vue, on ajoute une colonne dans l'en-tête, et une cellule dans chaque ligne. L'opérateur point d'interrogation flèche évite une erreur si le besoin n'a pas de domaine, et un tiret s'affiche à la place.

Scène 05 · Démonstration — compte `demo.responsable_drh@exemple.invalid`, copie de travail ; pages : `/besoins`

> Résultat : la colonne domaine apparaît dans la liste. Aucune compilation n'était nécessaire, car nous n'avons utilisé aucune nouvelle classe de style.

Scène 06 · Diapositive · *Exercice 2 : la date d'embauche des agents*

> Deuxième exercice : enregistrer la date d'embauche des agents, de façon facultative. Un nouveau champ touche toujours les mêmes couches, dans le même ordre : la migration, le modèle, la validation, le formulaire, la fiche, et le test.

Scène 07 · Commandes · *Étape 1 : créer la migration*

> On crée une nouvelle migration avec make migration. Ne modifiez jamais une migration déjà appliquée : créez-en toujours une nouvelle. Laravel crée le fichier, dont le nom commence par la date et l'heure. On le complète, puis on l'applique avec php artisan migrate.

Scène 08 · Code — `database/migrations/2026_10_21_080428_ajouter_date_embauche_aux_agents.php#L10-L25` · *La migration*

> Dans la méthode up, on ajoute une colonne de type date, nullable, c'est indispensable car les agents existants n'ont pas de date d'embauche, placée après la colonne fonction. La méthode down supprime la colonne : elle permet d'annuler la migration.

Scène 09 · Code — `tuto2.diff#L1-L41` · *Le modèle et la validation*

> Dans la classe de validation AgentRequest, on ajoute la règle : facultatif, une date, et pas dans le futur, avec un message en français. Dans le modèle Agent, on ajoute le champ à la liste fillable, sinon il serait ignoré à l'enregistrement, et un cast date, pour manipuler une vraie date. Remarquez que le contrôleur n'a pas besoin d'être modifié : il enregistre déjà toutes les données validées.

Scène 10 · Code — `tuto2.diff#L42-L65` · *Le formulaire et la fiche*

> Dans le formulaire, une seule ligne : le composant champ, de type date. Il gère tout seul la valeur, l'ancienne saisie et le message d'erreur. Dans la fiche de l'agent, une ligne affiche la date au format français grâce à la fonction date fr.

Scène 11 · Démonstration — compte `demo.responsable_drh@exemple.invalid`, copie de travail ; pages : `/agents/creer`

> Essayons : on crée un agent fictif avec sa date d'embauche, et on enregistre. La fiche affiche bien la nouvelle information.

Scène 12 · Code — `tests/Feature/AgentDateEmbaucheTest.php#L8-L30` · *Le test automatique*

> Enfin, le test. Le premier vérifie que la date est enregistrée et affichée sur la fiche. Le second vérifie qu'une date dans le futur est refusée par la validation, et que rien n'est enregistré. La commande php artisan test, tiret tiret filter, suivie du nom du test, les exécute : deux tests réussis.

Scène 13 · Diapositive · *À retenir*

> À retenir : pour modifier une page, on remonte de l'adresse à la vue grâce à la commande route list. Pour un nouveau champ, on suit toujours le même ordre : migration, modèle, validation, vues, test. Dans le prochain épisode, nous créons une fonctionnalité complète, avec sa table, son écran, son menu, sa permission et ses tests.

Épisode 11 — Maintenance (2) : créer une fonctionnalité complète

Scène 01 · Titre · *Maintenance (2) : une fonctionnalité complète*

> Dans cet épisode, nous créons une fonctionnalité complète : un catalogue des salles de formation, géré par l'administrateur fonctionnel, avec sa propre permission. C'est l'exercice le plus complet ; il a été réalisé et testé sur une copie du projet.

Scène 02 · Commandes · *Étape 1 : générer les fichiers avec Artisan*

> On commence par générer les fichiers de base avec Artisan. make model Salle, tiret m, crée le modèle et sa migration. make controller crée le contrôleur dans le dossier Admin. make test crée le fichier de test. Il reste à les compléter.

Scène 03 · Code — `database/migrations/2026_10_21_080512_create_salles_table.php#L12-L29` · *Étape 2 : la table*

> La migration crée la table salles : un nom unique, un lieu, une capacité, les équipements, un indicateur actif, et les dates de création et de modification.

Scène 04 · Code — `app/Models/Salle.php` · *Étape 3 : le modèle*

> Le modèle Salle indique sa table, la liste des champs remplissables, et les conversions : actif en booléen, capacité en entier.

Scène 05 · Code — `tuto3.diff` · *Étapes 4, 6 et 8 : permission, routes, menu*

> Trois fichiers existants sont modifiés. Dans le catalogue des permissions, on ajoute la permission salles point gérer, et on l'ajoute à la proposition du rôle administrateur fonctionnel. Dans le menu du composant layout, une ligne ajoute l'entrée salles de formation, visible seulement avec cette permission. Dans les routes, un groupe protégé par la nouvelle permission déclare trois routes : la liste, l'ajout et la modification. Puis la commande php artisan db seed enregistre la nouvelle permission en base.

Scène 06 · Démonstration — compte `demo.admin_technique@exemple.invalid`, copie de travail ; pages : `/administration/roles/2/modifier`

> Attention : sur une installation existante, le seeder crée la permission mais ne modifie pas les rôles déjà en place. L'administrateur technique l'attribue donc au rôle administrateur fonctionnel, dans l'écran rôles et permissions. La case est ici déjà cochée, puisque l'exercice a été réalisé.

Scène 07 · Code — `app/Http/Controllers/Admin/SalleController.php#L13-L49` · *Étape 5 : le contrôleur*

> Le contrôleur suit le schéma habituel : valider, enregistrer, journaliser dans l'audit, rediriger avec un message. La règle unique, ignore, interdit deux salles de même nom, sauf la salle en cours de modification. La logique est ici assez simple pour rester dans le contrôleur. Dès qu'apparaîtrait une vraie règle métier, par exemple interdire deux sessions en même temps dans la même salle, il faudrait créer un service SalleService.

Scène 08 · Code — `resources/views/admin/salles/index.blade.php#L1-L17` · *Étape 7 : la vue*

> La vue est construite uniquement avec les composants existants : layout, entête, carte, champ et badge. Elle hérite donc automatiquement du style et de l'accessibilité de toute l'application, sans une ligne de CSS.

Scène 09 · Démonstration — compte `demo.admin_fonctionnel@exemple.invalid`, copie de travail ; pages : `/administration/salles`

> L'administrateur fonctionnel voit la nouvelle entrée dans son menu. Il ajoute une salle, avec son lieu et sa capacité, et l'enregistre. Le message de confirmation s'affiche et la salle apparaît dans la liste.

Scène 10 · Code — `tests/Feature/SalleTest.php#L8-L30` · *Étape 9 : les tests*

> Trois tests : le cas normal, où l'administrateur fonctionnel crée une salle et où l'audit est écrit ; la règle d'unicité du nom ; et le refus d'accès pour un chef de service. Testez toujours qu'un profil non autorisé est bien refusé.

Scène 11 · Commandes · *Résultat des tests*

> Les trois tests réussissent. Et surtout, la suite complète passe toujours : cent sept tests, les cent deux d'origine plus les cinq des exercices. Nous sommes donc sûrs de n'avoir rien cassé ailleurs.

Scène 12 · Diapositive · *Mémo : où ajouter quoi*

> Pour récapituler : une table ou une colonne, c'est une migration. Un modèle, un contrôleur ou une classe de validation se génèrent avec les commandes make. Les règles métier vont dans un service. Une route se déclare dans web point php avec sa permission, et une entrée de menu dans le composant layout. Une nouvelle permission se déclare dans le catalogue, s'enregistre avec db seed, et s'attribue aux rôles dans l'écran prévu. Dans le dernier épisode : les tests, le débogage, les sauvegardes et la mise en production.

Épisode 12 — Tests, débogage, sauvegardes et mise en production

Scène 01 · Titre · *Tests, débogage, sauvegardes et production*

> Dernier épisode. Nous voyons comment fonctionnent les tests automatisés, comment corriger un bug avec méthode, à partir d'un cas réel de ce projet, comment sauvegarder et restaurer les données, et comment préparer une mise en production sécurisée.

Scène 02 · Commandes · *La suite de tests : php artisan test*

> L'application est livrée avec cent deux tests automatisés, répartis dans douze fichiers. La commande php artisan test les exécute en une trentaine de secondes. Ils vérifient l'authentification, les autorisations, chaque module métier, la sécurité, et même la sauvegarde et la restauration. Lancez-les après chaque modification : s'ils passent tous, vous n'avez rien cassé.

Scène 03 · Code — `tests/TestCase.php#L29-L50` · *La base des tests : TestCase*

> Les tests utilisent une base séparée, nommée inps formation test, et la recréent vide à chaque test. Par sécurité, la classe TestCase refuse de s'exécuter si le nom de la base ne se termine pas par tiret bas test : impossible d'effacer la base réelle par erreur. Sa méthode utilisateur crée en une ligne un compte actif avec le rôle voulu : c'est l'outil de base de tous les tests.

Scène 04 · Code — `tests/Feature/BesoinPlanTest.php#L46-L60` · *Un test réel : le cycle de vie d'une demande*

> Voici un test réel. Un chef de service envoie une demande en ajoutant volontairement des champs interdits : un statut accepté et un autre auteur. Le test vérifie que la demande est bien créée en brouillon, avec le bon auteur : les champs interdits ont été ignorés. Puis il vérifie la soumission, la notification de la DRH, et qu'une demande soumise n'est plus modifiable par son auteur, avec une réponse quatre cent trois.

Scène 05 · Diapositive · *Corriger un bug : un cas réel*

> Passons au débogage, avec un vrai bug de la version un point zéro. Une fois un logo déposé, il s'affichait pour les administrateurs, mais apparaissait comme une image cassée pour tous les autres utilisateurs. Pour reproduire, on se connecte avec un compte sans la permission de gérer les paramètres. En ouvrant l'image seule dans un nouvel onglet, on obtient une réponse quatre cent trois : accès refusé. Aucune erreur n'apparaît dans les journaux : c'est une erreur de conception, pas de syntaxe.

Scène 06 · Code — `bug_v100.txt` · *La cause (version 1.0.0)*

> Voici la cause. La vue layout, affichée à tous les utilisateurs, demandait le logo à la route admin parametres logo. Or cette route était déclarée dans le groupe protégé par la permission parametres gerer. Le middleware des permissions refusait donc l'image à tous ceux qui n'avaient pas cette permission.

Scène 07 · Code — `app/Http/Controllers/IdentiteController.php@fichier` · *La correction (version 1.1.0)*

> La correction : une route publique, en lecture seule, limitée aux deux types logo et photo, servie par un petit contrôleur dédié. Il ne sert que des types d'image autorisés, avec une politique de sécurité qui interdit tout script. L'ancienne route a été supprimée. Un test vérifie désormais qu'un visiteur non connecté obtient bien le logo. La méthode générale : reproduire, lire le message ou les journaux, localiser, comprendre, écrire un test qui échoue, corriger, puis relancer toute la suite.

Scène 08 · Commandes · *Sauvegarder et restaurer*

> Pour les sauvegardes, la commande inps sauvegarde produit une archive contenant la base, les documents déposés, et un manifeste avec l'empreinte de chaque fichier. Si la variable BACKUP PASSWORD est définie, l'archive est chiffrée en A E S deux cent cinquante-six. La commande inps restaurer vérifie chaque empreinte avant de toucher à la base, et demande de retaper le nom de la base pour confirmer. Restaurez d'abord dans une base de vérification. Sous Windows, indiquez dans le fichier point env le chemin complet des outils mysqldump et mysql. Une sauvegarde n'a de valeur que si sa restauration a été testée, et une copie doit être conservée hors du serveur.

Scène 09 · Diapositive · *Checklist de mise en production*

> Enfin, la checklist de mise en production, détaillée dans le document installation production. Environnement production, mode débogage désactivé, HTTPS forcé. Clé de l'application et mot de passe des sauvegardes conservés en lieu sûr, hors du serveur. Seul le dossier public est exposé au web, MySQL n'est pas accessible depuis le réseau, et le compte applicatif a des droits limités. Le planificateur tourne, une sauvegarde a été faite et sa restauration testée. Aucun compte ni donnée de démonstration. Et la commande inps sante affiche OK sur toutes les lignes.

Scène 10 · Diapositive · *Fin de la formation*

> C'est la fin de cette formation. Le document PDF reprend toutes les explications, avec les extraits de code et leurs numéros de lignes, ainsi que la liste complète des routes, des permissions et des tables. Retenez la méthode : pour comprendre un écran, partez de son adresse, puis suivez la route, le

> contrôleur, le service, le modèle et la vue. Et pour toute modification : une copie de travail, Git, et les tests. Bon travail sur INPS Formation.